TUGAS 2

Pemrosesan Citra dalam Ranah Frekuensi

IF4073 – PEMROSESAN CITRA DIGITAL



Disusun oleh:

Muhammad Zahran Ramadhan Ardiana	13523104
Ahmad Evander Ruizhi Xavier	18223064

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR TABEL **5**
DAFTAR GAMBAR **7**
PERNYATAAN **8**
BAB I
PENDAHULUAN **9**
1.1 Deskripsi Singkat Program 9
1.2 Cakupan Metode dan Alur Komputasi Utama 9
1.3 Antarmuka dan Fitur Utama GUI 10
1.4 Struktur Organisasi Repositori 11
1.5 Dependensi Perangkat Lunak 12
1.6 Pengelompokan Data Uji 12
BAB II
TEORI SINGKAT **14**
2.1 Pemrosesan Citra dalam Ranah Frekuensi 14
2.2 Low-Pass Filters 15
2.2.1 Ideal Low-Pass Filter (ILPF) 15
2.2.2 Butterworth Low-Pass Filter (BLPF) 16
2.2.3 Gaussian Low-Pass Filter (GLPF) 17
2.3 High-Pass Filters 17
2.3.1 Ideal High-Pass Filter (IHPF) 18
2.3.2 Butterworth High-Pass Filter (BHPF) 18
2.3.3 Gaussian High-Pass Filter (GHPF) 19
2.4 Band-Reject Filters 19
2.4.1 Ideal Band-Reject Filter (IBRF) 20
2.4.2 Butterworth Band-Reject Filter (BBRF) 20
2.4.3 Gaussian Band-Reject Filter (GBRF) 21
2.5 Band-Pass Filters 21
2.6 Notch Filter 22
2.6.1 Ideal Line Notch Filter 23
2.6.2 Butterworth Line Notch Filter 23
2.6.3 Operasi Komplementer dan Pemutus Lingkaran (Break Circle) 23

2.7 Pencerahan Citra dalam Ranah Frekuensi 24
2.8 Restorasi Motion Blur 25
2.8.1 Point Spread Function (PSF) Motion Blur 25
2.8.2 Inverse Filtering 26
2.8.3 Wiener Filtering 27
2.9 Metrik Evaluasi Kualitas Citra 27
2.9.1 Mean Squared Error (MSE) 27
2.9.2 Peak Signal-to-Noise Ratio (PSNR) 28
BAB III
PERANCANGAN DAN IMPLEMENTASI 30
3.1 Struktur Program dan Alur Penggunaan 30
3.1.1 Komponen Struktur Program 30
3.1.1.1 Panel Kontrol & Analisis (Kiri) 32
3.1.1.2 Panel Visualisasi Citra (Kanan) 33
3.1.1.3 Panel Filter H (Kanan) 34
3.1.1.4 Panel Noise Peaks (Kanan) 35
3.1.2 Alur Penggunaan GUI (Workflow) 35
3.2 Implementasi Pass Filter 37
3.2.1 Ideal Pass Filter 37
3.2.2 Butterworth Pass Filter 39
3.2.3 Gaussian Pass Filter 40
3.3 Implementasi Band Filter 41
3.3.1 Ideal Band Filter 41
3.3.2 Butterworth Band Filter 43
3.3.3 Gaussian Band Filter 44
3.4 Implementasi Pencerahan Citra dalam Ranah Frekuensi 45
3.5 Implementasi Notch Filter 47
3.6 Implementasi Fungsi Restorasi Motion Blur 52
3.6.1 Pembentukan PSF Motion Blur 52
3.6.2 Inverse Filtering 53
3.6.3 Wiener Filtering 55
3.7 Implementasi Fungsi Pembantu 56
3.7.1 Grid Frekuensi dan Padding 56

3.7.2 Penerapan Filter dan Titik Masuk Metode..... 57
3.7.3 Penyusun Filter Gabungan (Notch Filter)..... 58
3.7.4 Deteksi Noise Peaks..... 60
3.7.5 Tuning Parameter dengan Golden-Section Search..... 61
BAB IV
PENGUJIAN PROGRAM DAN ANALISIS HASIL..... 64
4.1 Pengujian Image Smoothing / Blurring dalam Ranah Frekuensi..... 64
4.1.1 Identifikasi Citra Masukkan..... 64
4.1.2 Pemilihan Metode Pemrosesan..... 65
4.1.3 Analisis Hasil Pemrosesan..... 67
4.1.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji..... 69
4.2 Pengujian High-Pass Filtering pada Ranah Frekuensi..... 75
4.2.1 Identifikasi Citra Masukkan..... 75
4.2.2 Pemilihan Metode Pemrosesan..... 76
4.2.3 Analisis Hasil Pemrosesan..... 78
4.2.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji..... 80
4.3 Pengujian Peningkatan Kecerahan Citra pada Ranah Frekuensi..... 85
4.3.1 Identifikasi Citra Masukkan..... 85
4.3.2 Pemilihan Metode Pemrosesan..... 86
4.3.3 Analisis Hasil Pemrosesan..... 87
4.3.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji..... 89
4.4 Pengujian Restorasi Derau Periodik..... 94
4.4.1 Identifikasi Citra Terdegradasi..... 94
4.4.2 Pemilihan Metode Restorasi..... 95
4.4.3 Analisis Hasil Restorasi..... 97
4.4.4 Rekapitulasi Restorasi Keseluruhan Citra Uji..... 99
4.5 Pengujian Restorasi Motion Blur..... 105
4.5.1 Identifikasi Citra Terdegradasi..... 105
4.5.2 Pemilihan Metode Restorasi..... 106
4.5.3 Analisis Hasil Restorasi..... 107
4.5.4 Rekapitulasi Restorasi Keseluruhan Citra Uji..... 109
BAB V
KESIMPULAN..... 120

DAFTAR PUSTAKA.....122
LAMPIRAN.....123

DAFTAR TABEL

Tabel 3.1 Rentang pencarian parameter pada golden-section search..... 62
Tabel 4.1 Analisis awal citra masukan astronaut_color_gaussian_noise.png..... 64
Tabel 4.2 Metode pemrosesan yang diuji untuk image smoothing..... 65
Tabel 4.3 Analisis hasil image smoothing..... 67
Tabel 4.4 Analisis awal keseluruhan citra masukan untuk image smoothing..... 69
Tabel 4.5 Konfigurasi pemrosesan image smoothing setiap kasus uji..... 71
Tabel 4.6 Analisis hasil image smoothing setiap kasus uji..... 73
Tabel 4.7 Analisis awal citra masukan coins_gray.png..... 75
Tabel 4.8 Metode pemrosesan yang diuji untuk high-pass filtering..... 76
Tabel 4.9 Analisis hasil high-pass filtering..... 78
Tabel 4.10 Analisis awal keseluruhan citra masukan untuk high-pass filtering..... 80
Tabel 4.11 Konfigurasi pemrosesan high-pass filtering setiap kasus uji..... 81
Tabel 4.12 Analisis hasil high-pass filtering setiap kasus uji..... 83
Tabel 4.13 Analisis awal citra masukan retina_color_dark.png..... 85
Tabel 4.14 Metode pemrosesan yang diuji untuk peningkatan kecerahan..... 87
Tabel 4.15 Analisis hasil peningkatan kecerahan..... 87
Tabel 4.16 Analisis awal keseluruhan citra masukan untuk peningkatan kecerahan.. 89
Tabel 4.17 Konfigurasi pemrosesan peningkatan kecerahan setiap kasus uji..... 91
Tabel 4.18 Analisis hasil peningkatan kecerahan setiap kasus uji..... 92
Tabel 4.19 Analisis awal citra terdegradasi hubble_color_periodic_noise.png..... 94
Tabel 4.20 Metode restorasi yang diuji untuk menghilangkan derau periodik..... 95
Tabel 4.21 Analisis hasil restorasi citra dari derau periodik..... 97
Tabel 4.22 Analisis awal keseluruhan citra terdegradasi derau periodik..... 99
Tabel 4.23 Konfigurasi restorasi derau periodik setiap kasus uji..... 100
Tabel 4.24 Analisis hasil restorasi derau periodik setiap kasus uji..... 102
Tabel 4.25 Analisis awal citra terdegradasi
astronaut_color_motion_L19_A20_gaussian_noise.png..... 105
Tabel 4.26 Metode restorasi yang diuji untuk menghilangkan motion blur..... 106
Tabel 4.27 Analisis hasil restorasi citra dari motion blur..... 107
Tabel 4.28 Analisis awal keseluruhan citra terdegradasi motion blur..... 109
Tabel 4.29 Konfigurasi restorasi motion blur setiap kasus uji..... 113

Tabel 4.30 Analisis hasil restorasi motion blur setiap kasus uji.....116

DAFTAR GAMBAR

Gambar 3.1 Tampilan Panel GUI FrequencyDomainApp.....32
Gambar 3.2 Tampilan tab Filter H pada FrequencyDomainApp.....34
Gambar 3.3 Tampilan tab Noise Peaks pada FrequencyDomainApp.....35

PERNYATAAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Muhammad Zahran Ramadhan Ardiana



Ahmad Evander Ruizhi Xavier

BAB I
PENDAHULUAN

1.1 Deskripsi Singkat Program

Aplikasi ini merupakan perangkat lunak berbasis MATLAB yang dirancang untuk melakukan pemrosesan dan restorasi citra digital pada ranah frekuensi. Seluruh fungsionalitas pemrosesan diintegrasikan ke dalam satu GUI utama bernama `FrequencyDomainApp`. Seluruh fungsi transfer $H(u, v)$ dan pemodelan degradasi dibangkitkan secara mandiri tanpa memanfaatkan fungsi bawaan penapisan atau restorasi dari MATLAB (seperti `imgaussfilt`, `imfilter`, `fspecial`, `imsharpen`, `deconvwnr`, maupun `wiener2`). Penggunaan fungsi MATLAB dibatasi secara ketat hanya untuk operasi transformasi Fourier Diskrit (`fft2`, `ifft2`, `fftshift`, `ifftshift`), manipulasi matriks dasar, serta pustaka pendukung pada skrip eksplorasi internal.

1.2 Cakupan Metode dan Alur Komputasi Utama

Program ini mengimplementasikan lima metode utama pemrosesan citra pada ranah frekuensi:

- **Low-pass Filtering / Smoothing:** Memiliki varian *Ideal Low-Pass Filter* (ILPF), *Gaussian Low-Pass Filter* (GLPF), dan *Butterworth Low-Pass Filter* (BLPF) yang tersimpan pada direktori `src/pass_filters/`.
- **High-pass Filtering:** Memiliki varian *Ideal High-Pass Filter* (IHPF), *Gaussian High-Pass Filter* (GHPF), dan *Butterworth High-Pass Filter* (BHPF) pada direktori `src/pass_filters/`.

- **Image Brightening:** Melakukan skala pada seluruh matriks spektrum menggunakan satu faktor skalar α yang diturunkan dari komponen DC pada direktori `src/brightening/`.
- **Periodic Noise Restoration:** Menyediakan penapis *notch point*, *notch line*, serta penapis *band-reject* dan *band-pass* varian Ideal, Gaussian, dan Butterworth pada direktori `src/notch_filter/`, `src/band_filters/`, dan `src/utils/`.
- **Motion Blur Restoration:** Menyediakan pemodelan PSF *motion blur*, *Inverse Filtering*, dan *Wiener Filtering* pada direktori `src/motion_blur/`.

Seluruh metode penapisan menerapkan alur komputasi yang seragam. Citra masukan di-*pad* dua kali lipat menggunakan teknik *zero-padding*, ditransformasikan via `fft2`, digeser via `fftshift`, dikalikan dengan matriks fungsi transfer $H(u, v)$, ditransformasikan balik via `ifftshift` dan `ifft2`, lalu dipotong (*crop*) kembali ke ukuran dimensi citra asli. Khusus untuk restorasi *motion blur*, pembentukan PSF $H(u, v)$ dilakukan langsung pada dimensi citra asli tanpa melalui proses *padding*. Program mendukung pemrosesan untuk citra derajat keabuan (*grayscale*) maupun citra berwarna (RGB), di mana pemrosesan citra berwarna dilakukan secara independen pada setiap saluran warna.

1.3 Antarmuka dan Fitur Utama GUI

Aplikasi menyediakan berbagai fitur interaktif pada antarmuka pengguna untuk mempermudah eksekusi dan analisis:

- Pemilihan citra uji resmi (beserta citra referensi) atau berkas citra lain lewat "Dari File...".

- Parameter filter dan model degradasi dapat diubah lewat GUI, dengan preview $H(u, v)$ pada tab Filter H sebelum diterapkan.
- Tab Utama menampilkan citra masukan, hasil, referensi, spektrum Fourier ketiganya, tabel noise peaks, serta log langkah dengan PSNR dan MSE terhadap referensi.
- Tab Noise Peaks mendeteksi peak derau periodik pada spektrum, dan hasilnya dapat diisikan ke filter notch lewat tombol "Dari Peaks".
- Filter builder untuk notch/band: beberapa filter (Ideal, Gaussian, Butterworth, Band, Notch Line) digabung dalam mode reject atau pass.
- Tombol Tune mencari parameter terbaik terhadap citra referensi dengan golden-section search (diameter dan orde, target mean, K atau threshold).
- Langkah dapat dirantai, dengan Undo (hingga 20 langkah) dan Reset.

1.4 Struktur Organisasi Repositori

Repositori program disusun secara modular ke dalam direktori berikut:

- `FrequencyDomainApp.m`: Skrip utama sebagai titik masuk (*entry point*) dan peluncur antarmuka GUI.
- `gui/`: Berisi skrip `AppState.m` untuk pengelolaan status aplikasi/komponen GUI serta `datasetIndex.m` untuk pembaca struktur citra uji.
- `src/`: Berisi titik masuk pemrosesan `applyFrequencyMethod.m`, sub-direktori fungsi utama (`pass_filters/`, `band_filters/`, `brightening/`, `notch_filter/`, `motion_blur/`), serta fungsi pembantu/utilitas `utils/` (terdiri atas `freqGrid`, `padImage2x`, `applyFrequencyFilter`, `buildFilter`, `subtractNotchPattern`, `findNoisePeaks`, `plotNoisePeaks`, serta modul

pengoptimalan parameter `tunePassFilter`, `tuneBrightness`, dan `tuneMotionRestoration`).

- `02_citra_test_images/`: Berkas citra uji resmi yang terbagi menjadi folder B hingga F.
- `TestScriptDump/`: Skrip eksplorasi dan pengujian internal selama proses pengembangan.

1.5 Dependensi Perangkat Lunak

Untuk menjalankan program ini, diperlukan lingkungan perangkat lunak dengan spesifikasi sebagai berikut:

- **MATLAB:** Versi R2025a atau lebih baru (teruji secara penuh pada versi R2026a). Batasan versi minimum ditentukan oleh penggunaan properti `Theme` pada komponen `uifigure`.
- **Image Processing Toolbox:** Membutuhkan fungsi pendukung pembacaan dan konversi citra, meliputi `imregionalmax`, `rgb2gray`, `imshow`, `im2double`, dan `im2uint8`.

1.6 Pengelompokan Data Uji

Data uji yang digunakan dalam evaluasi program tersimpan pada direktori `02_citra_test_images/` dan dikelompokkan berdasarkan bagian tugas:

- `B_smoothing_blurring_frequency`: Folder citra uji *image smoothing / blurring*.
- `C_high_pass_filtering_frequency`: Folder citra uji untuk *high-pass filtering*, tanpa citra referensi.
- `D_brightness_frequency`: Folder citra uji untuk peningkatan kecerahan.

- `E_periodic_noise_restoration`: Folder citra uji untuk restorasi derau periodik.
- `F_motion_blur_restoration`: Folder citra uji untuk restorasi *motion blur*.

Pencocokan citra referensi dilakukan secara otomatis dari sub-folder `reference/` berdasarkan kesamaan awalan nama berkas citra masukan (seperti pencocokan `brick_gray_periodic_noise.png` dengan `brick_gray.png`).

BAB II
TEORI SINGKAT

2.1 Pemrosesan Citra dalam Ranah Frekuensi

Pemrosesan citra dalam ranah frekuensi dilakukan dengan memanipulasi komponen-komponen frekuensi hasil transformasi citra. Pemrosesan ini memiliki beberapa keunggulan dibandingkan pemrosesan citra dalam ranah spasial pada umumnya, seperti reduksi derau secara efisien, penajaman tepi objek, dan peminimalan artefak visual. Transformasi citra berukuran $M \times N$ dari ranah spasial $f(x, y)$ ke ranah frekuensi $F(u, v)$ dilakukan menggunakan *Discrete Fourier Transform* dua dimensi (2D-DFT), yang dirumuskan sebagai berikut:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Variabel u dan v merepresentasikan variabel frekuensi spasial masing-masing pada arah horizontal dan vertikal. Pada koordinat asal frekuensi $(u = 0, v = 0)$, diperoleh nilai frekuensi nol yang dikenal sebagai komponen DC yaitu $F(0, 0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)$. Komponen DC ini menggambarkan jumlah atau nilai rata-rata tingkat intensitas piksel dari keseluruhan citra. Dalam visualisasi spektrum frekuensi, area di dekat pusat transformasi merepresentasikan frekuensi rendah yang memuat informasi perubahan intensitas lambat atau area homogen, sedangkan area yang semakin menjauhi titik pusat merepresentasikan frekuensi tinggi yang memuat detail halus, tepi objek, dan derau.

Proses penapisan (*filtering*) pada ranah frekuensi berlangsung secara berurutan dalam satu alur kerja yang terstruktur. Citra masukan $f(x, y)$ terlebih dahulu diubah ke ranah frekuensi menggunakan 2D-DFT untuk menghasilkan spektrum frekuensi

$F(u,v)$. Selanjutnya, spektrum frekuensi $F(u,v)$ dikalikan secara berbasis elemen dengan fungsi transfer penapis $H(u,v)$ untuk menghasilkan spektrum terfilter $G(u,v) = F(u,v) \cdot H(u,v)$. Spektrum terfilter $G(u,v)$ tersebut kemudian dikembalikan lagi ke ranah spasial untuk memperoleh citra hasil pemrosesan $g(x,y)$ menggunakan *Inverse Discrete Fourier Transform* dua dimensi (2D-IDFT), sesuai dengan persamaan berikut:

$$g(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} G(u,v) e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

Melalui alur pemrosesan ini, manipulasi terhadap komponen frekuensi spesifik dapat dilakukan secara presisi sebelum citra dikonversi kembali ke bentuk visual pada ranah spasial.

2.2 Low-Pass Filters

Low-Pass Filters atau LPF bekerja dengan cara meredam komponen frekuensi tinggi dan mempertahankan komponen frekuensi rendah pada spektrum citra. Proses peredaman frekuensi tinggi ini secara fisik menghasilkan efek buram (*blurring*) sehingga LPF sangat efektif digunakan untuk mereduksi derau berfrekuensi tinggi seperti derau Gaussian. Penentuan batas komponen frekuensi yang diloloskan atau diredam diatur oleh variabel frekuensi ambang (*cutoff frequency*) D_0 dengan mengukur jarak Euclidean $D(u,v)$ dari setiap titik frekuensi (u,v) terhadap pusat spektrum $(M/2, N/2)$ yang dirumuskan sebagai berikut:

$$D(u,v) = \sqrt{\left(u - \frac{M}{2}\right)^2 + \left(v - \frac{N}{2}\right)^2}$$

2.2.1 Ideal Low-Pass Filter (ILPF)

Ideal Low-Pass Filter (ILPF) menerapkan batas pemotongan frekuensi secara ekstrem tanpa adanya transisi. Seluruh komponen frekuensi yang berada di dalam radius D_0 dari pusat spektrum diloloskan secara utuh dengan nilai bobot 1, sedangkan seluruh frekuensi di luar radius D_0 dibuang sepenuhnya dengan nilai bobot 0. Fungsi transfer $H(u,v)$ untuk ILPF dinyatakan dalam persamaan berikut:

$$H(u,v) = \begin{cases} 1, & \text{jika } D(u,v) \leq D_0 \\ 0, & \text{jika } D(u,v) > D_0 \end{cases}$$

Meskipun sangat sederhana secara konseptual, pemotongan frekuensi yang sangat tajam pada ILPF menghasilkan artefak berupa efek *ringing* yang signifikan pada citra hasil rekonstruksi. *Ringing* muncul sebagai pola gelombang atau bayangan melingkar di sekitar tepi objek akibat timbulnya fluktuasi pada transformasi Fourier balik dari fungsi diskontinu.

2.2.2 Butterworth Low-Pass Filter (BLPF)

Butterworth Low-Pass Filter (BLPF) merancang kurva transisi yang lebih halus antara frekuensi yang diloloskan dan frekuensi yang diredam sehingga tidak memotong frekuensi secara mendadak seperti pada ILPF. Tingkat kecuraman transisi kurva pada BLPF dikendalikan oleh orde penapis n. Fungsi transfer $H(u,v)$ untuk BLPF dirumuskan sebagai berikut:

$$H(u,v) = \frac{1}{1+\left[\frac{D(u,v)}{D_0}\right]^{2n}}$$

Penggunaan kurva transisi yang bertahap ini secara efektif mampu meredam kemunculan artefak *ringing* pada citra hasil pemrosesan. Semakin tinggi nilai orde n yang digunakan, karakteristik BLPF akan semakin mendekati ILPF, sedangkan nilai orde yang lebih rendah akan menghasilkan efek transisi yang semakin mulus.

2.2.3 Gaussian Low-Pass Filter (GLPF)

Gaussian Low-Pass Filter (GLPF) menggunakan fungsi distribusi Gaussian eksponensial untuk membentuk transisi penapisan yang sangat mulus dan kontinu dari pusat frekuensi. Karakteristik transisi yang meluruh secara eksponensial ini membuat GLPF tidak menghasilkan artefak *ringing* pada ranah spasial. Fungsi transfer $H(u, v)$ untuk GLPF dirumuskan sebagai berikut:

$$H(u, v) = e^{-\frac{D^2(u,v)}{2D_0^2}}$$

Parameter D_0 bertindak sebagai deviasi standar yang menentukan lebar sebaran kurva Gaussian pada spektrum frekuensi. Karena sifatnya yang bebas dari efek *ringing* dan mampu menghasilkan efek penghalusan yang alami, GLPF menjadi pilihan penapis yang baik untuk pemrosesan citra medis dan restorasi kualitas visual.

2.3 High-Pass Filters

High-Pass Filters atau HPF bekerja dengan cara meredam komponen frekuensi rendah dan meloloskan komponen frekuensi tinggi pada spektrum citra. Karena komponen frekuensi rendah menggambarkan area homogen dengan variasi intensitas yang lambat, peredaman komponen ini menyebabkan area latar belakang yang seragam menjadi gelap sementara area dengan perubahan intensitas tajam seperti tepi objek dan detail halus diperkuat. Oleh karena itu, penapisan HPF utamanya digunakan untuk penajaman citra (*image sharpening*) dan penonjolan struktur tepi (*edge detection*). Penentuan batas frekuensi yang diloloskan atau diredam tetap dikendalikan oleh parameter *cutoff frequency* D_0 berbasis jarak Euclidean $D(u, v)$ terhadap pusat spektrum. Secara umum, fungsi

transfer HPF (H_{hp}) memiliki hubungan terbalik dengan LPF (H_{lp}) yang dinyatakan pada rumus berikut:

$$H_{hp}(u, v) = 1 - H_{lp}(u, v).$$

2.3.1 Ideal High-Pass Filter (IHPF)

Ideal High-Pass Filter (IHPF) menerapkan batas pemotongan frekuensi secara ekstrem tanpa daerah transisi. Seluruh komponen frekuensi rendah yang berada di dalam radius D_0 dari pusat spektrum dibuang sepenuhnya dengan nilai bobot 0, sedangkan seluruh komponen frekuensi tinggi di luar radius D_0 diloloskan secara utuh dengan nilai bobot 1. Fungsi transfer $H(u, v)$ untuk IHPF dinyatakan dalam persamaan berikut:

$$H(u,v) = \begin{cases} 0, & \text{jika } D(u,v) \leq D_0 \\ 1, & \text{jika } D(u,v) > D_0 \end{cases}$$

Meskipun IHPF mampu menajamkan batas tepi objek secara sangat jelas, sifat pemotongan fungsi yang diskontinu pada batas D_0 menyebabkan timbulnya artefak *ringing* yang kuat pada ranah spasial. *Ringing* ini mewujud sebagai fluktuasi gelombang osilasi di sekitar garis tepi objek sehingga dapat mengurangi kejelasan visual citra hasil rekonstruksi.

2.3.2 Butterworth High-Pass Filter (BHPF)

Butterworth High-Pass Filter (BHPF) menyediakan kurva transisi yang bertahap dan melandai antara daerah peredaman frekuensi rendah dan pelolosan frekuensi tinggi. Kemiringan kurva transisi ini diatur oleh nilai orde penapis n. Fungsi transfer $H(u, v)$ untuk BHPF dirumuskan sebagai berikut:

$$H(u,v) = \frac{1}{1+\left[\frac{D_0}{D(u,v)}\right]^{2n}}$$

Dengan adanya kurva transisi kontinu, BHPF mampu meredam kemunculan artefak *ringing* jika dibandingkan dengan IHPF. Pengaturan orde n memberikan fleksibilitas; orde yang lebih rendah menghasilkan transisi yang lebih halus untuk meminimalkan artefak, sedangkan orde yang lebih tinggi akan memperkuat penajaman tepi namun mendekati sifat pemotongan tajam seperti IHPF.

2.3.3 Gaussian High-Pass Filter (GHPF)

Gaussian High-Pass Filter (GHPF) menggunakan fungsi eksponensial Gaussian untuk membentuk kurva pelolosan frekuensi tinggi yang sangat mulus dari pusat spektrum. Sifat matematis kurva Gaussian yang kontinu dan turunannya yang tidak memiliki titik diskontinuitas membuat penapis ini tidak menimbulkan artefak *ringing* pada ranah spasial. Fungsi transfer $H(u,v)$ untuk GHPF dirumuskan sebagai berikut:

$$H(u,v) = 1 - e^{-\frac{D^2(u,v)}{2D_0^2}}$$

Parameter D_0 menentukan tingkat kelebaran kurva penapisan Gaussian pada ranah frekuensi. GHPF mampu menghasilkan efek penajaman tepi dan detail halus yang alami tanpa memunculkan fluktuasi sinyal gelombang yang merusak garis batas objek sehingga menjadi metode penajaman frekuensi tinggi yang ideal untuk pemrosesan citra presisi tinggi.

2.4 Band-Reject Filters

Band-Reject Filters atau BRF digunakan untuk mereduksi atau mengeliminasi derau periodik pada citra apabila lokasi umum komponen derau dalam ranah frekuensi telah diketahui atau dapat diestimasi. Derau periodik umumnya muncul dalam bentuk konsentrasi energi atau pasangan titik terang (*spikes*) yang tersusun simetris

pada spektrum Fourier. Penapis BRF bekerja dengan cara memblokir atau meredam komponen frekuensi yang berada di dalam pita frekuensi tertentu dengan lebar pita W di sekitar jari-jari pusat pita D_0, serta meloloskan komponen frekuensi di luar pita tersebut tanpa mengalami redaman. Jarak frekuensi $D(u,v)$ dihitung dari koordinat (u,v) ke titik pusat spektrum sebagaimana telah didefinisikan pada Sub-bab 2.2.

2.4.1 Ideal Band-Reject Filter (IBRF)

Ideal Band-Reject Filter (IBRF) memotong komponen frekuensi di dalam pita penapisan secara tajam dan diskontinu tanpa adanya daerah transisi. Seluruh komponen frekuensi yang berada dalam rentang jari-jari $[D_0 - \frac{W}{2}, D_0 + \frac{W}{2}]$ diblokir secara penuh dengan nilai fungsi transfer 0, sedangkan frekuensi di luar rentang tersebut diloloskan secara utuh dengan nilai 1. Fungsi transfer $H(u,v)$ untuk IBRF dirumuskan sebagai berikut:

$$H(u,v) = \begin{cases} 1, & \text{jika } D(u,v) < D_0 - \frac{W}{2} \\ 0, & \text{jika } D_0 - \frac{W}{2} \leq D(u,v) \leq D_0 + \frac{W}{2} \\ 1, & \text{jika } D(u,v) > D_0 + \frac{W}{2} \end{cases}$$

Karakteristik pemotongan yang sangat ekstrem ini membuat IBRF sangat responsif dalam membuang pita derau, namun diskontinuitas pada batas pita memicu timbulnya artefak *ringing* atau fluktuasi gelombang osilasi pada citra hasil rekonstruksi.

2.4.2 Butterworth Band-Reject Filter (BBRF)

Butterworth Band-Reject Filter (BBRF) merancang kurva penapisan dengan transisi yang lebih bertahap antara daerah frekuensi yang diloloskan dan diredam. Pada BBRF, frekuensi di titik tengah pita D_0 diblokir secara penuh, sedangkan frekuensi di sekitar tepi pita dilemahkan secara berangsur. Tingkat kecuraman transisi kurva

diatur oleh orde penapis n, di mana nilai n yang rendah menghasilkan transisi mendekati penapis Gaussian dan nilai n yang tinggi mendekati penapis Ideal. Fungsi transfer $H(u, v)$ untuk BBRF dengan orde n dirumuskan sebagai berikut:

$$H(u,v) = \frac{1}{1+\left[\frac{D(u,v)\cdot W}{D^2(u,v)-D_0^2}\right]^{2n}}$$

Ketiadaan diskontinuitas tajam pada kurva BBRF secara efektif meminimalkan kemunculan artefak *ringing* pada ranah spasial dibandingkan dengan IBRF.

2.4.3 Gaussian Band-Reject Filter (GBRF)

Gaussian Band-Reject Filter (GBRF) menggunakan fungsi eksponensial berbasis distribusi Gaussian untuk membentuk daerah transisi penapisan yang sangat mulus dan kontinu. Perubahan nilai fungsi transfer dari daerah pelolosan menuju daerah pemblokiran terjadi secara lancar tanpa adanya lonjakan diskontinu. Fungsi transfer $H(u, v)$ untuk GBRF dirumuskan sebagai berikut:

$$H(u,v) = 1 - e^{-\frac{1}{2}\left[\frac{D^2(u,v)-D_0^2}{D(u,v)\cdot W}\right]^2}$$

Sifat kurva Gaussian yang kontinu dan memiliki turunan yang mulus menjadikan GBRF sama sekali tidak menghasilkan artefak *ringing* pada citra hasil restorasi, sehingga sangat ideal untuk mengeliminasi derau periodik tanpa merusak kualitas estetika citra.

2.5 Band-Pass Filters

Band-Pass Filters atau BPF melakukan operasi yang berlawanan dengan penapis penolak pita (BRF). Penapis BPF bekerja dengan cara mengisolasi dan meloloskan hanya komponen frekuensi yang berada di dalam rentang pita tertentu dengan

lebar pita W di sekitar jari-jari pusat pita D_0, serta meredam seluruh komponen frekuensi di luar pita tersebut. Karena sifat operasinya yang merupakan kebalikan dari BRF, fungsi transfer BPF dapat diperoleh secara langsung melalui hubungan komplementer berikut:

$$H_{bp}(u, v) = 1 - H_{br}(u, v)$$

Implementasi BPF diturunkan dari ketiga varian BRF yang telah didefinisikan pada Sub-bab 2.4, yaitu *Ideal Band-Pass Filter* (IBPF), *Butterworth Band-Pass Filter* (BBPF), dan *Gaussian Band-Pass Filter* (GBPF). Pada IBPF, fungsi transfer bernilai 1 hanya untuk frekuensi di dalam pita $[D_0 - \frac{W}{2}, D_0 + \frac{W}{2}]$ dan bernilai 0 di luar pita tersebut. Sementara itu, pada BBPF dan GBPF, kurva pelolosan frekuensi di dalam pita membentuk daerah transisi yang melandai dan mulus sesuai persamaan komplementer dari BBRF dan GBRF. Penapis BPF sangat efektif digunakan untuk mengisolasi pola derau atau tekstur tertentu dari citra agar dapat dianalisis secara terpisah.

2.6 Notch Filter

Notch Filters bekerja secara selektif dengan meredam atau meloloskan komponen frekuensi pada daerah spesifik di dalam spektrum Fourier. Dalam laporan ini, jenis penapis notch yang diimplementasikan adalah penapis notch berbentuk garis (*Line Notch Filter*). Penapis ini dirancang khusus untuk mengeliminasi derau pemindaian (*scanline noise*) atau pola garis periodik yang membentang sepanjang sumbu horizontal atau vertikal spektrum. Penapis ini menghitung jarak tegak lurus D dari setiap koordinat frekuensi terhadap garis posisi notch P (baik vertikal $D = |U - P|$ maupun horizontal $D = |V - P|$), dengan setengah lebar garis dinotasikan sebagai $D_0 = \frac{W}{2}$.

2.6.1 Ideal Line Notch Filter

Ideal Line Notch Filter menerapkan batas pemotongan frekuensi secara tegas (*hard-edge*) tanpa adanya daerah transisi di sepanjang garis notch. Pada mode penolakan (*Notch-Reject*), seluruh komponen frekuensi yang berada di dalam rentang jarak $D \leq D_0$ dibuang sepenuhnya dengan nilai fungsi transfer 0, sedangkan frekuensi di luar garis diloloskan dengan nilai 1. Fungsi transfer $H(u, v)$ untuk *Ideal Line Notch-Reject Filter* dirumuskan sebagai berikut:

$$H(u,v) = \begin{cases} 0, & \text{jika } D \leq D_0 \\ 1, & \text{jika } D > D_0 \end{cases}$$

Sifat pemotongan diskontinu ini sangat responsif memotong garis derau, namun berisiko memicu timbulnya artefak *ringing* pada citra hasil restorasi.

2.6.2 Butterworth Line Notch Filter

Butterworth Line Notch Filter menyediakan kurva transisi yang melandai di sekitar tepi garis notch untuk meminimalkan kemunculan artefak *ringing*. Tingkat kelandaian transisi dikendalikan oleh orde penapis n, yang ditentukan berdasarkan parameter *blur*. Fungsi transfer $H(u, v)$ untuk *Butterworth Line Notch-Reject Filter* dirumuskan sebagai berikut:

$$H(u,v) = 1 - \frac{1}{1+\left[\frac{D}{D_0}\right]^{2n}}$$

Nilai fungsi transfer akan bernilai 0 tepat pada garis notch dan secara bertahap bergerak melandai mendekati 1 seiring bertambahnya jarak D dari garis pusat.

2.6.3 Operasi Komplementer dan Pemutus Lingkaran (Break Circle)

Penapis notch meloloskan garis (*Line Notch-Pass Filter*) melakukan fungsi sebaliknya dari penapis penolak garis, yaitu hanya meloloskan komponen frekuensi

di sepanjang garis notch dan meredam frekuensi lainnya. Fungsi transfer *Notch-Pass* diperoleh melalui hubungan komplementer sederhana:

$$H_{NPF}(u,v) = 1 - H_{NRF}(u,v)$$

Selain itu, untuk mencegah terbuangnya komponen frekuensi utama di titik pusat spektrum akibat pemotongan garis yang melintas penuh, diimplementasikan fitur pemutus berbentuk lingkaran (*break circle*) ber-radius R_c pada posisi (u_c, v_c). Jarak koordinat terhadap pusat pemutus dihitung menggunakan $D_{circle} = \sqrt{(U - u_c)^2 + (V - v_c)^2}$, sehingga nilai fungsi transfer di dalam radius $D_{circle} \leq R_c$ dikembalikan ke nilai dasar agar informasi frekuensi penting tetap terjaga.

2.7 Pencerahan Citra dalam Ranah Frekuensi

Pencerahan citra pada ranah frekuensi memanfaatkan sifat dasar linearitas Transformasi Fourier Diskrit (DFT 2D) serta karakteristik komponen DC. Komponen DC yang terletak pada koordinat titik asal spektrum Fourier $F(0,0)$ merepresentasikan total jumlah intensitas seluruh piksel pada citra ranah spasial. Nilai rata-rata intensitas citra $\bar{f}$ untuk citra berukuran $M \times N$ berhubungan langsung dengan komponen $F(0,0)$ melalui persamaan berikut:

$$\bar{f} = \frac{1}{M \cdot N} F(0,0)$$

Berdasarkan sifat linearitas DFT 2D, perkalian seluruh spektrum frekuensi $F(u,v)$ dengan sebuah faktor skalar α akan menghasilkan efek perubahan intensitas yang sebanding pada ranah spasial $g(x,y) = \alpha \cdot f(x,y)$. Operasi pengangkatan kecerahan citra dapat dirumuskan melalui perkalian spektrum sebagai berikut:

$$G(u,v) = \alpha \cdot F(u,v)$$

Nilai faktor skalar global α dihitung berdasarkan rasio antara target rata-rata kecerahan yang diinginkan $\bar{f}_{target}$ dengan rata-rata kecerahan awal citra $\bar{f}_{awal}$, yaitu $\alpha = \frac{\bar{f}_{target}}{\bar{f}_{awal}}$. Untuk citra berwarna (RGB), nilai α dihitung secara menyeluruh dari akumulasi komponen DC ketiga saluran warna. Penggunaan nilai α tunggal yang seragam pada seluruh saluran warna memastikan bahwa rasio antar-komponen RGB tidak mengalami distorsi, sehingga pencerahan citra terjadi secara proporsional tanpa mengubah persepsi warna asli.

2.8 Restorasi Motion Blur

Motion blur merupakan salah satu bentuk degradasi citra yang terjadi akibat adanya pergerakan relatif antara kamera dan objek selama proses pembentukan citra (*sensor exposure time*). Pergerakan ini dapat disebabkan oleh getaran kamera maupun objek bergerak cepat pada adegan. Proses restorasi citra bertujuan untuk mengestimasi kembali citra asli tanpa cacat $f(x, y)$ dari citra terdegradasi $g(x, y)$ dengan memodelkan fungsi transfer degradasi atau *Point Spread Function* (PSF) $H(u, v)$ serta menerapkan teknik penapisan ranah frekuensi yang sesuai.

2.8.1 Point Spread Function (PSF) Motion Blur

Fungsi sebaran titik atau *Point Spread Function* (PSF) memodelkan efek pemburaman akibat pergeseran linier sejauh pergerakan a pada sumbu horizontal dan b pada sumbu vertikal selama durasi pencahayaan T. Dalam koordinat polar, pergeseran tersebut dinyatakan oleh panjang pergerakan piksel L dan sudut arah pergerakan θ, di mana $a = L\cos\theta$ dan $b = L\sin\theta$. Berdasarkan pemodelan integrasi pergerakan kontinu pada ranah frekuensi, fungsi transfer degradasi $H(u, v)$ dirumuskan sebagai berikut:

$$H(u,v) = \frac{T}{\pi(ua+vb)}\sin[\pi(ua + vb)]e^{-j\pi(ua+vb)}$$

Persamaan tersebut mengandung fungsi sinc yang merepresentasikan peredaman amplitudo spektrum secara sinusoidal, serta faktor fase eksponensial $e^{-j\pi(ua+vb)}$ yang merepresentasikan pergeseran posisi spasial. Pada implementasi praktis dengan matriks kernel simetris terpusat, faktor fase eksponensial dapat disederhanakan sehingga fungsi transfer $H(u,v)$ murni dibentuk oleh kurva sinc ber-nilai riil untuk mencegah terjadinya pergeseran posisi piksel (*pixel displacement*) pada citra hasil rekonstruksi.

2.8.2 Inverse Filtering

Inverse Filtering merupakan pendekatan restorasi paling mendasar yang mengestimasi citra asli dengan membagi spektrum Fourier citra terdegradasi $G(u,v)$ secara langsung dengan fungsi transfer degradasi $H(u,v)$. Apabila citra tidak dipengaruhi oleh derau aditif, spektrum citra hasil rekonstruksi $\hat{F}(u,v)$ dirumuskan sebagai berikut:

$$\hat{F}(u,v) = \frac{G(u,v)}{H(u,v)}$$

Meskipun sederhana, penapisan terbalik sangat rentan terhadap kehadiran derau aditif $N(u,v)$. Jika citra terdegradasi mengandung derau $G(u,v) = H(u,v)F(u,v) + N(u,v)$, maka persamaan rekonstruksi menjadi $\hat{F}(u,v) = F(u,v) + \frac{N(u,v)}{H(u,v)}$. Pada titik-titik frekuensi di mana fungsi $H(u,v)$ bernilai nol atau sangat mendekati nol (seperti pada titik perpotongan nol fungsi sinc *motion blur*), komponen $\frac{N(u,v)}{H(u,v)}$ akan membesar secara ekstrem, sehingga derau mendominasi citra hasil restorasi dan menimbulkan artefak visual yang merusak.

2.8.3 Wiener Filtering

Wiener Filtering mengatasi kelemahan utama penapisan terbalik dengan memperhitungkan karakteristik statistik derau aditif dan sinyal citra. Penapis ini merancang fungsi transfer restorasi yang meminimalkan kesalahan kuadrat rata-rata antara citra asli $f(x, y)$ dan citra hasil estimasi $\hat{f}(x, y)$. Estimasi spektrum citra hasil restorasi $\hat{F}(u, v)$ dirumuskan sebagai berikut:

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + K} \right] G(u,v)$$

Parameter K merepresentasikan rasio daya derau terhadap daya sinyal citra asli atau *Noise-to-Signal Ratio* (NSR). Kehadiran faktor peredam K pada penyebut mencegah terjadinya pembagian dengan nilai nol saat $|H(u, v)|$ mendekati nol, sehingga amplifikasi derau dapat diredam dengan efektif. Jika tidak terdapat derau $(K = 0)$, persamaan penapis Wiener secara otomatis kembali menjadi persamaan penapis terbalik biasa.

2.9 Metrik Evaluasi Kualitas Citra

Pengujian kualitas hasil pemrosesan dan restorasi citra memerlukan metrik kuantitatif objektif untuk mengukur tingkat kemiripan atau derajat distorsi antara citra terproses dengan citra acuan. Apabila citra referensi asli (*ground truth*) tersedia, metrik *Mean Squared Error* (MSE) dan *Peak Signal-to-Noise Ratio* (PSNR) dapat digunakan untuk membandingkan efektivitas serta tingkat akurasi dari berbagai metode penapisan maupun algoritma restorasi yang diuji.

2.9.1 Mean Squared Error (MSE)

Mean Squared Error (MSE) mengukur nilai rata-rata dari kuadrat selisih intensitas piksel antara citra referensi asli $f(x, y)$ dan citra hasil pemrosesan atau restorasi $\hat{f}(x, y)$. Metrik ini memberikan gambaran kuantitatif mengenai akumulasi besarnya kesalahan (*error*) atau distorsi visual yang dihasilkan oleh suatu algoritma. Untuk citra berukuran $M \times N$ piksel, nilai MSE dirumuskan secara matematis sebagai berikut:

$$MSE = \frac{1}{M \cdot N} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[f(x, y) - \hat{f}(x, y) \right]^2$$

Dalam evaluasi performa restorasi citra, nilai MSE yang semakin mendekati nol mengindikasikan bahwa perbedaan antara citra hasil pemrosesan dan citra referensi semakin kecil, yang menandakan tingkat akurasi rekonstruksi yang semakin tinggi. Pada citra berwarna (RGB), nilai MSE dihitung dengan mengakumulasikan rata-rata kesalahan pada ketiga saluran warna (*red*, *green*, dan *blue*).

2.9.2 Peak Signal-to-Noise Ratio (PSNR)

Peak Signal-to-Noise Ratio (PSNR) merupakan metrik pengujian yang mengukur rasio antara nilai daya maksimum dari sinyal citra terhadap daya derau distorsinya yang direpresentasikan oleh nilai MSE. Nilai PSNR dinyatakan dalam satuan desibel (dB) dan dirumuskan sebagai berikut:

$$PSNR = 10 \cdot \log_{10}\left(\frac{MAX_I^2}{MSE}\right)$$

Parameter MAX_I merepresentasikan nilai intensitas piksel maksimum yang mungkin dicapai oleh citra, yaitu bernilai 255 untuk format citra berkedalaman 8-bit per saluran warna. Karena nilai PSNR berbanding terbalik dengan MSE, nilai PSNR yang

semakin tinggi mengindikasikan kualitas citra hasil pemrosesan yang semakin baik dan semakin mendekati citra aslinya. Secara visual, nilai PSNR yang tinggi menandakan bahwa distorsi atau derau sisa pada citra hasil restorasi semakin sulit terdeteksi oleh indra penglihatan manusia.

BAB III
PERANCANGAN DAN IMPLEMENTASI

3.1 Struktur Program dan Alur Penggunaan

Program aplikasi perbaikan kualitas citra ini dirancang menggunakan arsitektur berbasis fungsi MATLAB GUI dengan satu jendela utama (`FrequencyDomainApp`) yang terbagi secara sistematis ke dalam dua area panel utama, yaitu Panel Kontrol & Analisis di sebelah kiri dan Panel Visualisasi Citra di sebelah kanan.

3.1.1 Komponen Struktur Program

Program memiliki struktur sebagai berikut:

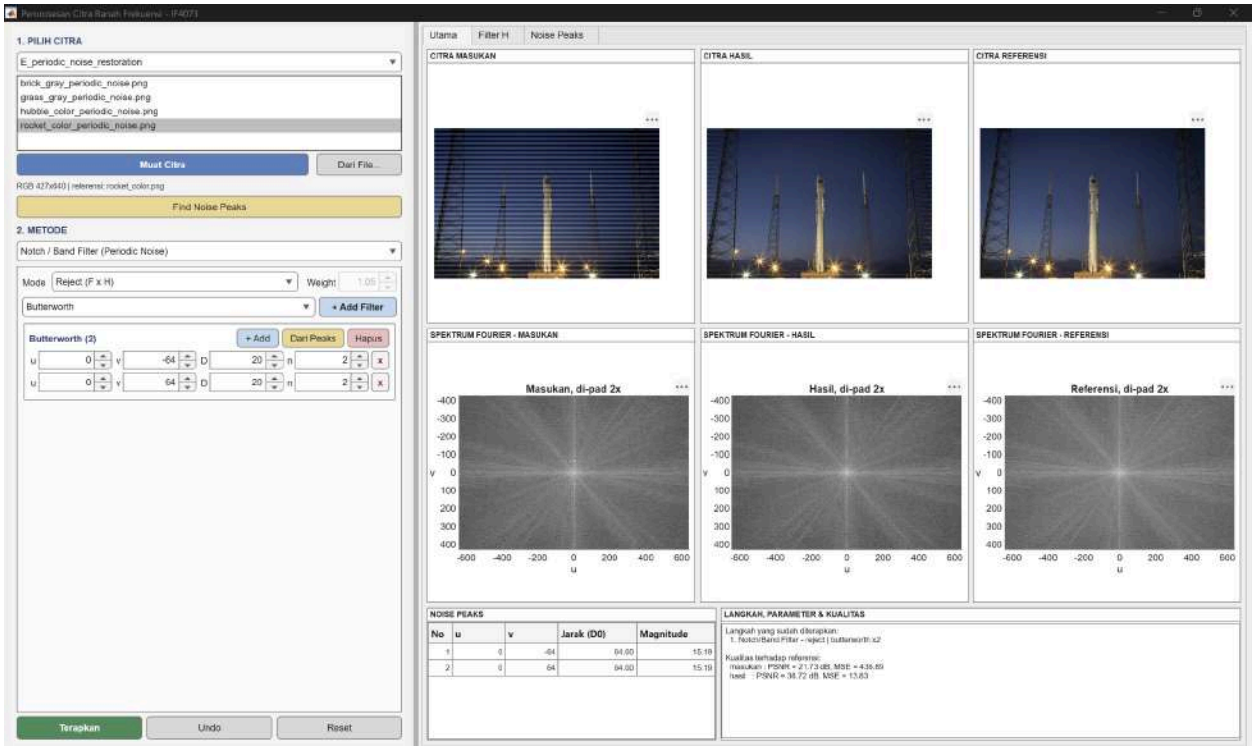
```
tugas2-citra-frequency-domain-processing/
│    .gitignore
│    FrequencyDomainApp.m
│
├──02_citra_test_images
│   │    README.md
│   │
│   ├──B_smoothing_blurring_frequency/
│   │
│   ├──C_high_pass_filtering_frequency/
│   │
│   ├──D_brightness_frequency/
│   │
│   ├──E_periodic_noise_restoration/
│   │
│   └──F_motion_blur_restoration/
│
├──gui
│       AppState.m
│       datasetIndex.m
│
├──src
│   │    applyFrequencyMethod.m
│   │
│   ├──band_filters/
│   │       bandButterworth.m
│   │       bandGaussian.m
│   │       bandIdeal.m
│   │
│   ├──brightening/
│   │       brightenFrequency.m
│   │
│   ├──motion_blur/
│   │       inverseFilter.m
│   │       motionBlurPSF.m
```

```
│   │       wienerFilter.m
│   │
│   ├──notch_filter/
│   │       notchLineFilter.m
│   │
│   ├──pass_filters/
│   │       butterworthPass.m
│   │       gaussianPass.m
│   │       idealPass.m
│   │
│   └──utils/
│           applyFrequencyFilter.m
│           buildFilter.m
│           findNoisePeaks.m
│           freqGrid.m
│           padImage2x.m
│           plotNoisePeaks.m
│           subtractNotchPattern.m
│           tuneBrightness.m
│           tuneMotionRestoration.m
│           tunePassFilter.m
```

Folder 02_citra_test_images/ berisi citra uji resmi yang dikelompokkan per bagian tugas (B sampai F). Setiap kelompok memiliki subfolder input/ dan, kecuali kelompok C, subfolder reference/.

Folder src/ berisi inti algoritma yang seluruhnya dibuat sendiri, dibagi menjadi enam kelompok yaitu pass_filters/ (ILPF/IHPF, GLPF/GHPF, BLPF/BHPF), band_filters/ (band-reject dan band-pass Ideal, Gaussian, Butterworth), brightening/ (peningkatan kecerahan), notch_filter/ (notch berbentuk garis), motion_blur/ (PSF, inverse filtering, Wiener filtering), dan utils/ (fungsi pembantu). Berkas applyFrequencyMethod.m menjadi titik masuk yang memanggil metode sesuai pilihan di GUI.

Folder gui/ berisi AppState.m, kelas handle penyimpan state dan komponen GUI, serta datasetIndex.m yang membaca struktur folder citra uji beserta daftar berkas input dan referensinya.



Gambar 3.1 Tampilan Panel GUI FrequencyDomainApp

3.1.1.1 Panel Kontrol & Analisis (Kiri)

1. Sub-panel Pilih Citra: dropdown kategori (FolderBox) berisi subfolder dataset, daftar berkas input (ImageBox), tombol "Muat Citra" yang memuat citra terpilih beserta referensinya, dan tombol "Dari File..." untuk citra di luar dataset (tanpa referensi). Label informasi menampilkan jenis citra, ukuran, dan nama referensi. Tombol "Find Noise Peaks" membuka tab Noise Peaks dan aktif setelah citra dimuat.
2. Sub-panel Metode dan Parameter: dropdown metode berisi lima pilihan, yaitu Lowpass Filter (Smoothing), Highpass Filter, Brightness, Notch / Band Filter (Periodic Noise), dan Motion Blur Restoration. Panel parameter bersifat dinamis, hanya panel metode terpilih yang tampil:

- Lowpass dan Highpass: bentuk filter (Ideal, Gaussian, Butterworth), diameter, orde n (aktif hanya untuk Butterworth), epoch tuning dan tombol Tune.
- Brightness: target mean, epoch tuning dan tombol Tune.
- Notch / Band: mode gabungan (Reject atau Pass), weight, dan filter builder. Pengguna memilih tipe (Ideal, Gaussian, Butterworth, Band Ideal, Band Gaussian, Band Butterworth, Notch Line) lalu menambah card; tiap card memiliki baris filter dengan tombol "+ Add", "Dari Peaks", dan "Hapus".
- Motion Blur: jenis restorasi (Wiener atau Inverse), panjang L, sudut theta (terisi otomatis dari nama berkas), K (NSR) atau threshold |H|, serta Tune.

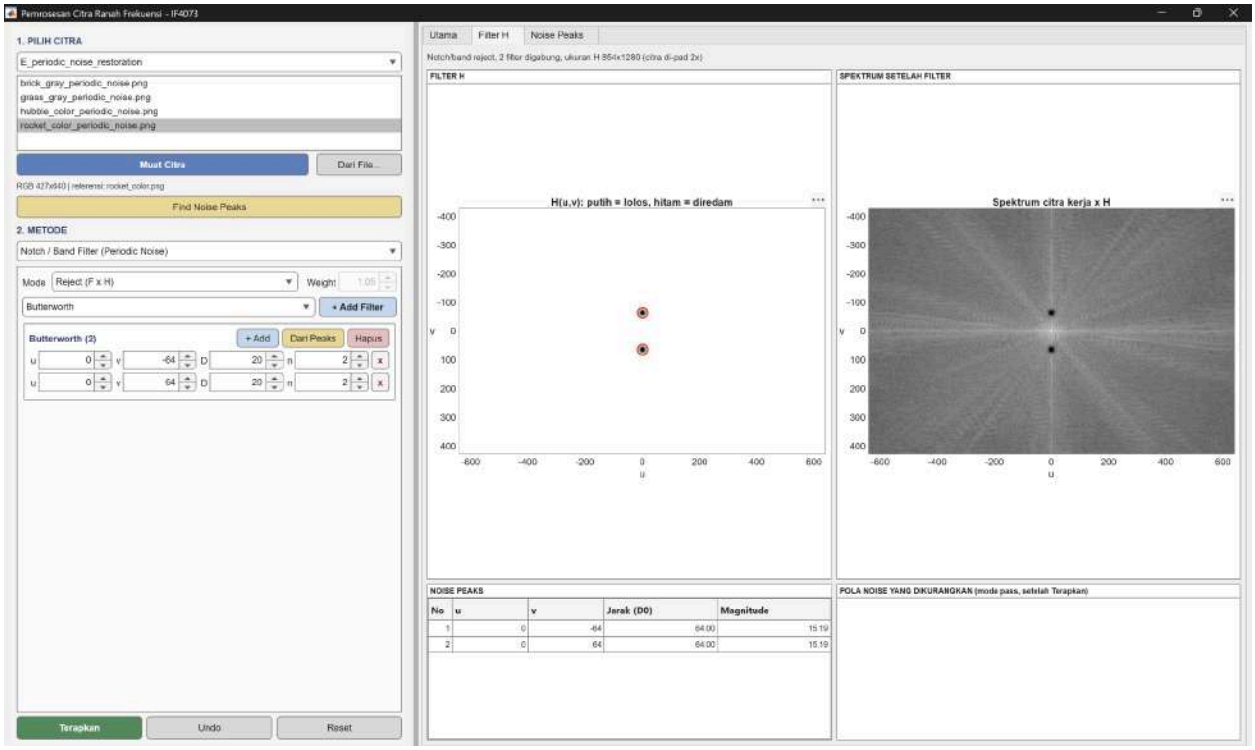
3. Tombol aksi: "Terapkan" menjalankan metode pada hasil terakhir (atau citra masukan jika belum ada hasil) sehingga langkah dapat dirantai, "Undo" membatalkan langkah terakhir (hingga 20 langkah), dan "Reset" mengembalikan ke citra masukan.

3.1.1.2 Panel Visualisasi Citra (Kanan)

Panel kanan berupa tiga tab. Tab Utama berisi baris pertama citra masukan, citra hasil, dan citra referensi. Baris kedua spektrum Fourier ketiganya (magnitude skala log dari citra yang di-pad 2x, sumbu u dan v) serta baris ketiga tabel noise peaks (No, u, v, jarak, magnitude) dan kotak teks "Langkah, Parameter & Kualitas" yang memuat riwayat langkah beserta PSNR dan MSE masukan dan hasil terhadap referensi. Klik kanan pada citra atau spektrum menyediakan menu "Salin gambar".

3.1.1.3 Panel Filter H (Kanan)

Tab Filter H menampilkan H(u,v) yang dihitung dari parameter yang sedang diisi (diperbarui otomatis sebelum Terapkan ditekan), spektrum citra kerja yang sudah dikali H, tabel noise peaks (posisinya ditandai lingkaran merah pada gambar H), dan pola noise yang dikurangkan pada mode pass. Untuk metode Brightness tab ini hanya menampilkan keterangan karena tidak ada filter H, dan untuk motion blur yang ditampilkan adalah |H| PSF tanpa padding.

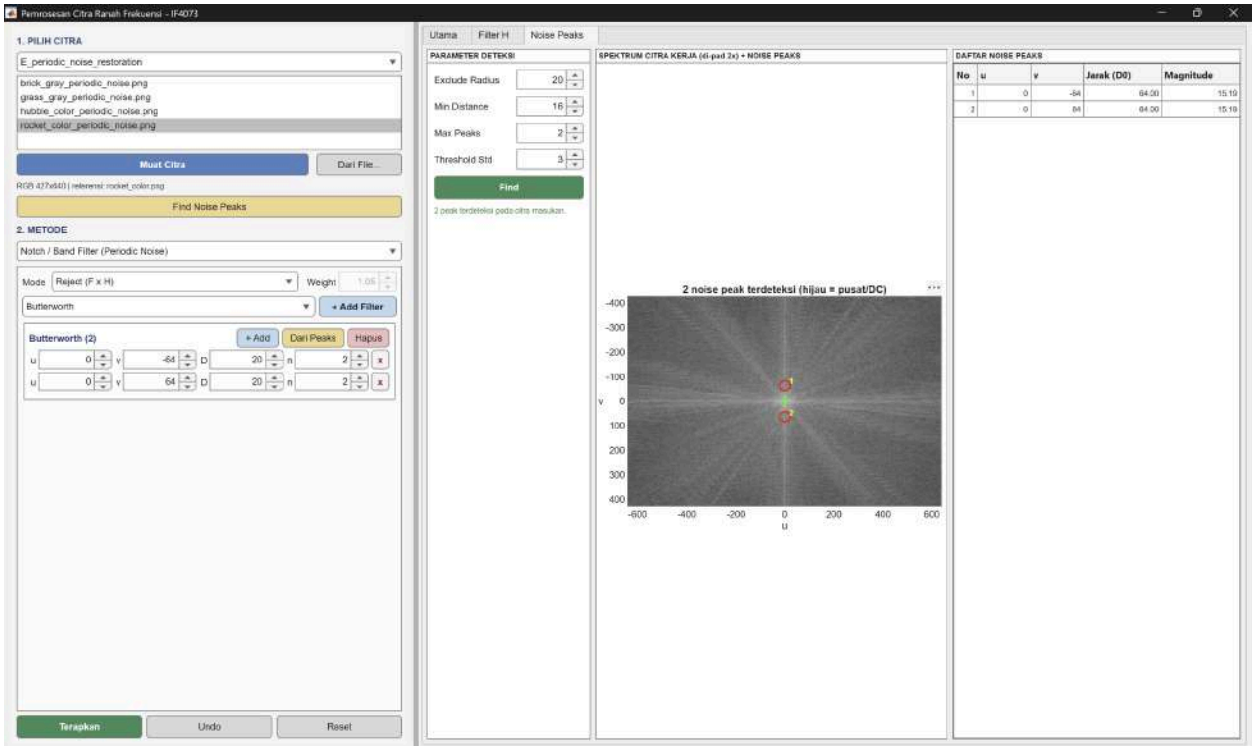


Gambar 3.2 Tampilan tab Filter H pada FrequencyDomainApp

...

3.1.1.4 Panel Noise Peaks (Kanan)

Tab Noise Peaks dibuat saat tombol "Find Noise Peaks" pertama kali ditekan. Tab ini memuat parameter deteksi (Exclude Radius 20, Min Distance 16, Max Peaks 10, Threshold Std 3 sebagai nilai awal), tombol Find, spektrum citra kerja yang di-pad 2x lengkap dengan lingkaran dan nomor tiap peak, serta daftar peaks dengan koordinat (u,v) yang sama dengan koordinat filter H.



Gambar 3.3 Tampilan tab Noise Peaks pada FrequencyDomainApp

3.1.2 Alur Penggunaan GUI (Workflow)

Alur interaksi pengguna saat mengoperasikan aplikasi ini dijabarkan melalui langkah-langkah berikut:

1. Inisialisasi dan pemindaian dataset:

   FrequencyDomainApp menambahkan src/ dan gui/ ke path, membangun UI (initApp), lalu memanggil datasetIndex() agar dropdown kategori dan daftar berkas terisi otomatis (onFolderChanged).

2. Pemuatan citra:

   p=Pengguna memilih kategori dan berkas, lalu menekan "Muat Citra" (loadSelectedImage) atau "Dari File...". Program membaca citra, mencari referensi yang namanya menjadi awalan nama citra masukan (readReference, misalnya brick_gray_periodic_noise.png dengan brick_gray.png), menyeragamkan citra ke uint8 grayscale atau RGB (toUint8Image), mengisi L dan theta otomatis jika nama berkas memuat _L..._A... (fillMotionFromName), lalu menampilkan citra, spektrum, dan PSNR/MSE awal.

3. Pemilihan metode dan parameter:

   Callback onMethodChanged menampilkan panel parameter metode terpilih. Setiap perubahan parameter filter langsung memperbarui preview H di tab Filter H (refreshFilterPreview) tanpa mengubah hasil.

4. Analisis spektrum untuk derau periodik:

   Pengguna menekan "Find Noise Peaks" (onOpenPeaksTab), mengatur parameter deteksi, lalu menekan "Find" (onFindPeaks). findNoisePeaks dijalankan pada citra kerja yang sudah di-pad 2x sehingga koordinat (u,v) peak sama dengan koordinat filter H. Pada card filter titik, tombol "Dari Peaks" (onFillFromPeaks) membuat satu baris filter untuk tiap peak, filter garis dan pita diisi manual dari pengamatan spektrum.

5. Tuning (opsional):

   Tombol Tune menjalankan golden-section search terhadap citra referensi. Nilai terbaik hanya diisikan ke field parameter dan belum diterapkan.

6. Eksekusi:

   Tombol "Terapkan" (onApply) mengumpulkan parameter (collectInputs), memanggil applyFrequencyMethod pada citra kerja, menyimpan keadaan sebelumnya ke riwayat, lalu menampilkan hasil, spektrum hasil, dan PSNR/MSE di kotak log.

7. Manajemen riwayat:

   "Undo" membatalkan langkah terakhir dan "Reset" membuang semua langkah. Citra atau spektrum dapat disalin ke clipboard lewat klik kanan, "Salin gambar".

3.2 Implementasi Pass Filter

Sub-bab ini membahas implementasi kelompok *pass filters* yang direalisasikan di dalam direktori `src/pass_filters/`. Implementasi dirancang secara efisien dengan menggabungkan fungsi *Low-Pass Filter* dan *High-Pass Filter* ke dalam satu modul fungsi tunggal untuk setiap varian (Ideal, Butterworth, dan Gaussian). Penentuan jenis penapisan (LPF atau HPF) dikendalikan melalui parameter logika *boolean* `isLowPass`, sementara fleksibilitas posisi titik pusat penapis ditentukan oleh koordinat relatif (u_0, v_0) terhadap pusat spektrum.

3.2.1 Ideal Pass Filter

Implementasi *Ideal Pass Filter* direalisasikan melalui file modul `idealPass.m`. Fungsi ini menerima lima parameter masukan utama, yaitu `sizeImg` yang

merepresentasikan dimensi citra `[M, N]`, `u0` dan `v0` yang menentukan koordinat titik pusat penapis relatif terhadap pusat spektrum (bernilai *default* 0 jika tidak diisi), `diameter` yang menentukan lebar daerah pelolosan frekuensi, serta `isLowPass` sebagai penanda jenis penapisan. Keluaran dari fungsi ini adalah matriks dua dimensi `H` berukuran `sizeImg` dengan rentang nilai biner 0 atau 1.

Secara komputasi, fungsi diawali dengan pembentukan kisi koordinat frekuensi dua dimensi (U, V) memanfaatkan fungsi pembantu `freqGrid`. Selanjutnya, matriks jarak Euclidean $D(u, v)$ dihitung dari setiap titik kisi terhadap titik pusat penapis (u_0, v_0), dan jari-jari frekuensi ambang D_0 diperoleh dari setengah nilai `diameter`. Pada LPF, nilai fungsi transfer dibentuk secara biner melalui kondisi `D <= D0`, di mana seluruh komponen frekuensi di dalam radius D_0 bernilai 1 dan frekuensi di luar radius bernilai 0. Jika parameter `isLowPass` bernilai `false`, fungsi secara otomatis menerapkan operasi komplementer $H_{HPF} = 1 - H_{LPF}$ untuk menghasilkan penapis HPF. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = idealPass(sizeImg, u0, v0, diameter, isLowPass)
% filter Ideal (lowpass/highpass) berpusat di titik (u0, v0) relatif
% terhadap tengah spektrum.
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);

D = sqrt((U - u0).^2 + (V - v0).^2);
D0 = diameter / 2;

lowpass = double(D <= D0);

if isLowPass
    H = lowpass;
else
    H = 1 - lowpass;
end
end
```

3.2.2 Butterworth Pass Filter

Implementasi *Butterworth Pass Filter* direalisasikan melalui file modul `butterworthPass.m`. Berbeda dengan ideal filter, fungsi ini membutuhkan enam parameter masukan dengan penambahan parameter `n` yang merepresentasikan orde penapis untuk mengendalikan tingkat kecuraman daerah transisi. Parameter masukan lainnya meliputi `sizeImg`, `u0`, `v0`, `diameter`, dan `isLowPass`, dengan keluaran berupa matriks fungsi transfer dua dimensi `H` berpotensial nilai kontinu antara 0 hingga 1.

Alur algoritma diawali dengan kalkulasi kisi frekuensi dan matriks jarak Euclidean $D(u, v)$ terhadap titik pusat (u_0, v_0). Untuk mencegah terjadinya kesalahan komputasi akibat pembagian dengan nilai nol pada titik pusat $(D = 0)$, nilai jarak $D = 0$ secara khusus digantikan dengan konstanta presisi `eps` (`D(D == 0) = eps`). Fungsi transfer LPF kemudian dihitung menggunakan persamaan Butterworth $1/(1 + (D/D_0)^{2n})$, di mana D_0 merupakan hasil pembagian `diameter` dengan 2. Kurva transisi yang dihasilkan melandai secara mulus sesuai nilai orde `n`. Apabila nilai `isLowPass` bernilai `false`, fungsi melakukan pengurangan komplementer $1 - H_{LPF}$ untuk membentuk fungsi transfer HPF. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = butterworthPass(sizeImg, u0, v0, n, diameter, isLowPass)
% filter berpusat di titik (u0, v0) relatif terhadap tengah spektrum.
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);
```

```
D = sqrt((U - u0).^2 + (V - v0).^2);
D(D == 0) = eps;

D0 = diameter / 2;

lowpass = 1 ./ (1 + (D ./ D0).^(2 * n));

if isLowPass
    H = lowpass;
else
    H = 1 - lowpass;
end
end
```

3.2.3 Gaussian Pass Filter

Implementasi *Gaussian Pass Filter* direalisasikan melalui file modul `gaussianPass.m`. Modul ini menerima lima parameter masukan yang sejajar dengan varian ideal, yaitu `sizeImg`, `u0`, `v0`, `diameter`, dan `isLowPass`, serta menghasilkan keluaran matriks fungsi transfer H berukuran `sizeImg` dengan nilai kontinu bergradasi halus dari 0 hingga 1.

Proses komputasi dimulai dengan pembentukan kisi frekuensi dan perhitungan matriks jarak $D(u, v)$ terhadap titik pusat (u_0, v_0). Formulasi LPF dihitung menggunakan fungsi eksponensial Gaussian $\exp(-D^2/(2D_0^2 + \epsilon))$, di mana D_0 adalah setengah dari parameter `diameter`. Penambahan variabel nilai sangat kecil `eps` pada bagian penyebut dilakukan untuk mengantisipasi kestabilan numerik apabila parameter `diameter` bernilai nol. Sifat distribusi Gaussian menghasilkan kurva pelolosan frekuensi yang sangat mulus tanpa lonjakan diskontinuitas, sehingga bebas dari efek *ringing*. HPF dibentuk melalui operasi komplementer $1 - H_{LPF}$ saat parameter `isLowPass` bernilai `false`. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = gaussianPass(sizeImg, u0, v0, diameter, isLowPass)
% filter Gaussian (lowpass/highpass) berpusat di titik (u0, v0)
% relatif terhadap tengah spektrum.
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);

D = sqrt((U - u0).^2 + (V - v0).^2);
D0 = diameter / 2;

lowpass = exp(-(D.^2) ./ (2 * (D0^2) + eps));

if isLowPass
    H = lowpass;
else
    H = 1 - lowpass;
end
end
```

3.3 Implementasi Band Filter

Sub-bab ini membahas implementasi *band filters* yang direalisasikan di dalam direktori `src/band_filters/`. Berbeda dengan *pass filters* yang memotong frekuensi secara radial searah titik pusat, *band filters* bekerja pada daerah berbentuk cincin radial dengan lebar pita tertentu W dan radius pusat pita D_0. Seluruh fungsi di dalam modul ini mengintegrasikan *Band-Reject Filter* dan *Band-Pass Filter* ke dalam satu modul tunggal yang dikendalikan oleh parameter logika *boolean* `isBandPass` serta mendukung penyesuaian posisi titik pusat notch (u_0, v_0).

3.3.1 Ideal Band Filter

Implementasi *Ideal Band Filter* direalisasikan melalui file modul `bandIdeal.m`. Fungsi ini menerima enam parameter masukan utama, yaitu `sizeImg` sebagai dimensi matriks citra `[M, N]`, `u0` dan `v0` yang menentukan koordinat titik pusat cincin pita

relatif terhadap pusat spektrum (bernilai *default* 0 jika kosong), `diameter` yang menentukan diameter radial pusat pita ($2D_0$), `width` yang menentukan lebar pita W, serta `isBandPass` sebagai penanda jenis operasi. Keluaran dari fungsi ini adalah matriks fungsi transfer dua dimensi `H` berukuran `sizeImg` dengan nilai biner 0 atau 1.

Proses komputasi diawali dengan pembuatan kisi koordinat frekuensi 2D (U, V) menggunakan fungsi pembantu `freqGrid`. Jarak Euclidean $D(u, v)$ dihitung dari setiap titik kisi ke pusat (u_0, v_0), sedangkan posisi radius pusat pita D_0 diperoleh dari setengah nilai `diameter`. Pada BPF, fungsi membentuk pita biner `bandpass` dengan menetapkan nilai 1 hanya untuk komponen frekuensi yang berada di dalam rentang rentangan jarak $D_0 - W/2 \leq D \leq D_0 + W/2$, dan nilai 0 untuk frekuensi di luar pita tersebut. Jika parameter `isBandPass` bernilai `false`, fungsi melakukan operasi komplementer $1 - bandpass$ untuk menghasilkan fungsi transfer *Band-Reject*. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = bandIdeal(sizeImg, u0, v0, diameter, width, isBandPass)
% filter pita Ideal (band reject/pass), cincin radial berpusat di
% (u0, v0)
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);

D = sqrt((U - u0).^2 + (V - v0).^2);
D0 = diameter / 2;
W = width;

bandpass = double(D >= (D0 - W/2) & D <= (D0 + W/2));

if isBandPass
    H = bandpass;
else
    H = 1 - bandpass; % Bandreject
end
end
```

3.3.2 Butterworth Band Filter

Implementasi *Butterworth Band Filter* direalisasikan melalui file modul `bandButterworth.m`. Fungsi ini menerima tujuh parameter masukan dengan penambahan parameter orde penapis `n` untuk mengatur kecuraman kurva transisi pada batas pita frekuensi. Parameter lainnya meliputi `sizeImg`, `u0`, `v0`, `diameter`, `width`, dan `isBandPass`, dengan keluaran berupa matriks fungsi transfer dua dimensi `H` berpotensial nilai kontinu antara 0 hingga 1.

Algoritma diawali dengan kalkulasi kisi frekuensi 2D dan matriks jarak Euclidean $D(u, v)$. Selanjutnya, fungsi menghitung komponen pembilang `num = D .* W` dan penyebut `denom = D.^2 - D0^2`. Untuk menangani kondisi singularitas matematika di mana pembagian dengan nol terjadi ketika jarak D sama persis dengan radius pusat pita D_0 $(D = D_0 \Rightarrow denom = 0)$, nilai `denom` yang bernilai nol digantikan dengan nilai presisi mesin `eps` (`denom(denom == 0) = eps`). Fungsi transfer BRF dibentuk menggunakan formulasi $1/(1 + (num/denom)^{2n})$. Apabila parameter `isBandPass` bernilai `true`, fungsi membalikkan kurva melalui operasi komplementer $1 - bandreject$ untuk menghasilkan fungsi transfer BPF. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = bandButterworth(sizeImg, u0, v0, diameter, width, n, isBandPass)
% filter pita Butterworth (band reject/pass), cincin radial berpusat
% di (u0, v0)
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);

D = sqrt((U - u0).^2 + (V - v0).^2);
```

```
D0 = diameter / 2;
W = width;

num = D .* W;
denom = D.^2 - D0^2;

% Hindari pembagian dengan nol ketika D = D0
denom(denom == 0) = eps;

% Rumus Butterworth Bandreject
bandreject = 1 ./ (1 + (num ./ denom).^(2 * n));

if isBandPass
    H = 1 - bandreject; % Bandpass
else
    H = bandreject;     % Bandreject
end
end
```

3.3.3 Gaussian Band Filter

Implementasi *Gaussian Band Filter* direalisasikan melalui file modul `bandGaussian.m`. Modul ini menerima enam parameter masukan yang sejajar dengan varian ideal, yaitu `sizeImg`, `u0`, `v0`, `diameter`, `width`, dan `isBandPass`, serta menghasilkan keluaran matriks fungsi transfer `H` berukuran `sizeImg` dengan transisi kurva paling mulus dan kontinu.

Proses komputasi dimulai dengan pembentukan kisi frekuensi dan perhitungan matriks jarak $D(u,v)$ terhadap titik pusat (u_0, v_0). Komponen penyebut dibentuk melalui perkalian `denom = D .* W`. Untuk menjaga kestabilan numerik dan mencegah pembagian dengan nol pada titik pusat $D = 0$ atau saat `width` bernilai 0, nilai penyebut yang bernilai nol disesuaikan menggunakan `denom(denom == 0) = eps`. Formulasi BPF dihitung menggunakan fungsi eksponensial Gaussian $\exp(-0.5 * ((D^2 - D_0^2)/denom)^2)$. Jika parameter `isBandPass` bernilai `false`, fungsi menerapkan pemotongan komplementer $1 - bandpass$ untuk menghasilkan fungsi transfer BRF. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = bandGaussian(sizeImg, u0, v0, diameter, width, isBandPass)
% filter pita Gaussian (band reject/pass), cincin radial berpusat di
% (u0, v0), transisi paling halus.
%
% OUTPUT:
% H - filter 2D (nilai 0..1), ukuran sizeImg

if isempty(u0), u0 = 0; end
if isempty(v0), v0 = 0; end

[U, V] = freqGrid(sizeImg);

D = sqrt((U - u0).^2 + (V - v0).^2);
D0 = diameter / 2;
W = width;

denom = D .* W;
denom(denom == 0) = eps;

% Rumus Gaussian Bandpass
bandpass = exp(-0.5 * (((D.^2 - D0^2) ./ denom).^2));

if isBandPass
    H = bandpass;
else
    H = 1 - bandpass; % Bandreject
end
end
```

3.4 Implementasi Pencerahan Citra dalam Ranah Frekuensi

Implementasi pencerahan citra pada ranah frekuensi direalisasikan melalui modul fungsi `brightenFrequency.m` yang berada di dalam direktori `src/brightening/`. Modul ini dirancang untuk meningkatkan atau menyesuaikan tingkat kecerahan (*brightness*) citra masukan *grayscale* maupun RGB secara proporsional tanpa merusak rasio keseimbangan warna asli. Fungsi ini menerima dua parameter masukan, yaitu `img` yang merepresentasikan matriks citra asal dan `targetMean` yang menentukan target rata-rata kecerahan dalam rentang ter-normalisasi $[0, 1]$ (bernilai *default* 0,48 jika tidak ditentukan oleh pengguna). Keluaran dari fungsi ini adalah `imgBright`, yaitu matriks citra hasil pencerahan dengan tipe data dan dimensi yang identik dengan citra masukan.

Alur algoritma diawali dengan identifikasi tipe data awal masukan melalui fungsi `isa(img, 'uint8')` serta konversi citra ke dalam format `double` dalam rentang $[0, 1]$ menggunakan `im2double`. Selanjutnya, fungsi mengekstraksi dimensi citra $[M, N, C]$, di mana C mewakili jumlah saluran warna ($C = 1$ untuk *grayscale* dan $C = 3$ untuk RGB). Untuk menghitung rata-rata kecerahan awal citra secara presisi, fungsi melakukan perulangan pada setiap saluran warna c, menerapkan Transformasi Fourier Diskrit dua dimensi `fft2`, serta mengagregasikan bagian riil dari komponen DC yang terletak pada koordinat $(1, 1)$ spektrum (`F_temp(1,1)`). Rata-rata kecerahan citra saat ini (`currentMean`) diperoleh dari hasil pembagian total komponen DC terhadap jumlah saluran C dan total piksel $M \times N$.

Berdasarkan nilai `currentMean`, fungsi menghitung satu nilai faktor skalar global α dengan merasionalkan target kecerahan terhadap kecerahan awal ($\alpha = targetMean/currentMean$). Apabila nilai kecerahan awal bernilai nol, nilai α ditetapkan secara aman sebesar $1,0$ untuk menghindari pembagian dengan nol. Penggunaan satu nilai α global yang seragam untuk seluruh saluran warna menjamin bahwa proporsi intensitas Red, Green, dan Blue tetap terjaga.

Pada tahap pencerahan utama, fungsi melakukan Transformasi Fourier `fft2` pada setiap saluran warna c dan mengalikan seluruh matriks spektrum $F(u, v)$ dengan skalar α ($G = F \cdot \alpha$). Hasil perkalian spektrum kemudian ditransformasikan kembali ke ranah spasial menggunakan Transformasi Fourier Balik `ifft2`. Nilai imajiner sisa akibat ketelitian komputasi dibuang menggunakan fungsi `real`, dan intensitas piksel dibatasi secara ketat (*clamping*) pada rentang $[0, 1]$ menggunakan fungsi `max(0, min(1, res))` untuk mencegah efek *overflow*. Terakhir, jika citra masukan awal bertipe `uint8`, matriks hasil dikembalikan ke format `uint8` menggunakan `im2uint8`. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function imgBright = brightenFrequency(img, targetMean)
% Peningkatan kecerahan citra di ranah frekuensi tanpa mengubah rasio warna RGB
%
% INPUT:
% img        - Citra masukan (Grayscale 2D atau RGB 3D)
% targetMean - Target kecerahan rata-rata [0..1] (default: 0.48)
%
% OUTPUT:
% imgBright  - Citra hasil pencerahan

if nargin < 2 || isempty(targetMean)
    targetMean = 0.48;
end

isInputUint8 = isa(img, 'uint8');
imgDouble = im2double(img);
[M, N, C] = size(imgDouble);
imgBright = zeros(size(imgDouble));

% Hitung total rata-rata kecerahan awal dari komponen DC seluruh channel
totalDC = 0;
for c = 1:C
    F_temp = fft2(imgDouble(:,:,c));
    totalDC = totalDC + real(F_temp(1,1));
end
currentMean = (totalDC / C) / (M * N);

% Hitung satu nilai alpha global agar rasio R:G:B tidak terdistorsi
if currentMean > 0
    alpha = targetMean / currentMean;
else
    alpha = 1.0;
end

% Terapkan alpha yang sama ke seluruh channel di ranah frekuensi
for c = 1:C
    F = fft2(imgDouble(:,:,c));

    % Skaling spektrum dengan alpha global
    G = F * alpha;

    res = real(ifft2(G));
    imgBright(:,:,c) = max(0, min(1, res));
end

if isInputUint8
    imgBright = im2uint8(imgBright);
end
end
```

3.5 Implementasi Notch Filter

Implementasi *notch filter* direalisasikan melalui modul fungsi `notchLineFilter.m` yang berada di dalam direktori `src/notch_filter/`. Modul ini dirancang untuk mengatasi derau garis periodik melalui pembentukan *Line Notch Filter* pada

orientasi horizontal maupun vertikal. Selain itu, fungsi ini menyediakan fleksibilitas tinggi dengan mendukung bentuk tepi tegas (*Ideal*) maupun tepi melandai (*Butterworth*), operasi meloloskan (*Pass*) atau menolak (*Reject*), serta fitur opsional pemutus garis berbentuk lingkaran (*break circle*) untuk mengamankan komponen frekuensi penting seperti titik pusat DC.

Fungsi ini menerima sembilan parameter masukan, yaitu `sizeImg` sebagai dimensi citra `[M, N]`, `orientation` untuk menentukan arah garis (`'vertical'` atau `'horizontal'`), `position` sebagai lokasi garis relatif terhadap pusat spektrum, `width` sebagai lebar total garis, `blur` untuk menentukan tingkat keburaman tepi (bernilai 0 untuk tepi tegas *Ideal* dan > 0 untuk *Butterworth* dengan orde $n = 1/blur$), serta `isPass` sebagai penanda logika *boolean* (`true` untuk *Notch-Pass* dan `false` untuk *Notch-Reject*). Parameter opsional tambahan mencakup `breakPos` untuk menentukan posisi pusat pemutus pada sumbu bebas, `breakRadius` untuk radius pemutus R_c, serta `breakBlur` untuk tingkat keburaman tepi lingkaran pemutus. Keluaran fungsi berupa matriks dua dimensi `H` berukuran `sizeImg` dengan rentang nilai $[0, 1]$.

Alur komputasi diawali dengan penanganan parameter opsional default serta pembentukan kisi frekuensi dua dimensi (U, V) melalui fungsi `freqGrid`. Jarak tegak lurus D dari setiap koordinat frekuensi terhadap garis posisi notch dihitung berdasarkan orientasi yang dipilih, yaitu $D = |U - position|$ untuk orientasi vertikal atau $D = |V - position|$ untuk orientasi horizontal. Setengah lebar garis ditetapkan sebagai $D_0 = width/2$. Pada tahap pembentukan profil garis (`passShape`), jika parameter `blur == 0`, fungsi membentuk profil *Ideal* secara biner via kondisi `D <= D0`. Apabila `blur > 0`, fungsi menghitung orde penapis $n = 1/blur$ dan menerapkan

formulasi *Butterworth* $1/(1 + (D/D_0)^{2n})$ dengan mengamankan D_0 menggunakan `eps`. Jika parameter `isPass` bernilai `true`, matriks `H` mengambil nilai `passShape` dengan nilai acuan dasar `baseline = 0`; sedangkan jika `isPass` bernilai `false`, matriks `H` dibalik menjadi `1 - passShape` (*Notch-Reject*) dengan nilai `baseline = 1`.

Tahap akhir adalah pengaplikasian pemutus lingkaran (*break circle*) apabila parameter `breakPos` dan `breakRadius` diisi. Jarak Euclidean D_{circle} dihitung dari setiap titik kisi terhadap pusat lingkaran pemutus, yaitu $(u = position, v = breakPos)$ untuk garis vertikal atau $(u = breakPos, v = position)$ untuk garis horizontal. Profil lingkaran pemutus (`restoreShape`) dibentuk secara *Ideal* (`double(Dcircle <= breakRadius)`) jika `breakBlur == 0` atau secara *Butterworth* menggunakan orde $n_{break} = 1/breakBlur$ jika `breakBlur > 0`. Nilai fungsi transfer pada area pemutus kemudian dikembalikan ke nilai `baseline` melalui operasi penggabungan $H = H \cdot (1 - restoreShape) + baseline \cdot restoreShape$, sehingga garis notch terputus tepat pada area lingkaran tanpa merusak frekuensi utama. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = notchLineFilter(sizeImg, orientation, position, width, blur, isPass,
breakPos, breakRadius, breakBlur)
% Membuat filter berbentuk garis (horizontal/vertikal) bisa pass atau reject,
% dengan tepi tegas (ideal) atau buram (Butterworth-style).
% Opsional: garis bisa diputus oleh satu lingkaran di titik tertentu.
%
% INPUT:
%   sizeImg     - [rows, cols]
%   orientation - 'vertical' atau 'horizontal'
%   position    - lokasi garis, relatif terhadap pusat spektrum:
%                   'vertical'   -> posisi di sumbu U (horizontal)
%                   'horizontal' -> posisi di sumbu V (vertikal)
%   width       - lebar garis (total)
%   blur        - 0   -> tepi garis tegas (ideal, hard-edge)
%                 > 0 -> tepi garis buram, n = 1/blur (makin besar blur, makin
landai)
%   isPass      - true  -> PASS
%                 false -> REJECT
%
```

```
% INPUT OPSIONAL (garis terputus):
%   breakPos    - posisi pusat lingkaran pemutus, di sumbu BEBAS garis:
%                   garis vertikal   -> breakPos ini posisi di V
%                   garis horizontal -> breakPos ini posisi di U
%                 Kosongkan [] atau tidak usah diisi kalau tidak perlu.
%   breakRadius - radius lingkaran pemutus
%   breakBlur   - 0   -> tepi lingkaran pemutus tegas
%                 > 0 -> tepi lingkaran pemutus buram
%
% OUTPUT:
%   H - filter 2D (nilai 0..1), ukuran sizeImg


if nargin < 7, breakPos = []; end
if nargin < 8, breakRadius = []; end
if nargin < 9, breakBlur = 0; end


[U, V] = freqGrid(sizeImg);

% jarak tegak lurus ke garis
switch lower(orientation)
    case 'vertical'
        D = abs(U - position); % misal --> .. 3 2 1 | 1 2 3
    case 'horizontal'
        D = abs(V - position);
    otherwise
        error('notchLineFilter:badOrientation', ...
            'orientation harus ''vertical'' atau ''horizontal'', bukan "%s"',
orientation);
end

D0 = width / 2; % lebar setengah

% bentuk transisi: ideal (blur=0) atau butterworth (blur>0)
if blur == 0
    passShape = double(D <= D0);   % nilai 1 di dalam garis
else
    n = 1 / blur;
    D0safe = D0;
    D0safe(D0safe == 0) = eps;
    passShape = 1 ./ (1 + (D ./ D0safe).^(2 * n));   % nilai 1 di garis, gradasi ke
0
end

if isPass
    H = passShape;
    baseline = 0;
else
    H = 1 - passShape;
    baseline = 1;
end

% memutus garis dengan lingkaran
if ~isempty(breakPos) && ~isempty(breakRadius) && breakRadius > 0
    switch lower(orientation)
        case 'vertical'
            % pusat lingkaran: (u=position, v=breakPos)
            Dcircle = sqrt((U - position).^2 + (V - breakPos).^2);
        case 'horizontal'
            % pusat lingkaran: (u=breakPos, v=position)
```

```
            Dcircle = sqrt((U - breakPos).^2 + (V - position).^2);
    end

    if breakBlur == 0
        restoreShape = double(Dcircle <= breakRadius);
    else
        nBreak = 1 / breakBlur;
        restoreShape = 1 ./ (1 + (Dcircle ./ breakRadius).^(2 * nBreak));
    end

    H = H .* (1 - restoreShape) + baseline .* restoreShape;
end
end
```

Pada mode reject, spektrum citra langsung dikalikan filter gabungan, G(u,v) = F(u,v) · H(u,v), lalu dikembalikan dengan ifft2 melalui applyFrequencyFilter.m. Pada mode pass, filter gabungan hanya meloloskan komponen derau. Pola derau di ranah spasial diperoleh dengan ifft2 dari F · H_pass, lalu dikurangkan dari citra asli dengan bobot w (weight). Pendekatan ini serupa optimum notch filtering: derau diisolasi lebih dulu, sehingga citra hasil adalah citra asli dikurangi pola deraunya.

```
function [imgResult, noisePattern] = subtractNotchPattern(imgOriginal, imgPadded,
Hpass, origSize, weight)
% implementasi "optimum notch filtering" dipakai  ketika filter yang dibangun
buildFilter.m berupa notch PASS (isPass=true)

if nargin < 5
    weight = 1;
end

F = fft2(double(imgPadded));
F = fftshift(F);
Fnoise = F .* Hpass;

rawNoisePattern = ifft2(ifftshift(Fnoise));
rawNoisePattern = real(rawNoisePattern);
rawNoisePattern = rawNoisePattern(1:origSize(1), 1:origSize(2));  % crop balik

noisePattern = weight * rawNoisePattern;

imgResult = double(imgOriginal) - noisePattern;

imgResult = max(0, min(255, imgResult));
imgResult = uint8(imgResult);
end
```

3.6 Implementasi Fungsi Restorasi Motion Blur

Sub-bab ini membahas implementasi rangkaian fungsi restorasi citra terdegradasi *motion blur* yang diimplementasikan pada direktori `src/motion_blur/`. Rangkaian modul ini terbagi menjadi dua tahap utama, yaitu pemodelan matematis fungsi PSF $H(u, v)$ melalui `motionBlurPSF.m`, serta tahap pemulihan spektrum citra menggunakan metode *Inverse Filtering* melalui `inverseFilter.m` dan *Wiener Filtering* melalui `wienerFilter.m`.

3.6.1 Pembentukan PSF Motion Blur

Pemodelan PSF direalisasikan melalui file modul `motionBlurPSF.m`. Modul ini bertugas merekonstruksi matriks filter $H(u, v)$ berdasarkan estimasi pergerakan fisik kamera atau objek. Fungsi menerima tiga parameter masukan, yaitu `sizeImg` sebagai dimensi matriks citra `[baris, kolom]`, `L` sebagai panjang pergerakan piksel (*motion length*) dengan nilai *default* 1, serta `theta` sebagai sudut arah pergerakan dalam satuan derajat (*motion angle*) dengan nilai *default* 0. Keluaran dari fungsi ini adalah matriks dua dimensi `H` berukuran `sizeImg` dengan format koordinat frekuensi tidak ter-*shift* (komponen DC terletak pada titik origin `(1,1)`), sehingga dapat langsung dikalikan dengan spektrum citra hasil `fft2`.

Alur komputasi diawali dengan konversi sudut `theta` ke dalam satuan radian (`thetaRad`). Menggunakan kisi koordinat frekuensi dua dimensi (U, V) dari fungsi `freqGrid`, fungsi menghitung variabel proyeksi pergerakan Δ melalui formulasi $\Delta = (U/N) \cdot L\cos\theta + (V/M) \cdot L\sin\theta$, di mana U dibagi dengan jumlah kolom N dan V dibagi dengan jumlah baris M. Selanjutnya, kurva peredaman spektrum dihitung menggunakan fungsi sinc manual $\sin(\pi\Delta)/(\pi\Delta)$. Untuk menangani kasus khusus pembagian dengan nol saat $\Delta = 0$ (titik pusat spektrum), nilai sinc pada posisi tersebut ditetapkan bernilai 1 secara eksplisit menggunakan *masking*

(`sincVal(mask)`). Matriks filter H murni dibentuk oleh fungsi sinc riil tanpa menyertakan faktor fase eksponensial $e^{-j\pi\Delta}$, sehingga kernel pemburaman dianggap terpusat untuk mencegah pergeseran spasial sebesar $\sim L/2$ piksel pada citra hasil restorasi. Tahap akhir menerapkan fungsi `ifftshift` pada matriks sinc untuk memindahkan posisi komponen DC dari pusat matriks ke pojok kiri atas `(1,1)` agar selaras dengan keluaran standar `fft2`. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function H = motionBlurPSF(sizeImg, L, theta)
% Pembentukan fungsi transfer degradasi / PSF Motion Blur H(u,v)
%
% INPUT:
% sizeImg - Ukuran matriks citra [baris, kolom]
% L       - Panjang pergerakan piksel (motion length)
% theta   - Sudut arah pergerakan dalam derajat (motion angle)
%
% OUTPUT:
% H       - Matriks 2D fungsi degradasi H(u,v), TIDAK ter-shift (DC di
%           pojok kiri atas), langsung bisa dikali dengan fft2(citra)
%
% Kernel blur dianggap terpusat, jadi H murni sinc tanpa faktor fase
% exp(-j*pi*Delta). Faktor fase itu menggeser hasil restorasi ~L/2 piksel.

if isempty(L) || L == 0, L = 1; end
if isempty(theta), theta = 0; end

[U, V] = freqGrid(sizeImg);
thetaRad = theta * pi / 180;

% di freqGrid, U = frekuensi horizontal (sepanjang kolom) dan V = vertikal
% (sepanjang baris), jadi U dibagi jumlah kolom dan V dibagi jumlah baris
Delta = (U / sizeImg(2)) * L * cos(thetaRad) + (V / sizeImg(1)) * L * sin(thetaRad);

% Sinc manual: sin(pi*x) / (pi*x)
sincVal = ones(size(Delta));
mask = (Delta ~= 0);
sincVal(mask) = sin(pi * Delta(mask)) ./ (pi * Delta(mask));

H = ifftshift(sincVal);
end
```

3.6.2 Inverse Filtering

Inverse Filtering direalisasikan melalui modul `inverseFilter.m`. Fungsi ini bekerja pada ranah frekuensi dengan membagi spektrum citra terdegradasi $G(u,v)$

terhadap fungsi transfer degradasi $H(u,v)$. Fungsi menerima tiga parameter masukan, yaitu `G` yang merupakan matriks spektrum Fourier 2D citra terdegradasi, `H` sebagai matriks fungsi transfer PSF, serta `threshold` sebagai ambang batas magnitudo minimum $|H|$ untuk menapis kestabilan pembagian (bernilai *default* 0,01). Keluaran fungsi berupa `F_hat`, yaitu spektrum Fourier 2D citra hasil estimasi/restorasi.

Proses komputasi diawali dengan inisialisasi matriks matriks pembatas terbalik `H_inv` bernilai nol dengan ukuran yang sama dengan `H`. Untuk mencegah timbulnya pembagian dengan nilai nol atau nilai yang sangat kecil pada titik perpotongan nol fungsi sinc yang dapat memicu pembesaran derau ekstrem, fungsi membentuk *masking* logika `abs(H) >= threshold`. Operasi pembagian terbalik $1/H(u,v)$ hanya dilakukan pada titik-titik frekuensi yang lolos pengujian ambang batas tersebut, sedangkan titik frekuensi di bawah ambang batas tetap bernilai nol. Spektrum terestimasi `F_hat` kemudian diperoleh melalui perkalian titik matriks `G .* H_inv`. Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function F_hat = inverseFilter(G, H, threshold)
% Restorasi spektrum citra menggunakan Inverse Filtering
%
% INPUT:
% G         - Spektrum Fourier 2D citra terdegradasi (G(u,v))
% H         - Fungsi transfer degradasi PSF (H(u,v))
% threshold - Nilai ambang batas minimum |H| untuk mencegah lonjakan derau
%
% OUTPUT:
% F_hat     - Spektrum Fourier 2D citra hasil restorasi

if nargin < 3 || isempty(threshold)
    threshold = 0.01;
end

H_inv = zeros(size(H));

mask = abs(H) >= threshold;
H_inv(mask) = 1 ./ H(mask);

F_hat = G .* H_inv;
end
```

3.6.3 Wiener Filtering

Wiener Filtering direalisasikan melalui modul `wienerFilter.m`. Penapis ini mengoptimalkan proses restorasi dengan memperhitungkan faktor *Noise-to-Signal Ratio* (NSR) untuk menekan pembesaran derau aditif secara lebih halus. Fungsi menerima tiga parameter masukan, yaitu `G` sebagai spektrum Fourier 2D citra terdegradasi, `H` sebagai matriks fungsi transfer PSF, serta `K` sebagai parameter konstan rasio NSR (bernilai *default* 0,01). Keluaran dari fungsi ini adalah `F_hat`, yaitu matriks spektrum Fourier 2D citra terestimasi.

Proses komputasi dilakukan secara langsung melalui pembentukan matriks penapis Wiener $W(u,v)$ mengacu pada formulasi $W = H^{*}/(|H^{2}| + K)$, di mana H^{*} diperoleh menggunakan konjugat kompleks `conj(H)` dan $|H|^{2}$ diperoleh dari `abs(H).^2`. Konstanta K pada bagian penyebut bertindak sebagai faktor regulasi yang mencegah timbulnya pembagian dengan nol secara kontinu tanpa memerlukan pemotongan ambang batas yang kaku. Spektrum citra hasil pemulihan `F_hat` kemudian dihitung melalui perkalian titik spektrum terdegradasi dengan matriks penapis Wiener (`G .* W`). Implementasi lengkap logika ini dicantumkan pada modul berikut:

```
function F_hat = wienerFilter(G, H, K)
% Restorasi spektrum citra menggunakan Wiener Filtering
%
% INPUT:
% G     - Spektrum Fourier 2D citra terdegradasi (G(u,v))
% H     - Fungsi transfer degradasi PSF (H(u,v))
% K     - Parameter konstan Noise-to-Signal Ratio (NSR)
%
% OUTPUT:
% F_hat - Spektrum Fourier 2D citra hasil restorasi

if nargin < 3 || isempty(K)
    K = 0.01;
end
```

```
W = conj(H) ./ (abs(H).^2 + K);

F_hat = G .* W;
end
```

3.7 Implementasi Fungsi Pembantu

Fungsi pembantu berada di direktori src/utils/ dan dipakai bersama oleh seluruh metode. Sub-bab ini membahasnya menurut urutan pemakaian dalam satu proses: grid dan padding, penerapan filter, penyusunan filter gabungan, deteksi noise peaks, dan tuning parameter.

3.7.1 Grid Frekuensi dan Padding

freqGrid.m membentuk grid koordinat frekuensi (U, V) relatif terhadap pusat spektrum hasil fftshift, dengan rentang -floor(Q/2) sampai ceil(Q/2) - 1 untuk Q kolom dan P baris. Semua filter menghitung jarak D(u,v) dari grid ini sehingga posisi filter selalu konsisten dengan spektrum yang sudah dipusatkan.

padImage2x.m menambahkan zero-padding sehingga citra berukuran 2M × 2N dengan citra asli di pojok kiri atas, dan mengembalikan ukuran asli untuk memotong hasil. Perkalian di ranah frekuensi setara dengan konvolusi sirkular, sehingga tanpa padding bagian tepi citra "membungkus" dan memengaruhi sisi seberangnya (wrap-around). Akibatnya diameter filter dan posisi (u, v) pada GUI memakai koordinat spektrum ter-pad.

```
function [U, V] = freqGrid(sizeImg)
P = sizeImg(1);   % baris
Q = sizeImg(2);   % kolom

uRange = -floor(Q/2) : (ceil(Q/2) - 1);
vRange = -floor(P/2) : (ceil(P/2) - 1);
[U, V] = meshgrid(uRange, vRange);
end
```

```
function [imgPadded, origSize] = padImage2x(img)
[rows, cols, channels] = size(img);
origSize = [rows, cols];

paddedRows = rows * 2;
paddedCols = cols * 2;

imgPadded = zeros(paddedRows, paddedCols, channels, 'like', img);
imgPadded(1:rows, 1:cols, :) = img;
end

    D0safe = D0;
    D0safe(D0safe == 0) = eps;
    passShape = 1 ./ (1 + (D ./ D0safe).^(2 * n));   % nilai 1 di garis, gradasi ke
0
end

if isPass
    H = passShape;
    baseline = 0;
else
    H = 1 - passShape;
    baseline = 1;
end

% memutus garis dengan lingkaran
if ~isempty(breakPos) && ~isempty(breakRadius) && breakRadius > 0
    switch lower(orientation)
        case 'vertical'
            % pusat lingkaran: (u=position, v=breakPos)
            Dcircle = sqrt((U - position).^2 + (V - breakPos).^2);
        case 'horizontal'
            % pusat lingkaran: (u=breakPos, v=position)
            Dcircle = sqrt((U - breakPos).^2 + (V - position).^2);
    end

    if breakBlur == 0
        restoreShape = double(Dcircle <= breakRadius);
    else
        nBreak = 1 / breakBlur;
        restoreShape = 1 ./ (1 + (Dcircle ./ breakRadius).^(2 * nBreak));
    end

    H = H .* (1 - restoreShape) + baseline .* restoreShape;
end
end
```

3.7.2 Penerapan Filter dan Titik Masuk Metode

applyFrequencyFilter.m menerapkan filter H terpusat ke citra: citra di-pad 2x, lalu untuk setiap kanal dilakukan fft2, fftshift, perkalian dengan H, ifftshift, ifft2,

pengambilan bagian real, pemotongan ke ukuran asli, dan pembatasan ke rentang 0 sampai 255. Ukuran H harus sama dengan ukuran citra ter-pad; jika berbeda, fungsi menghentikan proses dengan pesan galat.

applyFrequencyMethod.m adalah titik masuk tunggal yang dipanggil GUI. Fungsi ini menerima citra, nama metode (lowpass, highpass, brightness, notch, atau motion), dan struct parameter, lalu memvalidasi parameter, membangun H, memanggil fungsi yang sesuai, dan mengembalikan citra hasil, nama metode, teks parameter, serta struct tambahan (H dan pola derau) untuk tampilan GUI.

```
function imgResult = applyFrequencyFilter(img, H)
[imgPadded, origSize] = padImage2x(img);
sizePad = [size(imgPadded, 1), size(imgPadded, 2)];

if ~isequal(size(H), sizePad)
    error('applyFrequencyFilter:ukuranBeda', ...
        'Ukuran H (%dx%d) harus sama dengan citra ter-pad (%dx%d).', ...
        size(H, 1), size(H, 2), sizePad(1), sizePad(2));
end

imgResult = zeros(size(img));

for c = 1:size(img, 3)
    F = fft2(double(imgPadded(:, :, c)));
    F = fftshift(F);
    G = F .* H;

    g = real(ifft2(ifftshift(G)));
    imgResult(:, :, c) = g(1:origSize(1), 1:origSize(2));   % crop balik
end

imgResult = max(0, min(255, imgResult));
imgResult = uint8(imgResult);
end
```

3.7.3 Penyusun Filter Gabungan (Notch Filter)

buildFilter.m menyatukan beberapa filter menjadi satu H. Masukannya adalah filterList, yaitu daftar struct yang tiap elemennya mewakili satu filter dengan field type (ideal, gaussian, butterworth, band_ideal, band_gaussian, band_butterworth, atau notchline), posisi u0 dan v0, diameter, serta parameter khusus tipe tersebut.

Setiap tipe diteruskan ke fungsi yang sesuai (Sub-bab 3.2, 3.3, dan 3.5). Penggabungan bergantung pada combineMode: reject mengalikan seluruh H_k, sedangkan pass memakai 1 - Π(1 - H_k). Flag isLowPass, isBandPass, dan isPass yang tidak diisi mengikuti combineMode, sehingga satu daftar filter yang sama dapat dipakai untuk reject maupun pass.

```
…

Hreject = ones(sizeImg);

for i = 1:numel(filterList)
    if iscell(filterList), spec = filterList{i}; else, spec = filterList(i); end
    u0 = getFieldOrDefault(spec, 'u0', 0);
    v0 = getFieldOrDefault(spec, 'v0', 0);

    switch lower(spec.type)
        case 'ideal'
            isLowPass = getFieldOrDefault(spec, 'isLowPass', isPassMode);
            Hk = idealPass(sizeImg, u0, v0, spec.diameter, isLowPass);
        case 'gaussian'
            isLowPass = getFieldOrDefault(spec, 'isLowPass', isPassMode);
            Hk = gaussianPass(sizeImg, u0, v0, spec.diameter, isLowPass);
        case 'butterworth'
            isLowPass = getFieldOrDefault(spec, 'isLowPass', isPassMode);
            Hk = butterworthPass(sizeImg, u0, v0, spec.n, spec.diameter, isLowPass);
        case 'band_ideal'
            isBandPass = getFieldOrDefault(spec, 'isBandPass', isPassMode);
            Hk = bandIdeal(sizeImg, u0, v0, spec.diameter, spec.width, isBandPass);
        case 'band_gaussian'
            isBandPass = getFieldOrDefault(spec, 'isBandPass', isPassMode);
            Hk = bandGaussian(sizeImg, u0, v0, spec.diameter, spec.width,
isBandPass);
        case 'band_butterworth'
            isBandPass = getFieldOrDefault(spec, 'isBandPass', isPassMode);
            Hk = bandButterworth(sizeImg, u0, v0, spec.diameter, spec.width, spec.n,
isBandPass);
        case 'notchline'
            isPass      = getFieldOrDefault(spec, 'isPass', isPassMode);
            breakPos    = getFieldOrDefault(spec, 'breakPos', []);
            breakRadius = getFieldOrDefault(spec, 'breakRadius', []);
            breakBlur   = getFieldOrDefault(spec, 'breakBlur', 0);
            Hk = notchLineFilter(sizeImg, spec.orientation, spec.position, ...
                spec.width, spec.blur, isPass, breakPos, breakRadius, breakBlur);
    end

    if isPassMode
        Hreject = Hreject .* (1 - Hk);   % gabungan (union) area yang diloloskan
    else
        Hreject = Hreject .* Hk;         % irisan area yang lolos
    end
end

if isPassMode, H = 1 - Hreject; else, H = Hreject; end
```

3.7.4 Deteksi Noise Peaks

findNoisePeaks.m menemukan peak derau periodik secara otomatis dari spektrum. Citra berwarna dikonversi ke grayscale, lalu dihitung magnitude spektrum skala log, mag = log(|F| + 1), yang sudah dipusatkan dengan fftshift. Area di sekitar pusat dengan radius ExcludeRadius (nilai default 20) dikecualikan karena ditempati komponen DC dan frekuensi rendah yang memang dominan pada citra apa pun. Batas/threshold ditetapkan sebagai rata-rata ditambah ThresholdStd (nilai default 3) kali simpangan baku magnitude di luar area tersebut. Kandidat adalah maksimum lokal (imregionalmax) yang berada di atas batas, diurutkan dari magnitude terbesar, lalu kandidat yang lebih dekat dari MinDistance (nilai default 16) ke peak yang sudah dipilih dibuang (non-maximum suppression) hingga MaxPeaks (nilai awal 10) peak. Setiap peak disimpan sebagai struct berisi row, col, u, v (relatif terhadap pusat), distance, dan magnitude.

plotNoisePeaks.m menampilkan spektrum tersebut dengan sumbu u dan v, lingkaran merah serta nomor urut pada setiap peak, dan tanda + hijau di pusat. Pada GUI, deteksi dijalankan pada citra kerja yang sudah di-pad 2x sehingga (u, v) peak dapat langsung dipakai sebagai posisi filter.

```
…
if size(img, 3) == 3, img = rgb2gray(img); end
img = double(img);

F = fftshift(fft2(img));
mag = log(abs(F) + 1);

[rows, cols] = size(mag);
centerRow = floor(rows/2) + 1;
centerCol = floor(cols/2) + 1;

% exclude area pusat
[X, Y] = meshgrid(1:cols, 1:rows);
```

```
distFromCenter = sqrt((X - centerCol).^2 + (Y - centerRow).^2);
validMask = distFromCenter > opt.ExcludeRadius;

% atur threshold
magValid = mag(validMask);
threshold = mean(magValid) + opt.ThresholdStd * std(magValid);

% cari local maxima di atas threshold
localMax = imregionalmax(mag);
candidateMask = localMax & validMask & (mag > threshold);
[candRows, candCols] = find(candidateMask);
candVals = mag(candidateMask);

% urutkan dari magnitude tertinggi, lalu buang titik yang terlalu berdekatan
[candVals, sortIdx] = sort(candVals, 'descend');
candRows = candRows(sortIdx);
candCols = candCols(sortIdx);

for i = 1:length(candRows)
    if length(selectedRows) >= opt.MaxPeaks, break; end
    d = sqrt((selectedRows - candRows(i)).^2 + (selectedCols - candCols(i)).^2);
    if isempty(selectedRows) || ~any(d < opt.MinDistance)
        selectedRows(end+1, 1) = candRows(i);
        selectedCols(end+1, 1) = candCols(i);
        selectedVals(end+1, 1) = candVals(i);
    end
end

% u = kolom - centerCol, v = baris - centerRow
```

3.7.5 Tuning Parameter dengan Golden-Section Search

Tombol Tune pada GUI memanggil tunePassFilter.m (diameter dan orde n), tuneBrightness.m (target mean), dan tuneMotionRestoration.m (K untuk Wiener atau threshold untuk inverse). Ketiganya memakai kerangka yang sama yaitu mencari satu parameter yang meminimalkan MSE hasil terhadap citra referensi, yang setara dengan PSNR terbesar, dengan **golden-section search**.

Golden-section search bekerja pada interval [a, b] yang diasumsikan hanya memiliki satu minimum. Dua titik dalam dihitung dengan rasio r = (√5 - 1)/2 ≈ 0,618, yaitu x1 = b - r(b - a) dan x2 = a + r(b - a), lalu MSE keduanya dibandingkan. Jika f(x1) ≤ f(x2), minimum berada di [a, x2] sehingga b diganti x2 dan x1 lama menjadi x2 baru; jika tidak, a diganti x1. Setiap langkah (epoch) hanya membutuhkan satu

evaluasi baru dan mempersempit interval menjadi 0,618 kali semula, sehingga 15 epoch (nilai awal) menyisakan sekitar 0,07% dari interval awal. Parameter terbaik diambil dari seluruh kandidat yang pernah dievaluasi, lalu hanya diisikan ke field GUI dan baru diterapkan setelah tombol Terapkan ditekan. Pada tuning filter dan restorasi motion blur, spektrum citra (dan PSF) dihitung satu kali, sedangkan evaluasi tiap kandidat memakai langkah kuantisasi dan clipping yang sama dengan proses sebenarnya sehingga PSNR yang dilaporkan sama dengan PSNR setelah filter diterapkan.

Tabel 3.1 Rentang pencarian parameter pada golden-section search

Parameter	Rentang pencarian	Skala dan catatan
Diameter (lowpass, highpass)	1 sampai diagonal spektrum ter-pad	Linear, dibulatkan ke bilangan bulat
Orde n (Butterworth)	0,5 sampai 2,5	Linear; dicari bergantian dengan diameter (diameter, n, diameter, n)
Target mean (brightness)	0,01 sampai 1	Linear, dibulatkan 4 desimal
K (Wiener)	10^-5 sampai 1	Log10, 3 angka penting
Threshold (inverse)	10^-4 sampai 1	Log10, 3 angka penting

Panjang L dan sudut theta pada restorasi motion blur tidak dicari karena merupakan parameter degradasi yang sudah diketahui. Karena asumsi satu minimum tidak dijamin untuk semua citra, hasil tuning diperlakukan sebagai titik awal yang dapat diubah manual.

```
…

function x = golden(f, a, b)
    r  = (sqrt(5) - 1) / 2;
    x1 = b - r * (b - a);   f1 = f(x1);
    x2 = a + r * (b - a);   f2 = f(x2);

    for langkah = 1:epochs
        if batal, break; end
        if f1 <= f2
            b = x2;  x2 = x1;  f2 = f1;
            x1 = b - r * (b - a);   f1 = f(x1);
        else
            a = x1;  x1 = x2;  f1 = f2;
            x2 = a + r * (b - a);   f2 = f(x2);
        end
    end

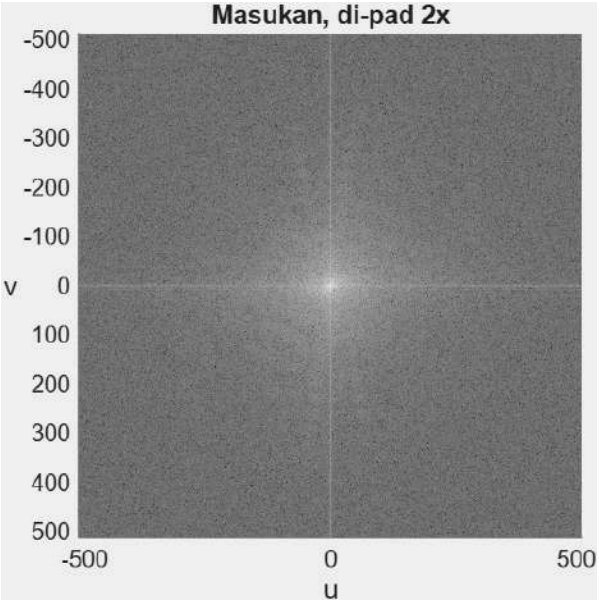
    if f1 <= f2, x = x1; else, x = x2; end
end
```

BAB IV
PENGUJIAN PROGRAM DAN ANALISIS HASIL

4.1 Pengujian *Image Smoothing / Blurring* dalam Ranah Frekuensi

4.1.1 Identifikasi Citra Masukkan

Tabel 4.1 Analisis awal citra masukan `astronaut_color_gaussian_noise.png`

Citra Masukkan	Spektrum Frekuensi Awal
	

Evaluasi Kualitas Citra Terhadap Referensi	
PSNR (dB)	**MSE**
23.61	283.07

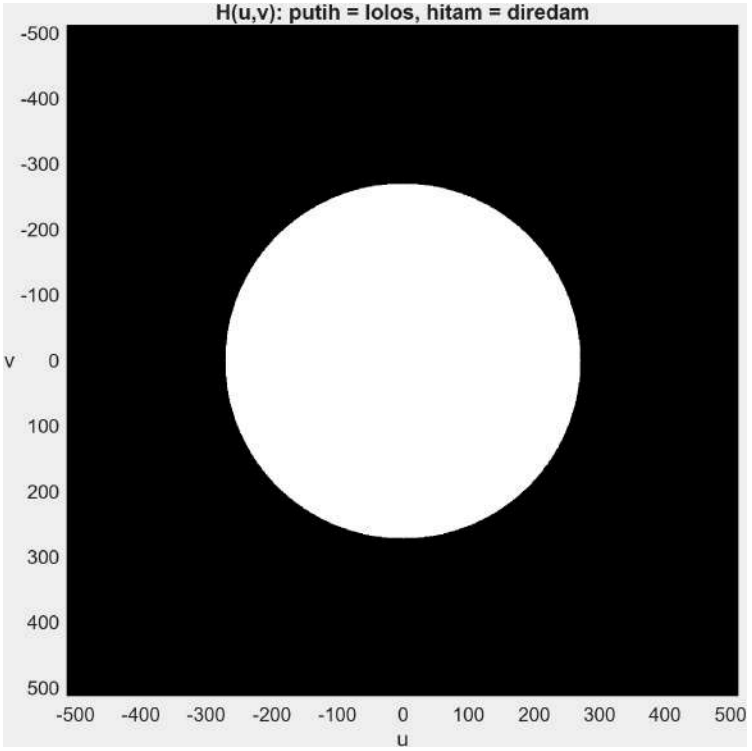
Citra masukan `astronaut_color_gaussian_noise.png` (Tabel 4.1) adalah citra RGB berukuran 512 × 512 piksel yang tercemar derau Gaussian, dengan PSNR terhadap referensi hanya 23,61 dB (MSE 283,07). Derau Gaussian bersifat acak sehingga energinya tersebar merata pada seluruh frekuensi spasial, sedangkan energi sinyal citra terkonsentrasi di frekuensi rendah dekat pusat spektrum. Karena itu,

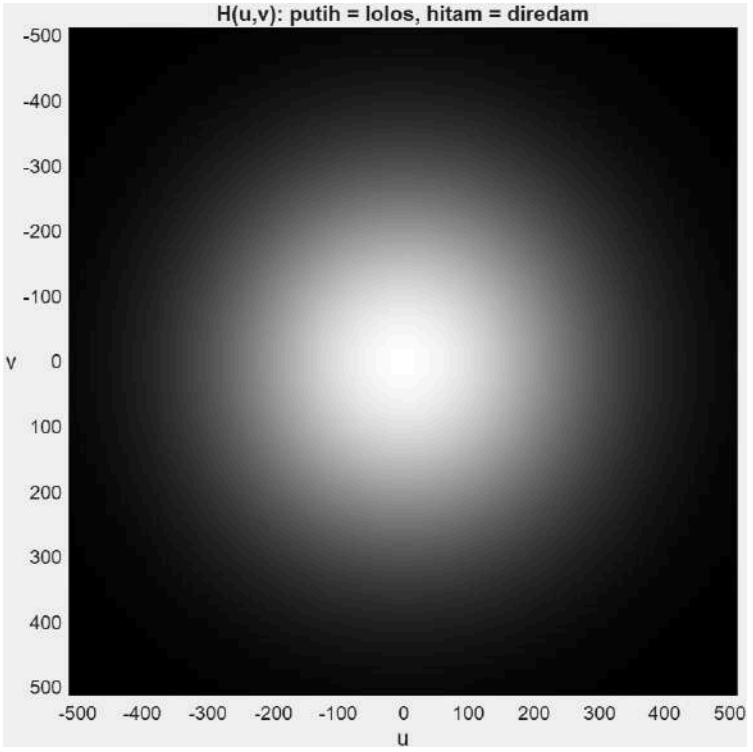
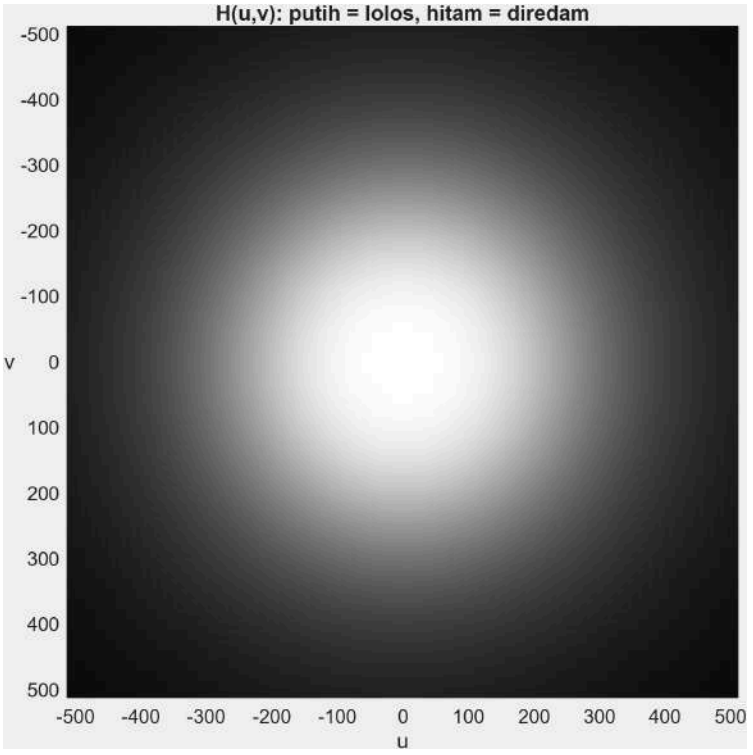
permasalahan ini ditangani dengan low-pass filtering yang meloloskan frekuensi rendah dan meredam frekuensi tinggi, dan tujuan pemrosesannya adalah membandingkan ILPF, GLPF, dan BLPF berdasarkan kedekatan hasilnya terhadap citra referensi.

4.1.2 Pemilihan Metode Pemrosesan

Berdasarkan permasalahan kualitas citra tersebut, dilakukan percobaan pemrosesan dengan metode sebagai berikut:

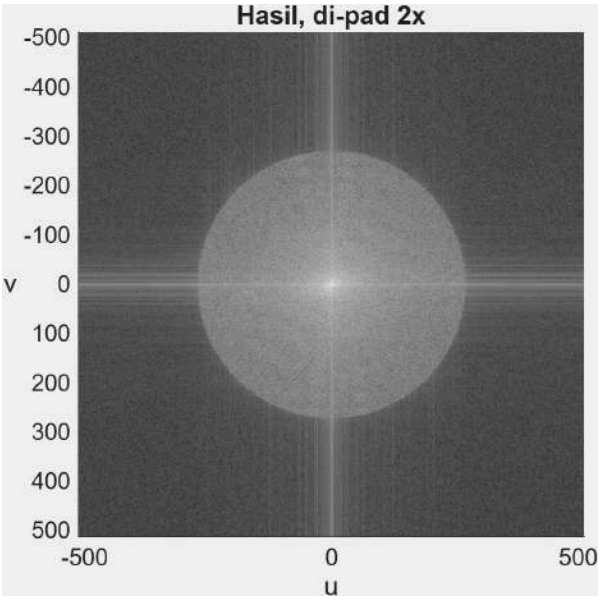
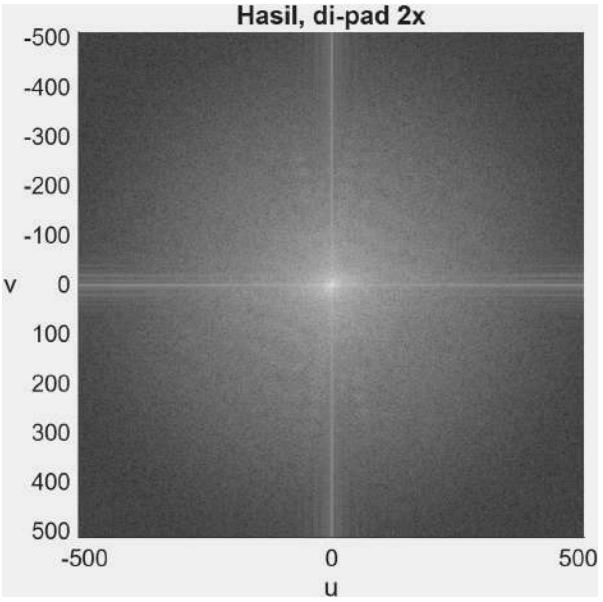
Tabel 4.2 Metode pemrosesan yang diuji untuk image smoothing

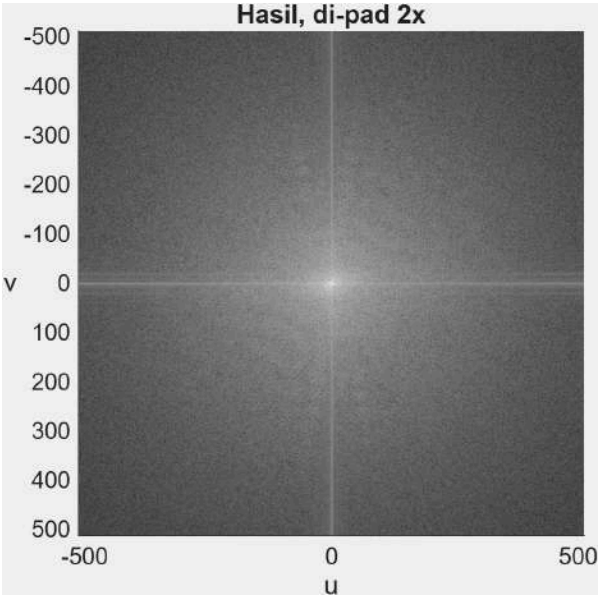
Kode Metode	Nama Metode	Parameter	Mask Filter $H(u, v)$
P1.1	Ideal Pass Filter	Diameter = 542	

P1.2	Gaussian Pass Filter	Diameter = 400	
P1.3	Butterworth Pass Filter	Diameter = 550 Orde n = 1.6	

4.1.3 Analisis Hasil Pemrosesan

Tabel 4.3 Analisis hasil image smoothing

Kode Metode Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir
P1.1		
P1.2		

P1.3		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Metode Rujukan	**PSNR (dB)**	**MSE**
P1.1	27.22	123.47
P1.2	28.08	101.24
P1.3	28.34	95.26

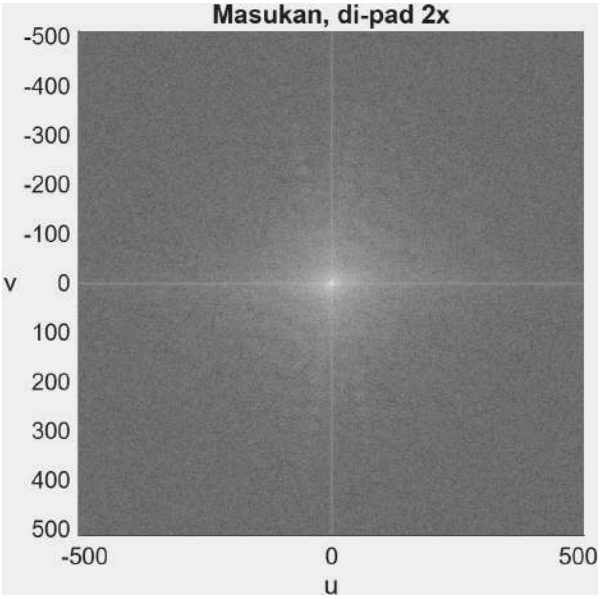
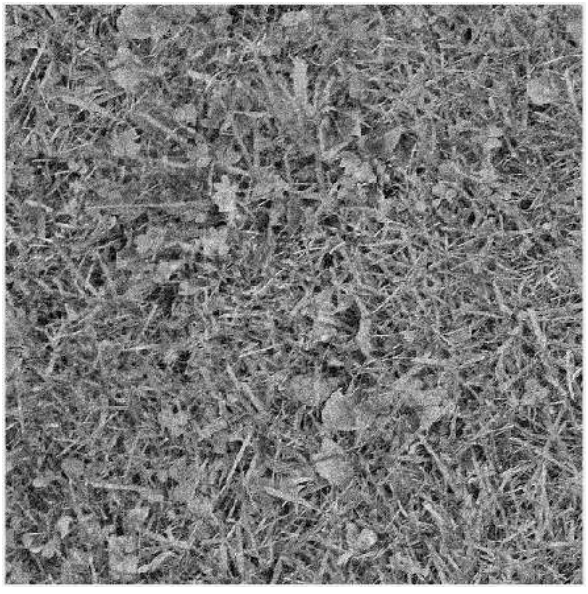
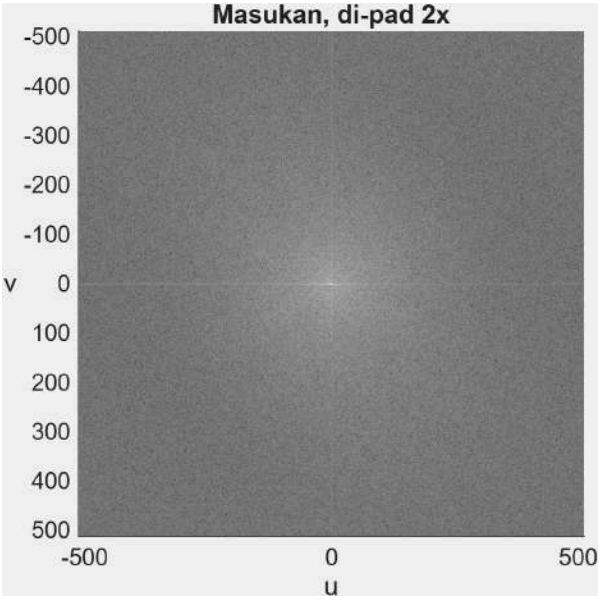
Ketiga penapis menaikkan kualitas citra dengan urutan BLPF, GLPF, lalu ILPF (Tabel 4.3). Selisihnya kecil sehingga hasil ketiganya sulit dibedakan secara visual. Perbedaan karakter lebih jelas pada spektrum akhir, yaitu ILPF memotong spektrum secara tegas pada radius cutoff, GLPF meluruh mulus, dan BLPF berada di antaranya sesuai orde n. ILPF paling rendah karena transisinya diskontinu, sedangkan BLPF unggul karena orde n memberi kurva yang lebih fleksibel.

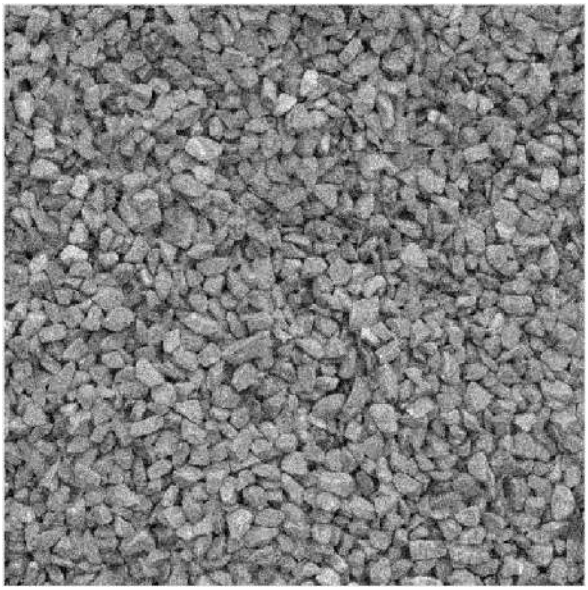
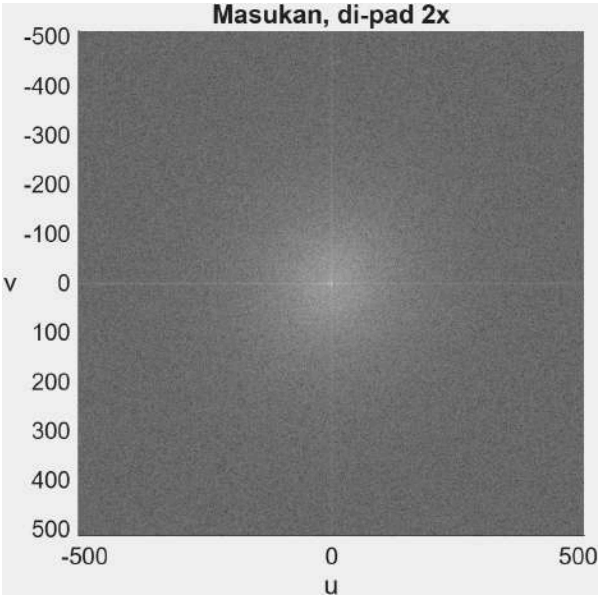
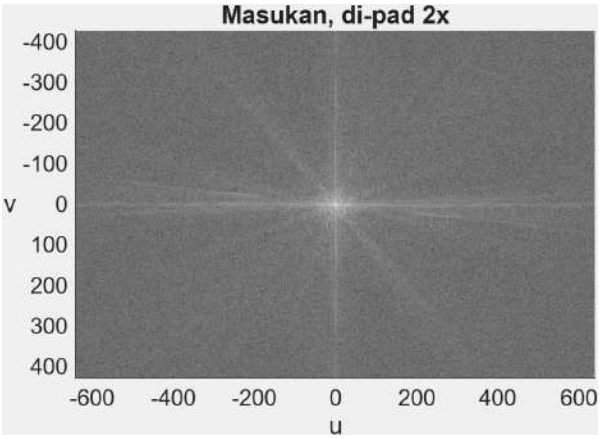
Tidak muncul artefak yang mengganggu pada ketiga hasil, padding 2x mencegah efek *wrap-around* di tepi citra. Keterbatasan utama low-pass filter adalah tidak

membedakan derau dari detail halus, sehingga tepi dan tekstur ikut diredam dan hasil tampak sedikit lebih buram.

4.1.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji

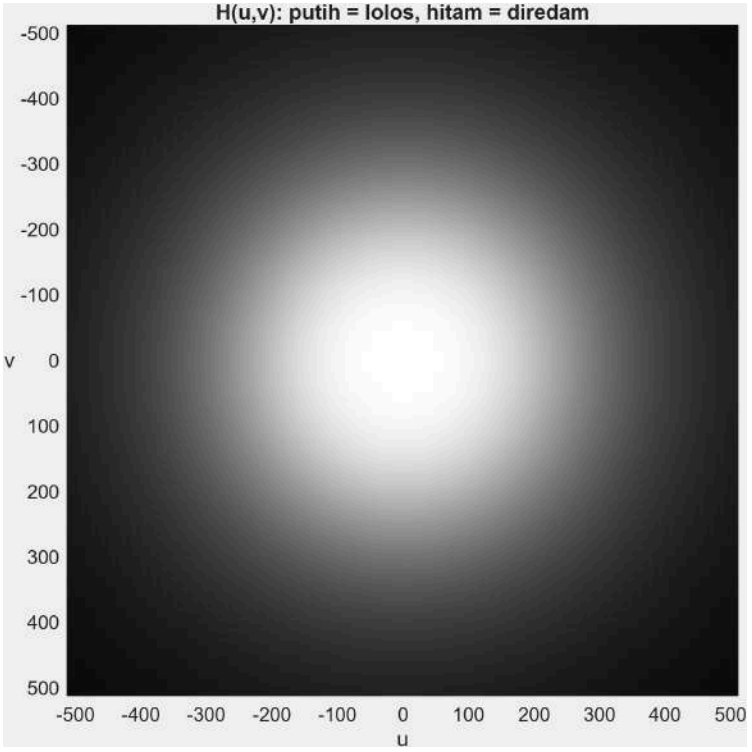
Tabel 4.4 Analisis awal keseluruhan citra masukan untuk image smoothing

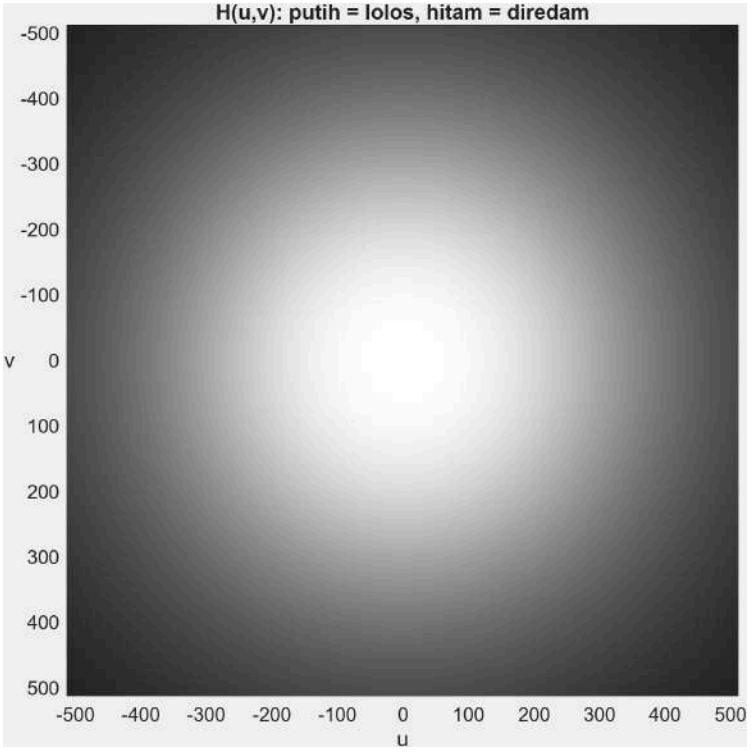
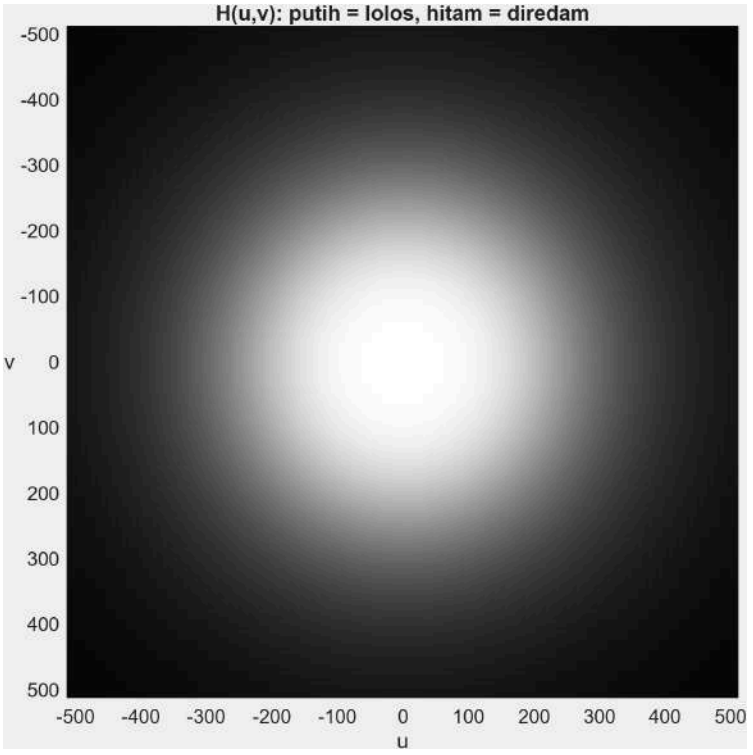
Kode Input	Citra Input	Spektrum Frekuensi Awal
C1.1		
C1.2		

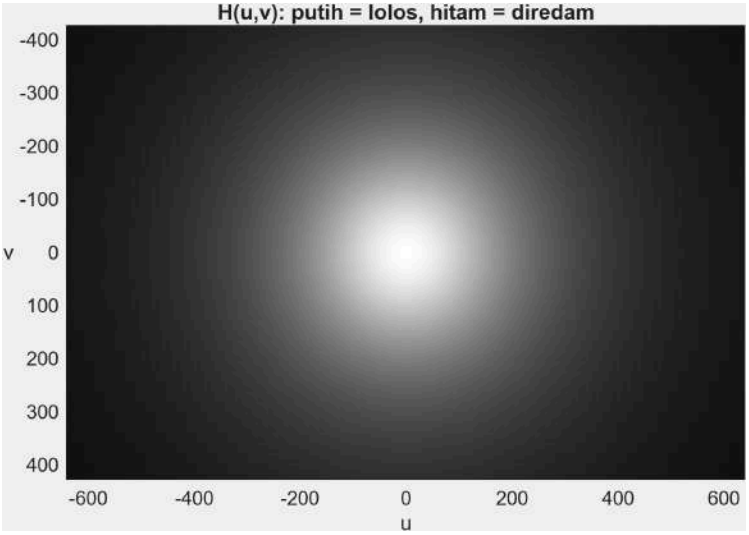
C1.3		
C1.4		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input	**PSNR (dB)**	**MSE**
C1.1	23.61	283.07
C1.2	22.12	398.81
C1.3	21.31	481.37
C1.4	22.29	383.81

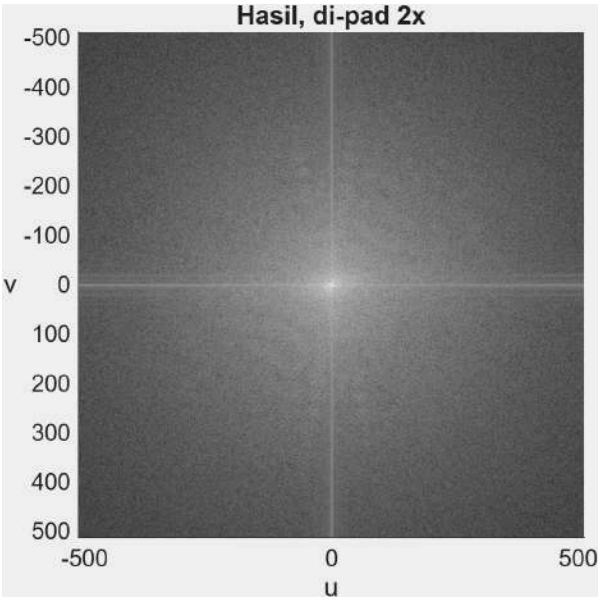
Tabel 4.5 Konfigurasi pemrosesan image smoothing setiap kasus uji

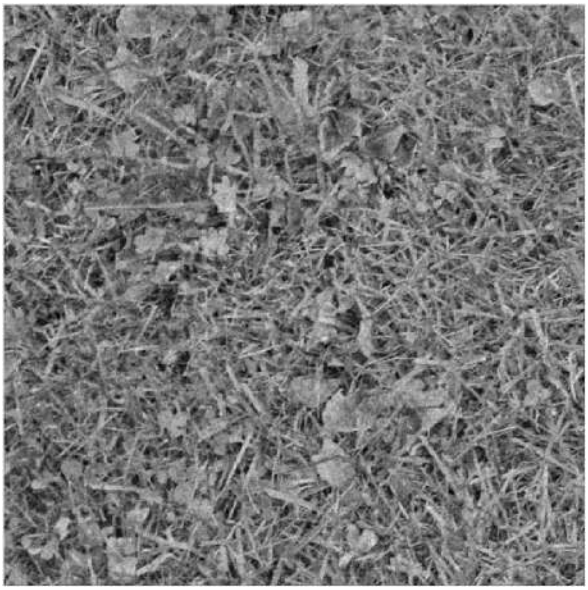
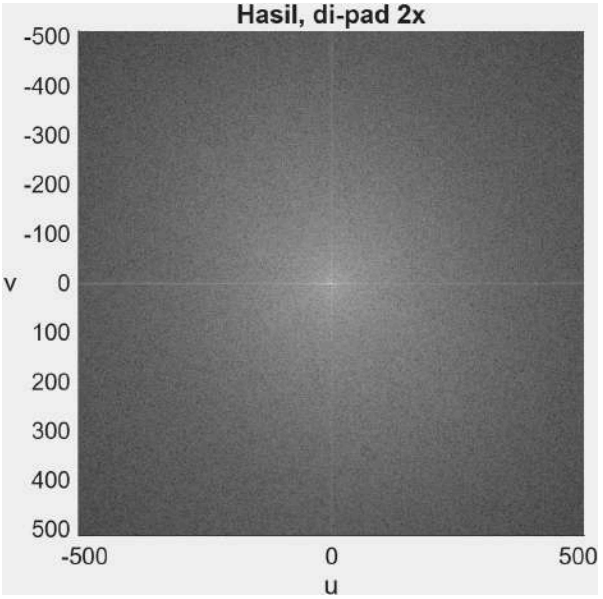
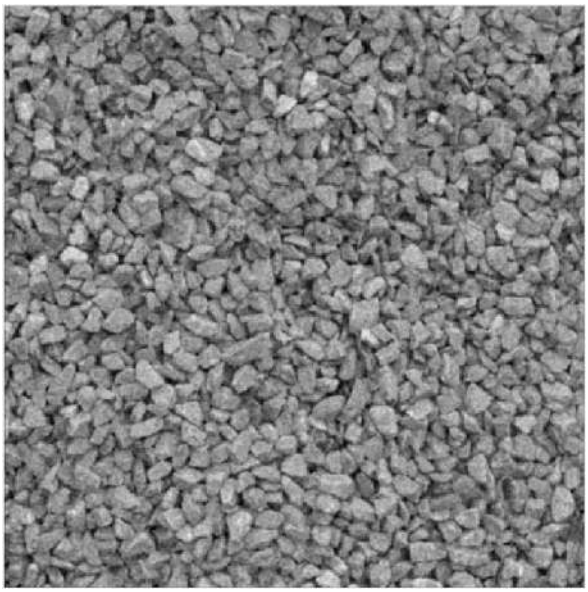
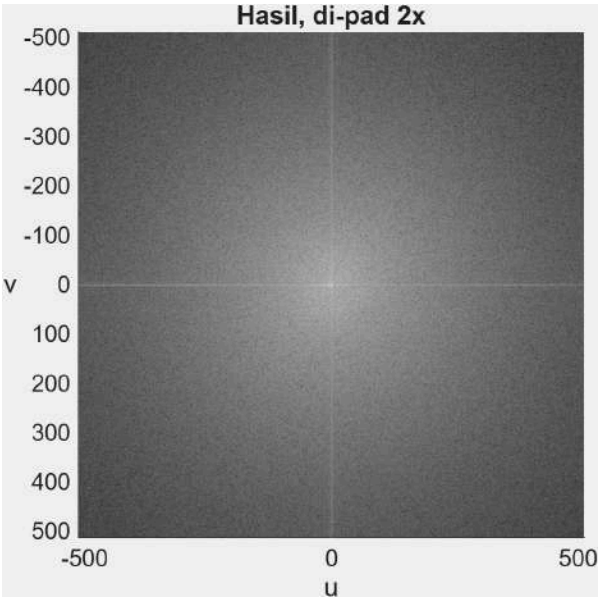
Kode Input Rujukan	**Nama Metode**	**Parameter**	**Mask Filter** $H(u, v)$
C1.1	Butterworth Pass Filter	Diameter = 550 Orde n = 1.6	

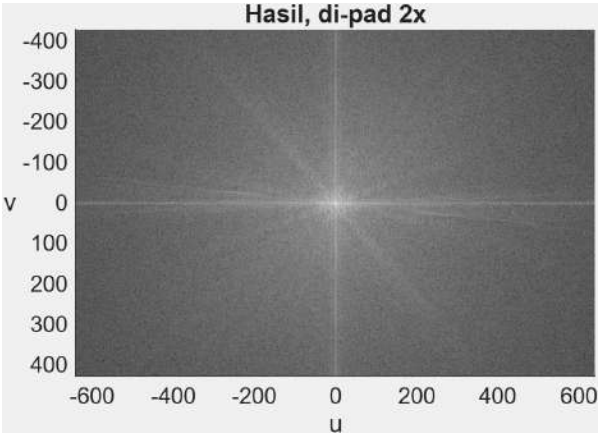
C1.2	Butterworth Pass Filter	Diameter = 767 Orde n = 1.33	
C1.3	Butterworth Pass Filter	Diameter = 546 Orde n = 1.81	

C1.4	Butterworth Pass Filter	Diameter = 395 Orde n = 0.97	

Tabel 4.6 Analisis hasil image smoothing setiap kasus uji

Kode Input Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir
C1.1		

C1.2		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u
C1.3		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u

C1.4		

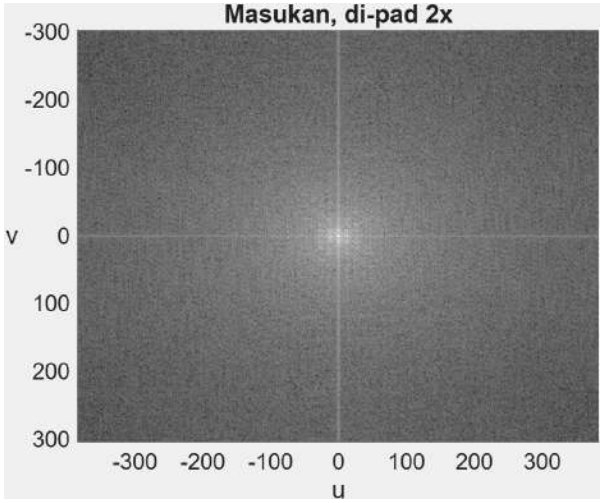
Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input Rujukan	**PSNR (dB)**	**MSE**
C1.1	28.34	95.26
C1.2	24.99	205.99
C1.3	26.44	147.66
C1.4	28.10	100.55

4.2 Pengujian *High-Pass Filtering* pada Ranah Frekuensi

4.2.1 Identifikasi Citra Masukkan

Tabel 4.7 Analisis awal citra masukan `coins_gray.png`

Citra Masukkan	Spektrum Frekuensi Awal

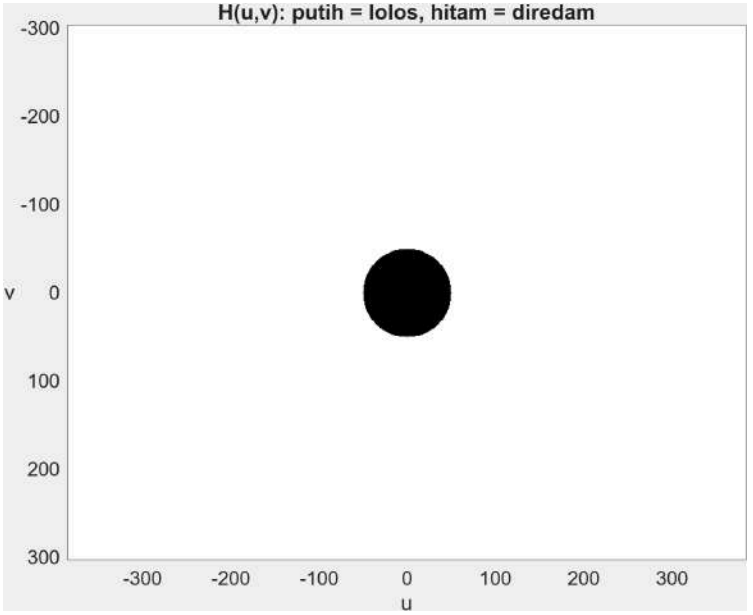
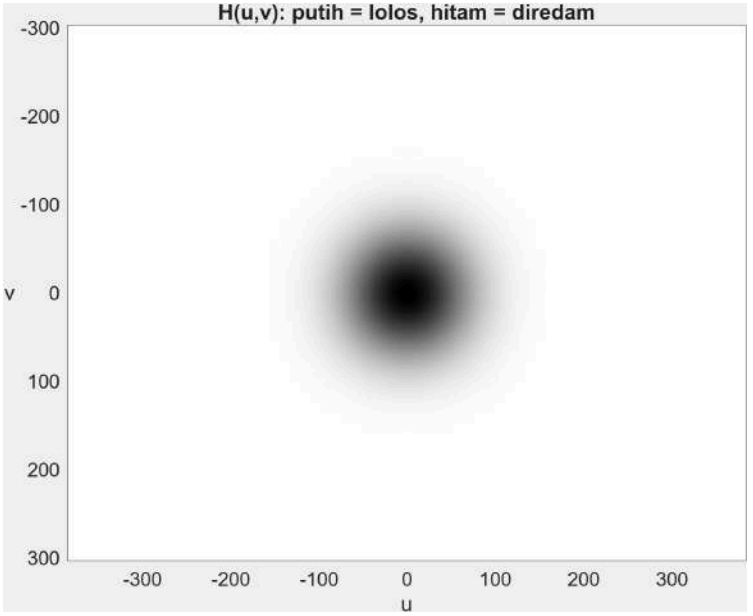
Citra masukan `coins_gray.png` (Tabel 4.7) adalah citra grayscale berisi koin pada latar yang relatif homogen. Bagian ini tidak memiliki citra referensi, sehingga kualitas hasil dinilai secara visual dan melalui spektrum. Tujuan pemrosesannya adalah mengekstrak tepi dan detail halus dengan meredam frekuensi rendah, serta **membandingkan** IHPF, GHPF, dan BHPF pada **diameter yang sama**.

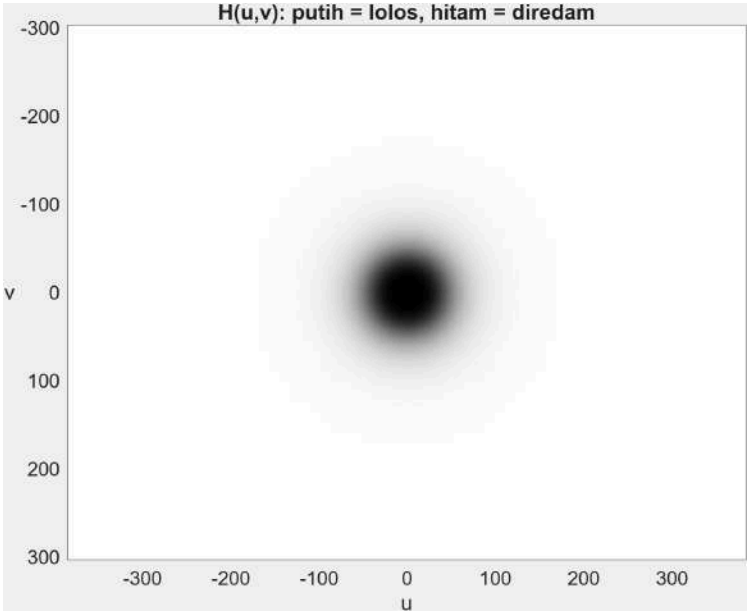
4.2.2 Pemilihan Metode Pemrosesan

Berdasarkan permasalahan kualitas citra tersebut, dilakukan percobaan pemrosesan dengan metode sebagai berikut:

Tabel 4.8 Metode pemrosesan yang diuji untuk high-pass filtering

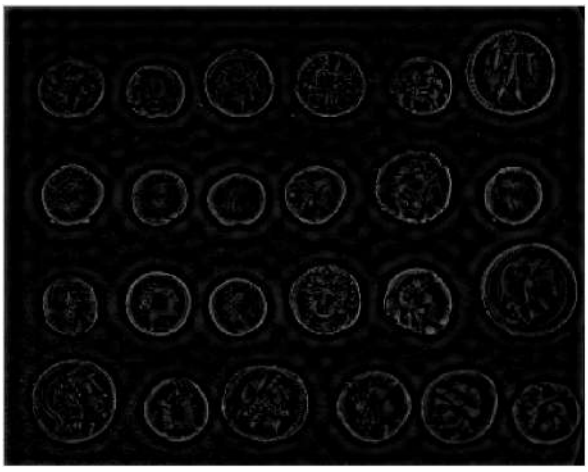
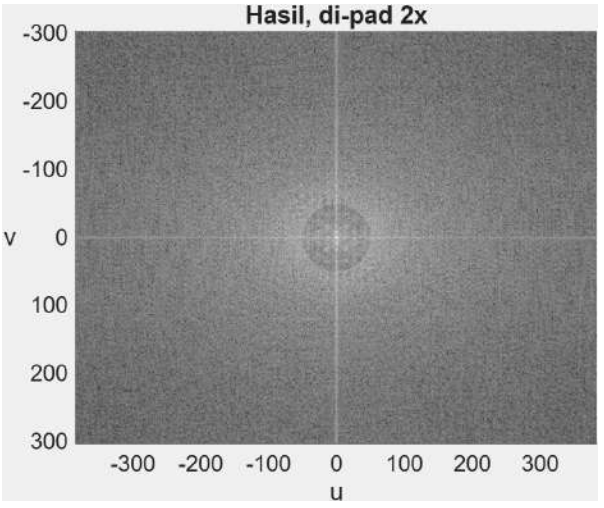
Kode Metode	Nama Metode	Parameter	Mask Filter $H(u, v)$
P2.1	Ideal Highpass Filter	Diameter = 100	

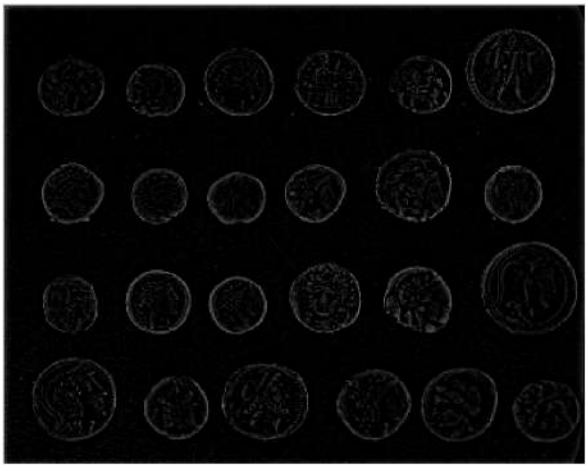
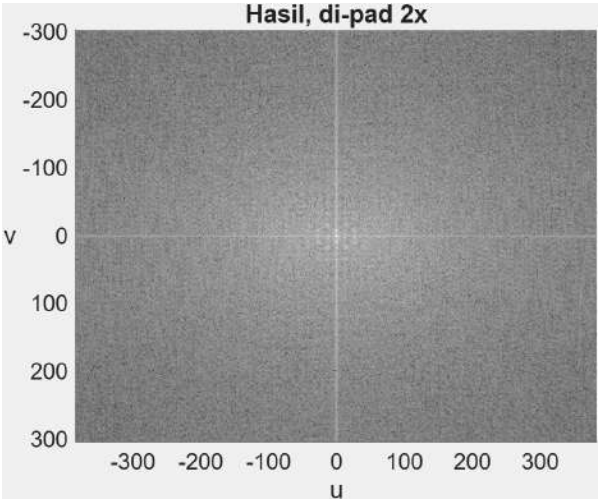
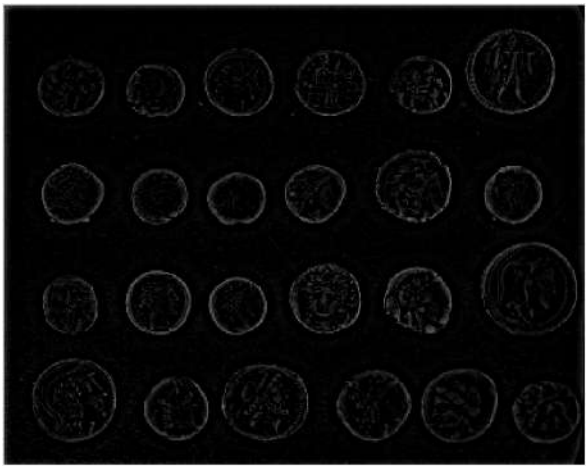
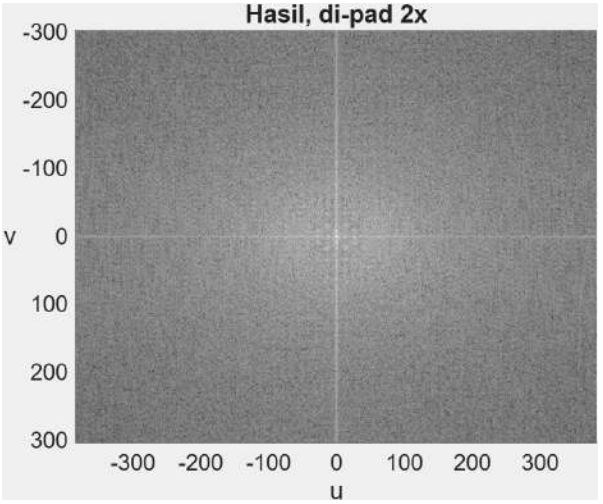
			
P2.2	Gaussian Highpass Filter	Diameter = 100	

P2.3	Butterworth Highpass Filter	Diameter = 100 Orde n = 2	

4.2.3 Analisis Hasil Pemrosesan

Tabel 4.9 Analisis hasil high-pass filtering

Kode Metode Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir
P2.1		

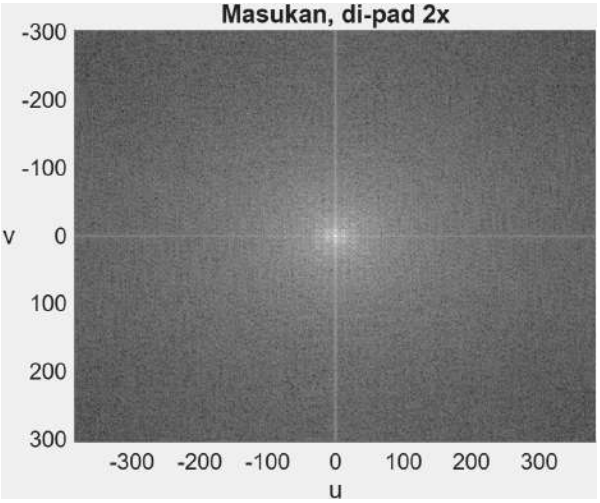
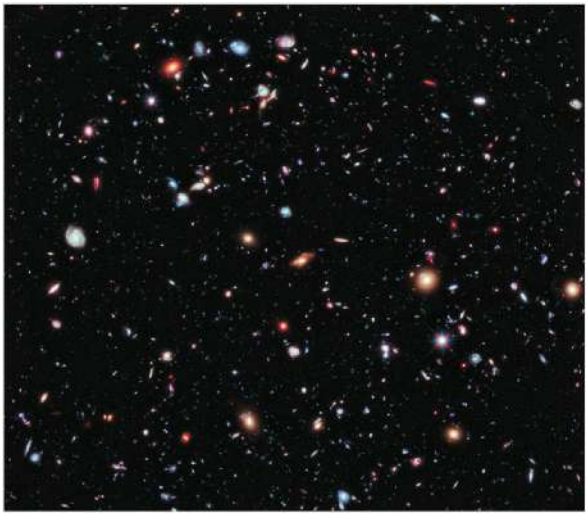
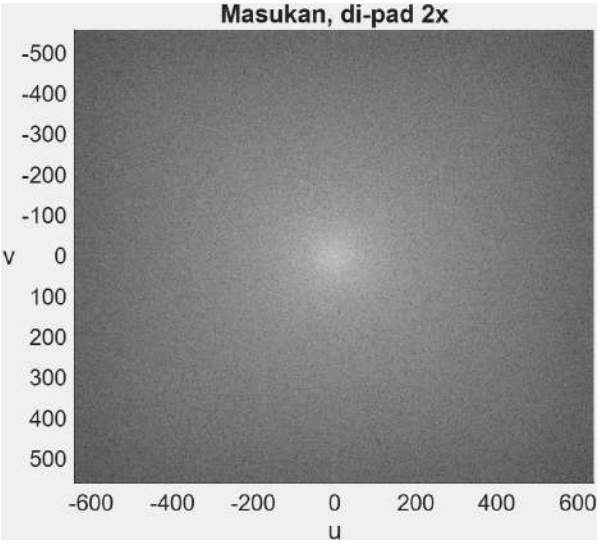
P2.2		
P2.3		

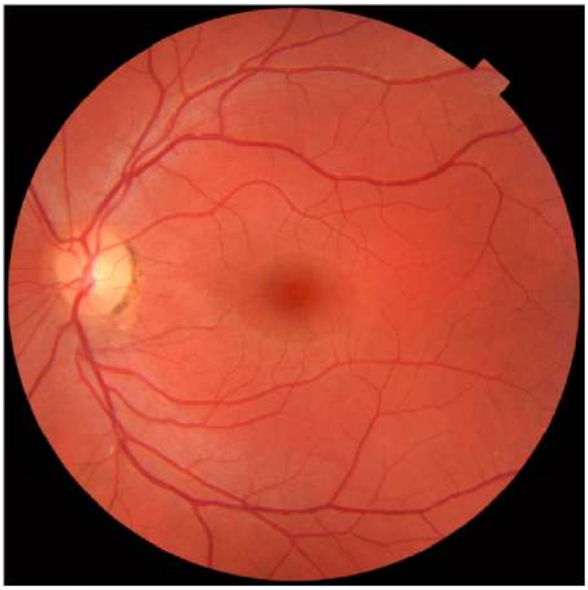
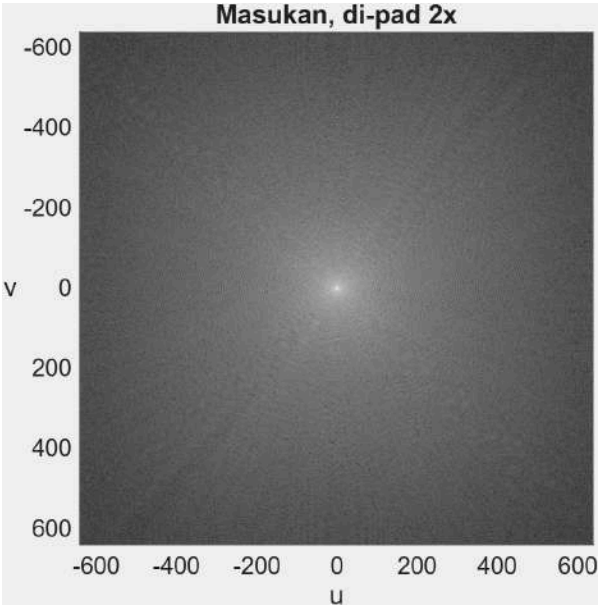
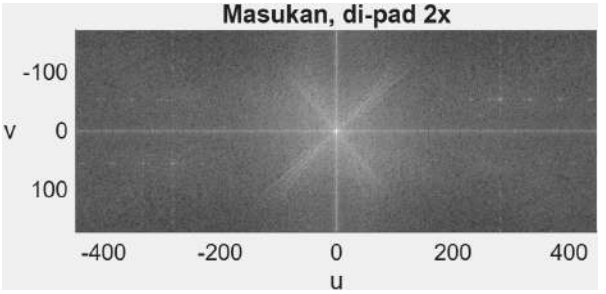
Ketiga penapis menghasilkan citra gelap dengan hanya tepi dan detail halus yang tampak, karena komponen DC dan frekuensi rendah dibuang (Tabel 4.9). **Diameter sengaja disamakan** agar perbedaan hanya bergantung pada bentuk transisi. Perbedaan terutama terlihat pada spektrum akhir, IHPF membuang daerah di dalam cutoff secara tegas, GHPF melemahkannya secara mulus, dan BHPF berada di antaranya, sedangkan pada citra hasil ketiganya sulit dibedakan.

Tidak tampak artefak yang jelas, termasuk ringing pada IHPF, padding 2x mencegah efek pembungkusan di tepi citra. Keterbatasannya adalah hilangnya informasi intensitas rata-rata (latar menjadi hitam) dan derau frekuensi tinggi yang ikut menguat.

4.2.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji

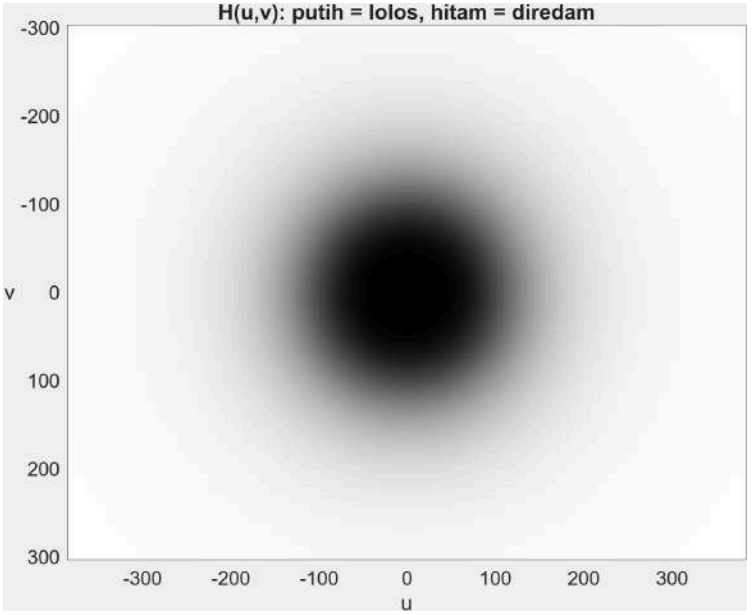
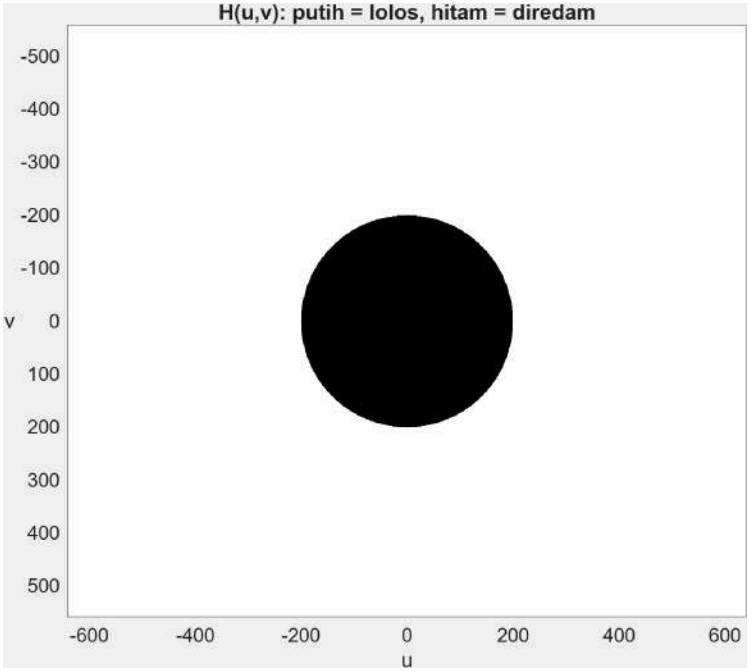
Tabel 4.10 Analisis awal keseluruhan citra masukan untuk high-pass filtering

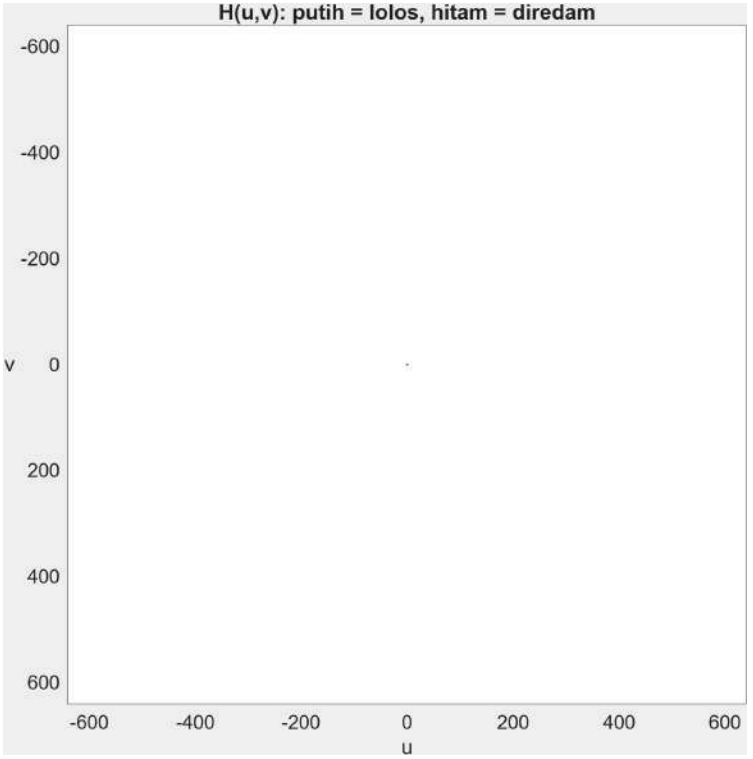
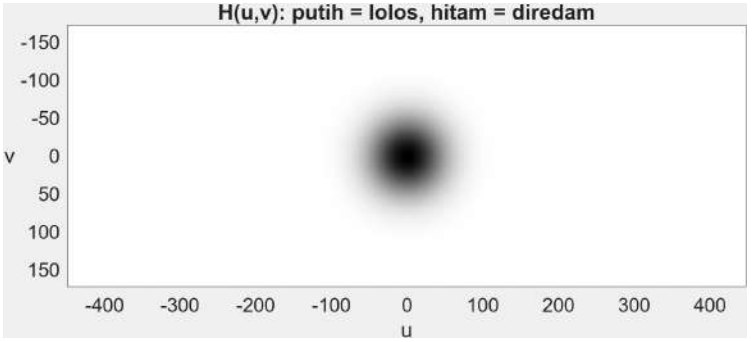
Kode Input	Citra Input	Spektrum Frekuensi Awal
C2.1		
C2.2		

C2.3		
C2.4		

Tabel 4.11 Konfigurasi pemrosesan high-pass filtering setiap kasus uji

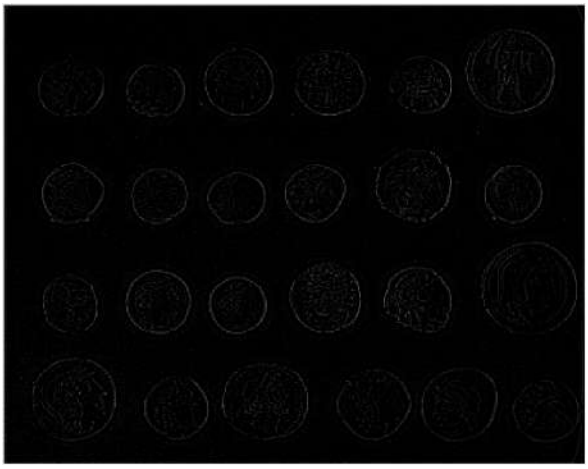
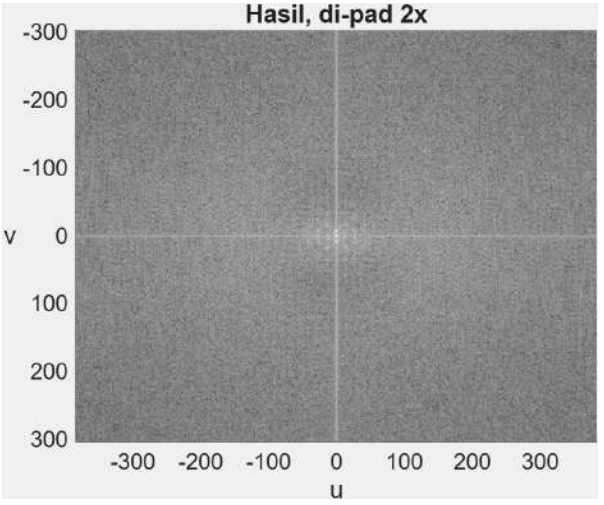
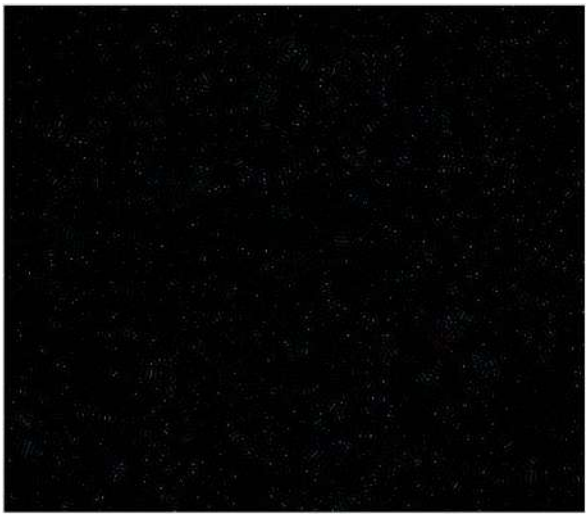
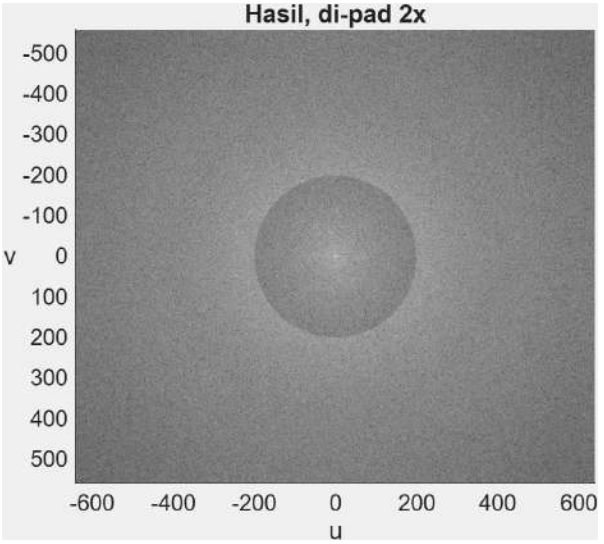
Kode Input Rujukan	**Nama Metode**	**Parameter**	**Mask Filter** $H(u, v)$

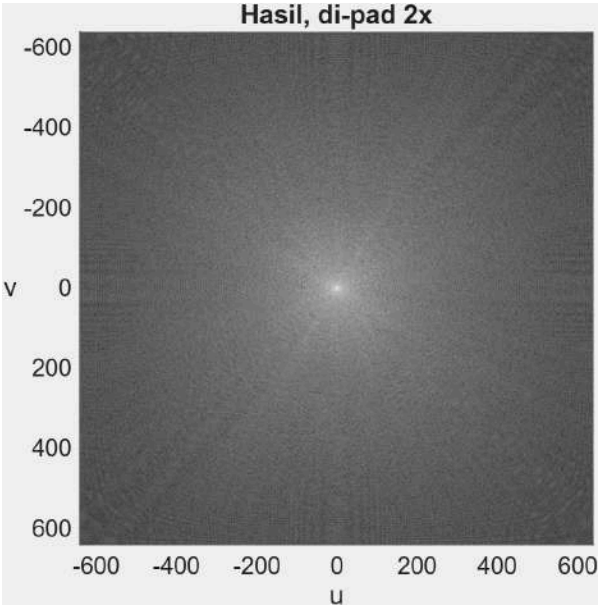
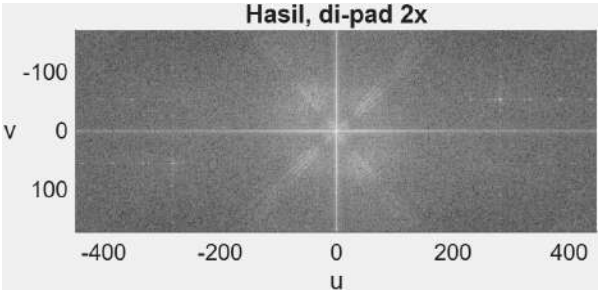
C2.1	Butterworth Highpass Filter	Diameter = 250 Orde = 2	
C2.2	Ideal Highpass Filter	Diameter = 400	

C2.3	Butterworth Highpass Filter	Diameter = 3 Orde n = 3	
C2.4	Gaussian Highpass Filter	Diameter = 70	

Tabel 4.12 Analisis hasil high-pass filtering setiap kasus uji

Kode Input Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir

C2.1		 Hasil, di-pad 2x -300 -200 -100 v 0 100 200 300 -300 -200 -100 0 100 200 300 u
C2.2		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -600 -400 -200 0 200 400 600 u

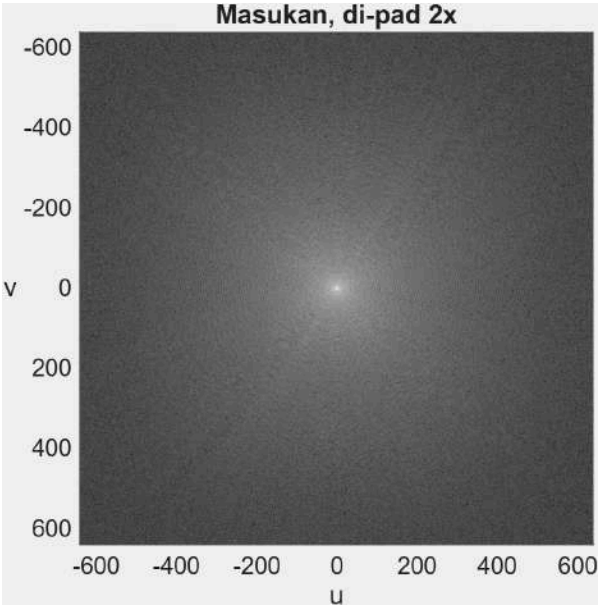
C2.3	 Ini diasumsikan yang highpass adalah uratnya agar lebih visible bukan hanya pinggiran tepi.	
C2.4		

4.3 Pengujian Peningkatan Kecerahan Citra pada Ranah Frekuensi

4.3.1 Identifikasi Citra Masukkan

Tabel 4.13 Analisis awal citra masukan `retina_color_dark.png`

Citra Masukkan	Spektrum Frekuensi Awal

	
Evaluasi Kualitas Citra Terhadap Referensi	
PSNR (dB)	**MSE**
10.02	**6473.36**

Citra masukan `retina_color_dark.png` (Tabel 4.13) adalah citra RGB yang gelap secara keseluruhan. Kecerahan rata-rata citra berbanding lurus dengan komponen DC F(0,0), sehingga citra gelap memiliki F(0,0) yang kecil. Tujuan pemrosesannya adalah menaikkan F(0,0) lewat perkalian spektrum agar rata-rata kecerahan hasil sedekat mungkin dengan citra referensi.

4.3.2 Pemilihan Metode Pemrosesan

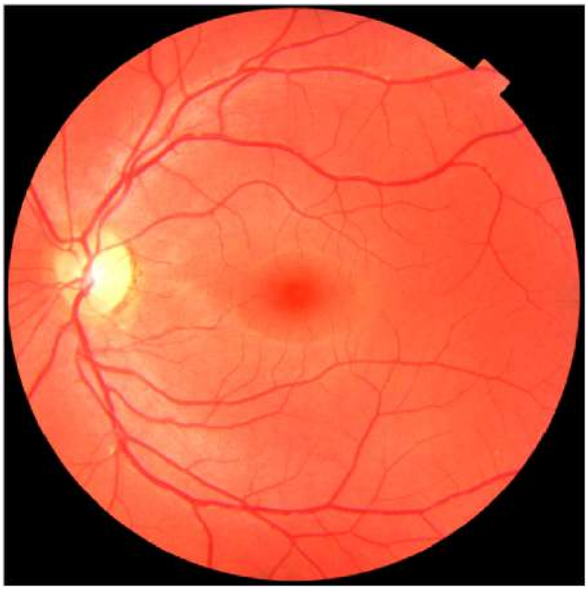
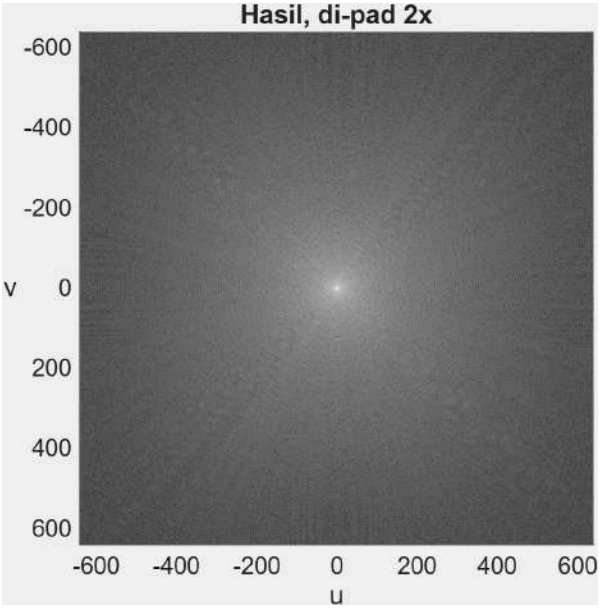
Berdasarkan permasalahan kualitas citra tersebut, dilakukan percobaan pemrosesan dengan metode sebagai berikut:

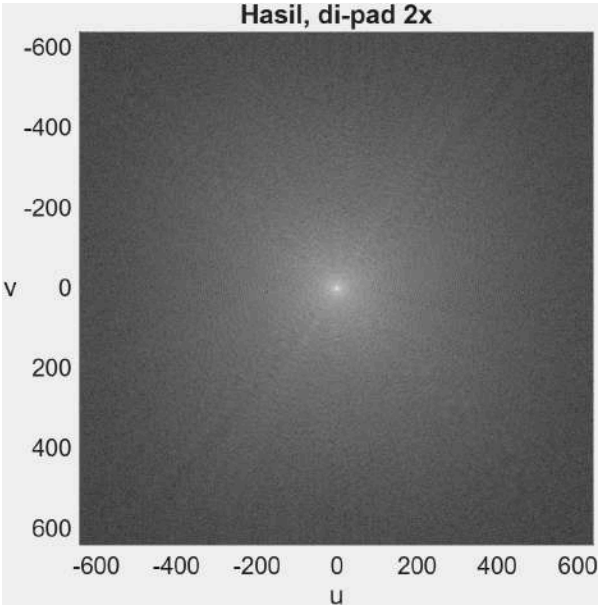
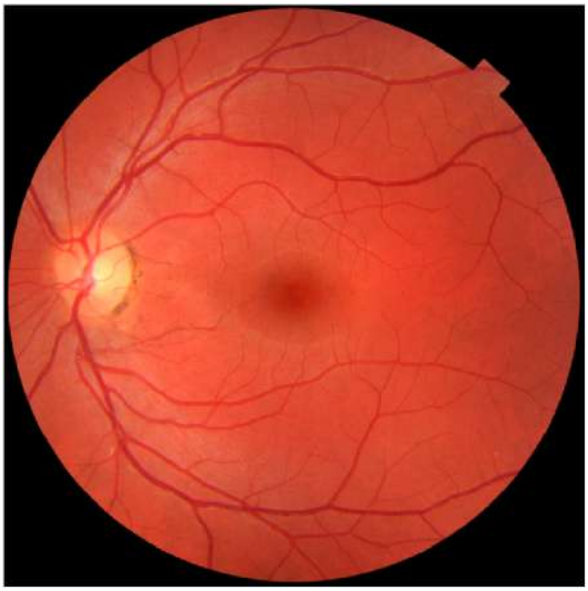
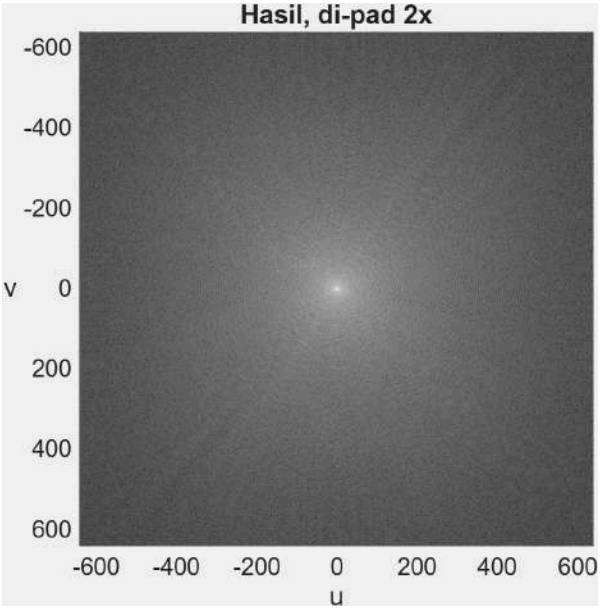
Tabel 4.14 Metode pemrosesan yang diuji untuk peningkatan kecerahan

Kode Metode	Nama Metode	Parameter
P3.1	Brightness	Target mean = 0.5
P3.2	Brightness	Target mean = 0.2
P3.3	Brightness	Target mean = 0.3359

4.3.3 Analisis Hasil Pemrosesan

Tabel 4.15 Analisis hasil peningkatan kecerahan

Kode Metode Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir
P3.1		

P3.2		
P3.3		

Evaluasi Kualitas Citra Terhadap Referensi

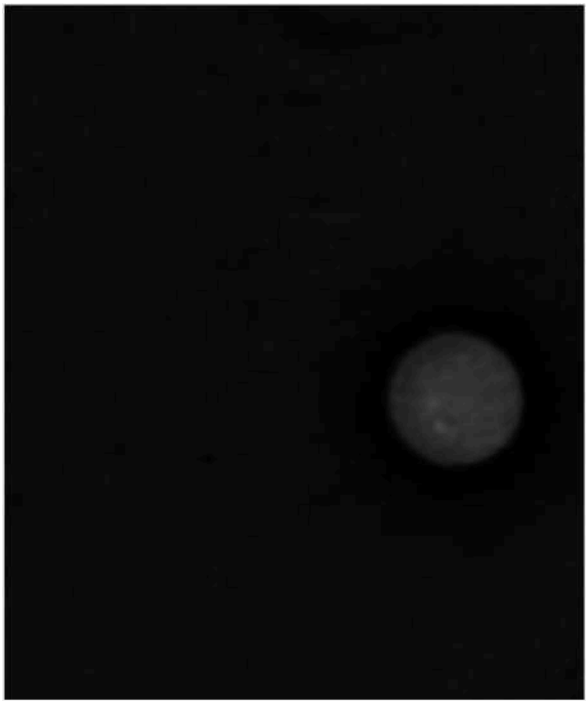
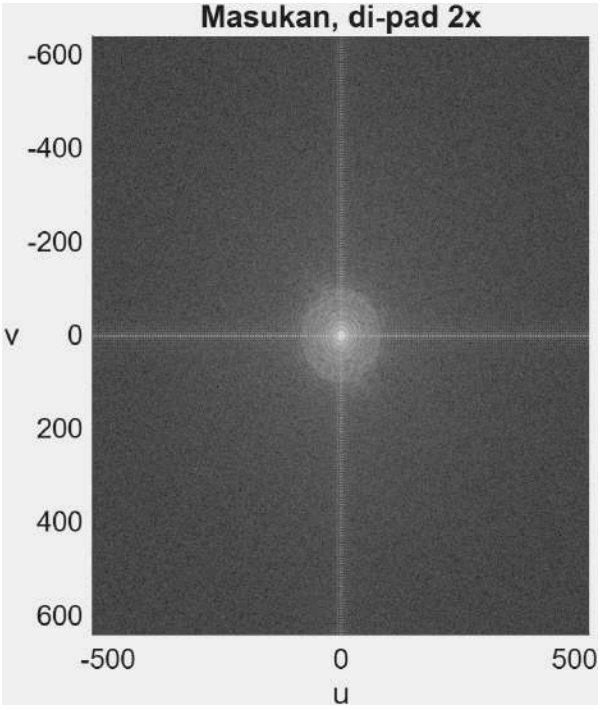
Kode Metode Rujukan	PSNR (dB)	MSE
P3.1	18.53	911.65
P3.2	14.45	2336.06
P3.3	30.42	59.06

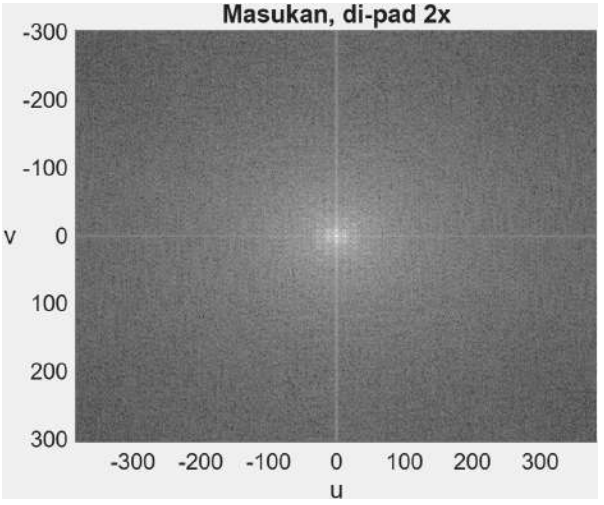
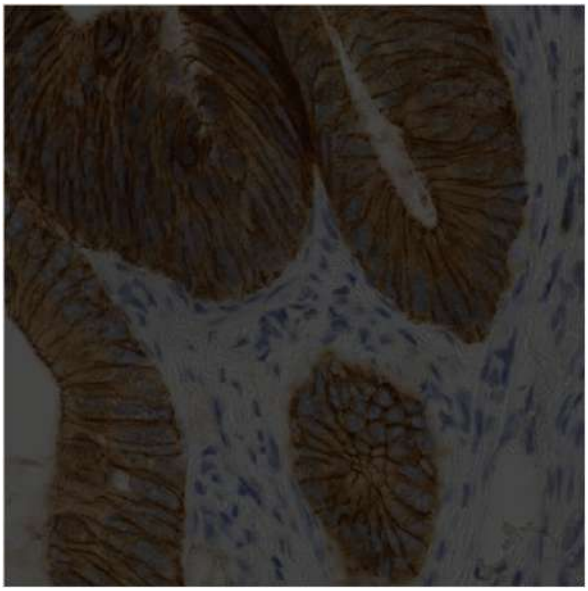
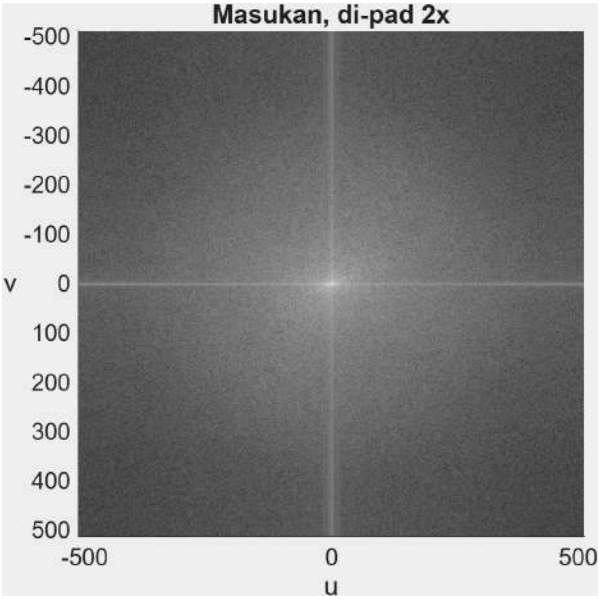
Target 0,5 membuat citra terlalu terang dan target 0,2 masih terlalu gelap, sehingga keduanya jauh dari referensi (Tabel 4.15). Target 0,3359 hasil Tune paling dekat dengan referensi dan memberi peningkatan PSNR terbesar. Spektrum akhir berbentuk sama dengan spektrum masukan dengan magnitude yang naik, karena seluruh spektrum dikalikan α yang sama.

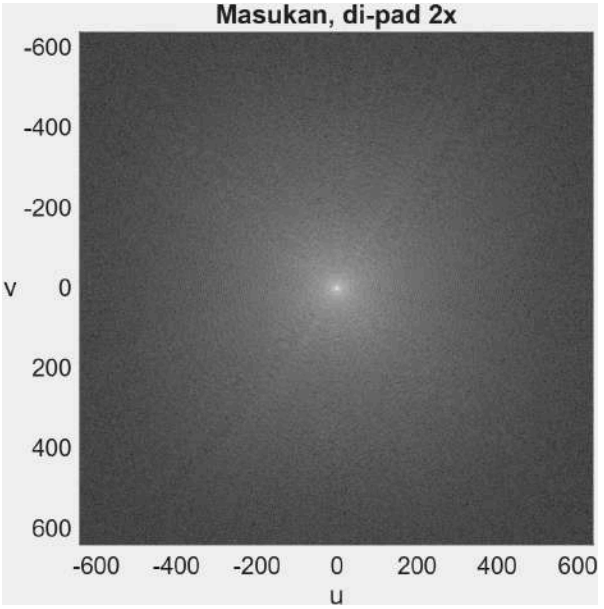
Keterbatasannya, α bersifat global dan linear sehingga kontras tidak diperbaiki secara adaptif dan derau ikut diperkuat. Pada target yang terlalu tinggi, piksel di atas batas dipotong (saturasi) sehingga detail area terang hilang dan rata-rata hasil tidak sama dengan target.

4.3.4 Rekapitulasi Pemrosesan Keseluruhan Citra Uji

Tabel 4.16 Analisis awal keseluruhan citra masukan untuk peningkatan kecerahan

Kode Input	Citra Input	Spektrum Frekuensi Awal
C3.1		

C3.2		 Masukan, di-pad 2x -300 -200 -100 v 0 100 200 300 -300 -200 -100 0 100 200 300 u
C3.3		 Masukan, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u

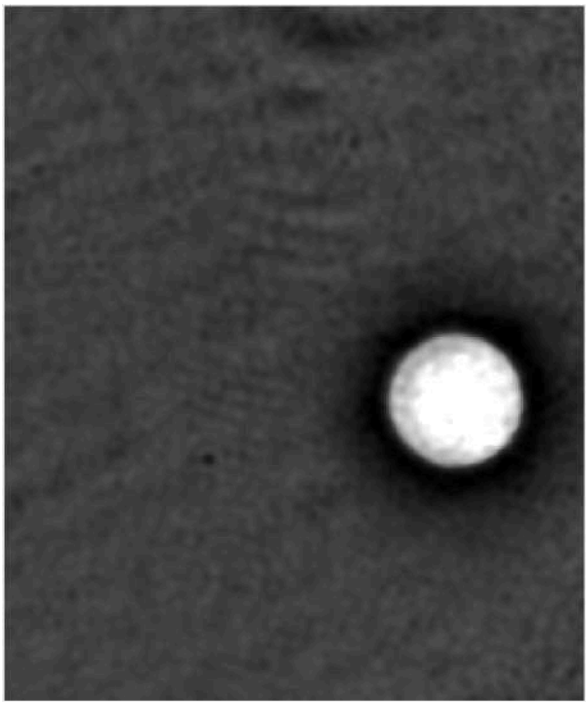
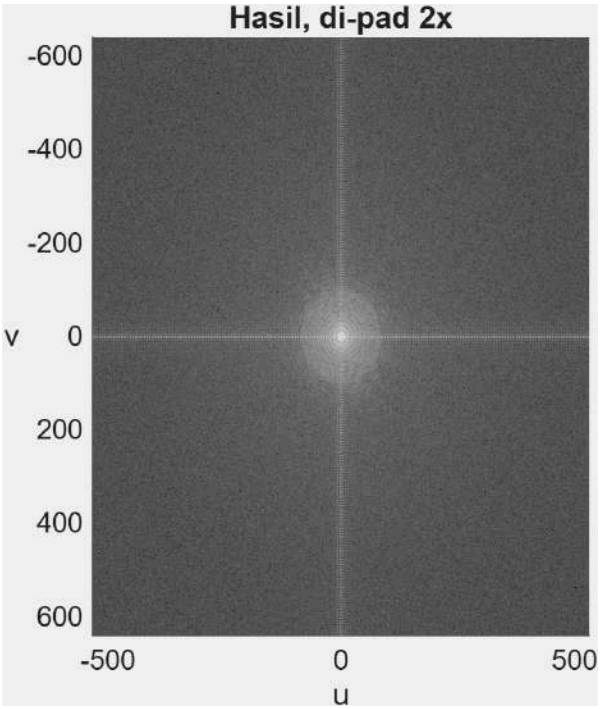
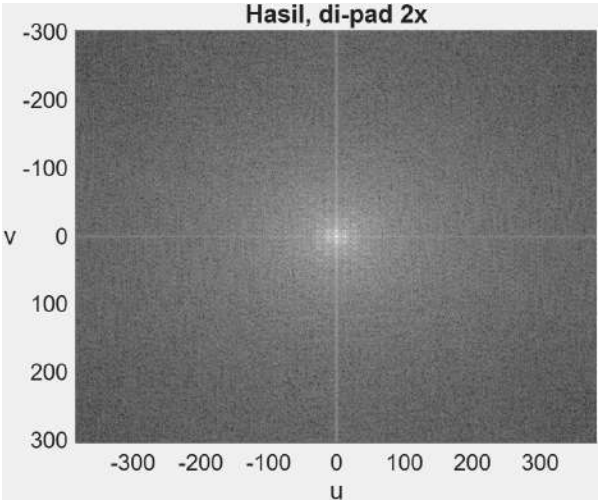
C3.4		

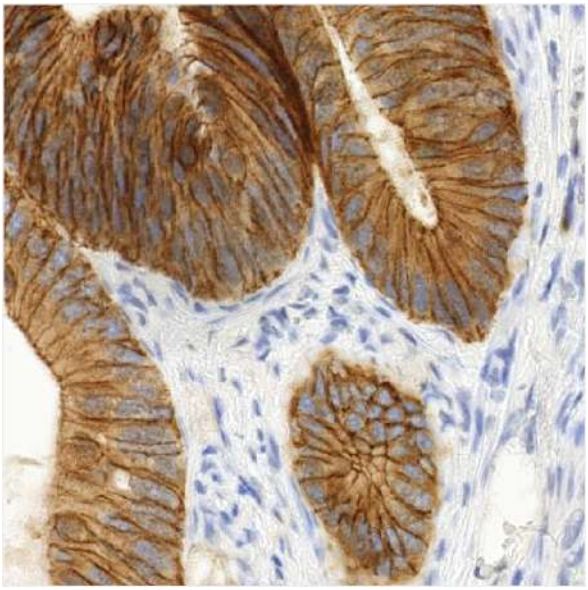
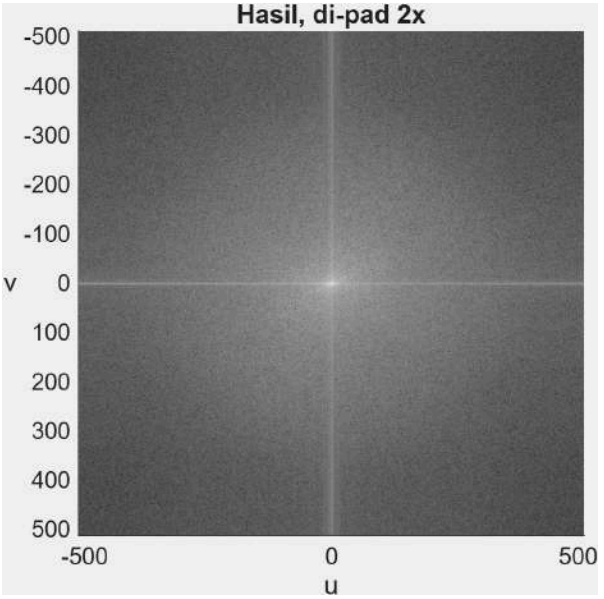
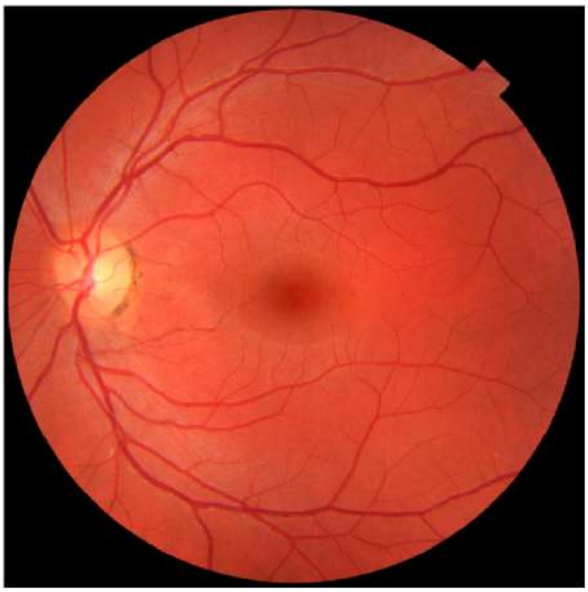
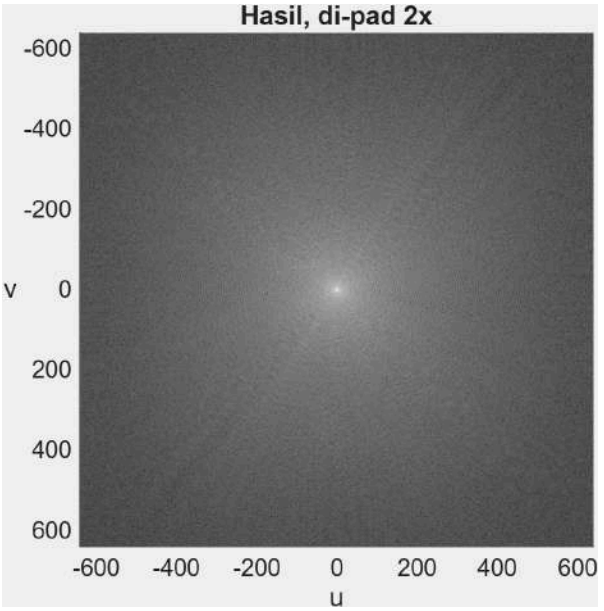
Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input	**PSNR (dB)**	**MSE**
C3.1	12.86	3363.02
C3.2	10.61	5651.57
C3.3	6.36	15022.41
C3.4	10.02	6473.36

Tabel 4.17 Konfigurasi pemrosesan peningkatan kecerahan setiap kasus uji

Kode Input Rujukan	Nama Metode	Parameter
C3.1	Brightness	Target mean = 0.2586
C3.2	Brightness	Target mean = 0.3624
C3.3	Brightness	Target mean = 0.6131
C3.4	Brightness	Target mean = 0.3359

Tabel 4.18 Analisis hasil peningkatan kecerahan setiap kasus uji

Kode Input Rujukan	Citra Hasil Pemrosesan	Spektrum Frekuensi Akhir
C3.1		
C3.2		

C3.3		
C3.4		

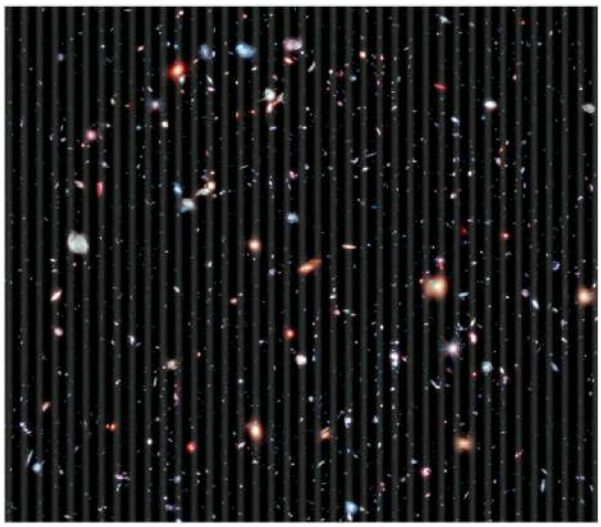
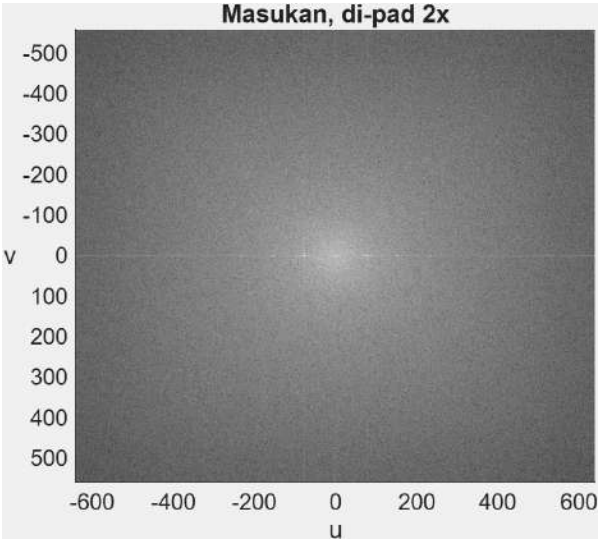
Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input Rujukan	**PSNR (dB)**	**MSE**
C3.1	28.31	96.00
C3.2	29.79	68.26
C3.3	27.29	121.45

C3.4	30.42	59.06

4.4 Pengujian Restorasi Derau Periodik

4.4.1 Identifikasi Citra Terdegradasi

Tabel 4.19 Analisis awal citra terdegradasi `hubble_color_periodic_noise.png`

Citra Terdegradasi	**Spektrum Frekuensi Awal**
	
Evaluasi Kualitas Citra Terhadap Referensi	
PSNR (dB)	**MSE**
22.39	375.00

Citra terdegradasi `hubble_color_periodic_noise.png` (Tabel 4.19) memiliki garis-garis vertikal berulang. Pada spektrum, pola ini muncul sebagai empat peak terang pada sumbu horizontal (v = 0), yaitu pasangan simetris u = ±77 beserta harmonisanya u = ±154. Karena derau terlokalisasi pada titik-titik tersebut, tujuannya adalah membuangnya dengan tiga pendekatan yang dibandingkan:

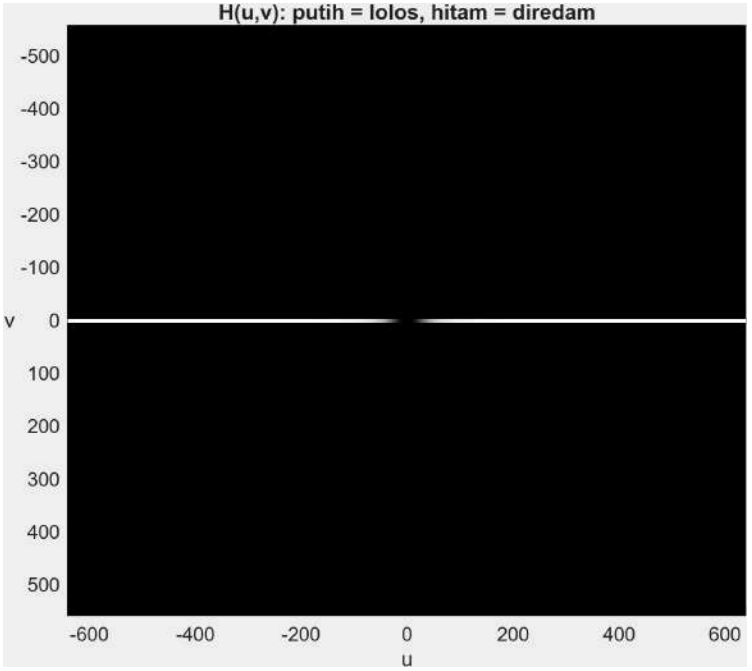
- notch pass garis horizontal yang polanya dikurangkan dari citra,

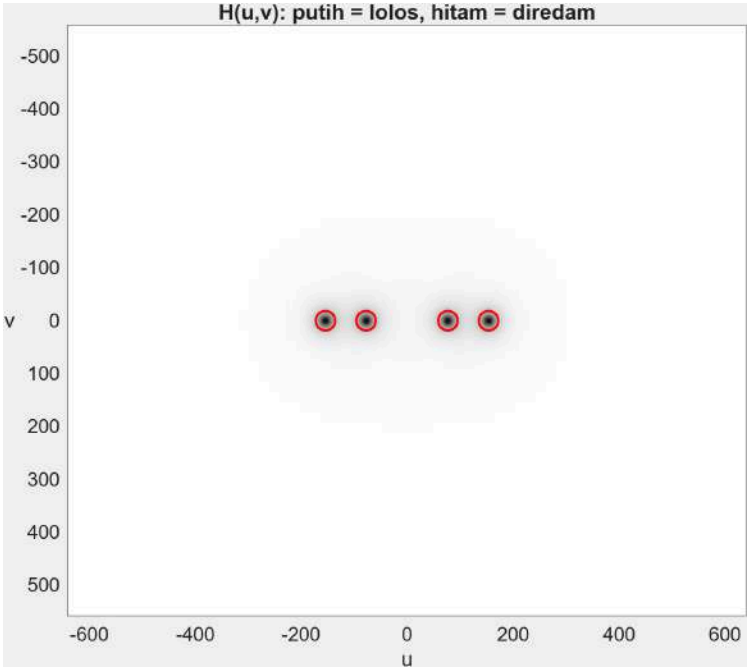
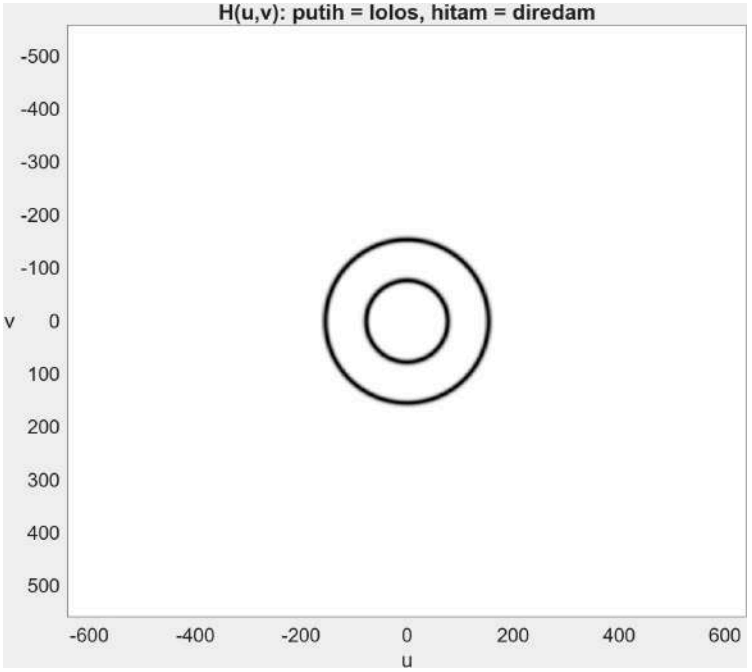
- notch reject Butterworth pada keempat peak, dan
- band-reject Butterworth pada kedua jari-jari.

4.4.2 Pemilihan Metode Restorasi

Berdasarkan permasalahan kualitas citra tersebut, dilakukan percobaan pemrosesan dengan metode sebagai berikut:

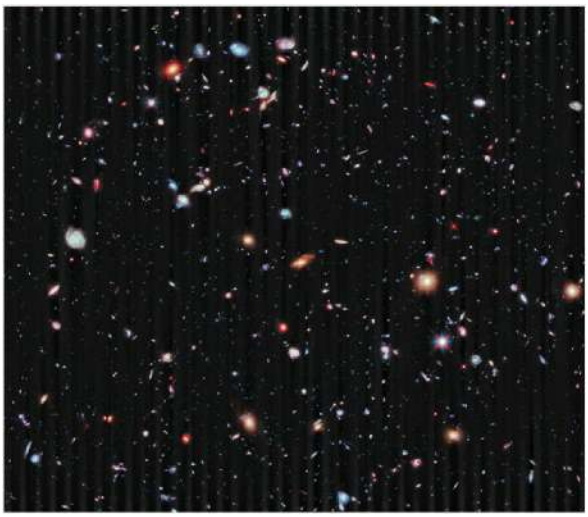
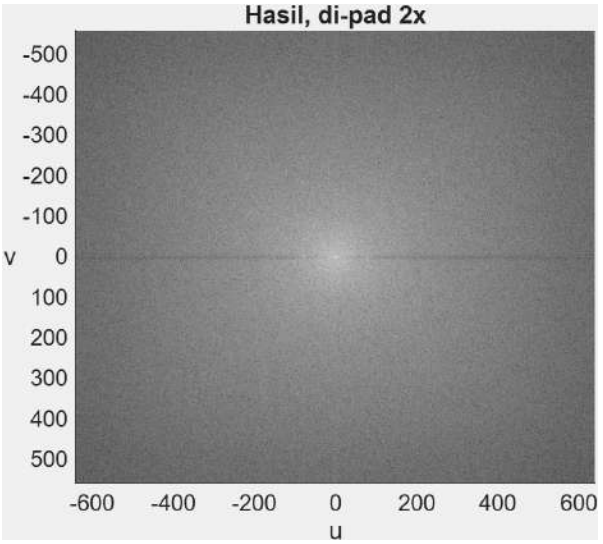
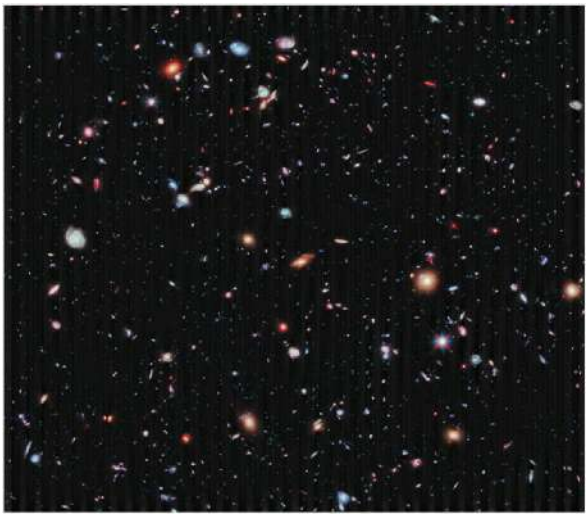
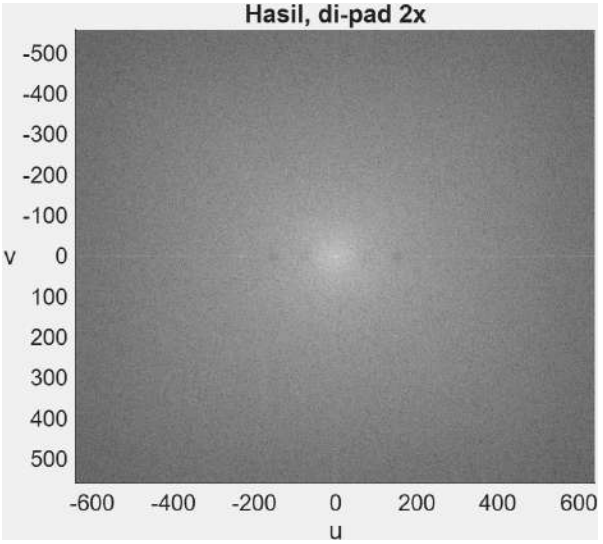
Tabel 4.20 Metode restorasi yang diuji untuk menghilangkan derau periodik

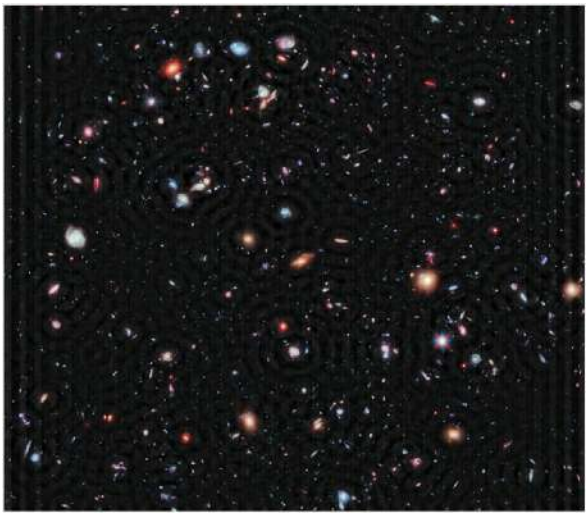
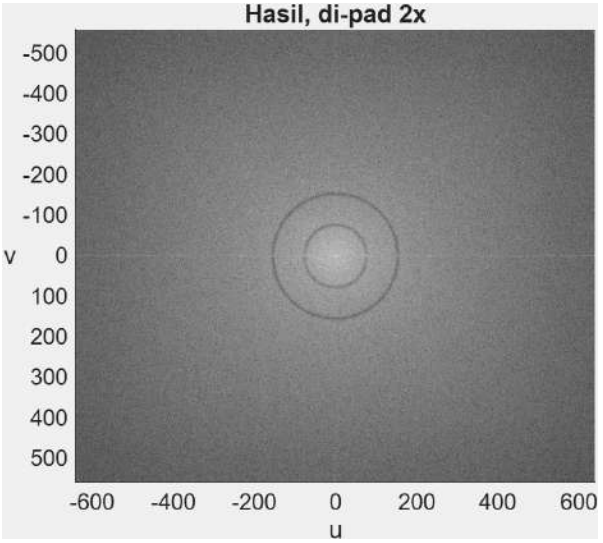
Kode Metode	**Nama Metode**	**Parameter**	**Mask Filter** $H(u, v)$
P4.1	Notch Pass Horizontal Line	Pos (position) – Garis Pos = 0 Lebar = 6 Blur garis = 0.9 –Lingkaran pemutus Pos = 0 Radius = 32 Blur = 0.7	

P4.2	Notch Filter Butterworth Reject	Butterworth Reject. Tiap butterworth: - Diameter = 20 - Orde = 1 Posisi (u,v) - (-77, 0) - (77, 0) - (-154, 0) - (154, 0)	 *Lingkaran merah: hanya penanda dari GUI di mana noise peaks berada
P4.3	Notch Filter BandReject Butterworth	Bandreject Butterworth. Tiap bandreject: - Posisi = (0,0) - WIdth = 10 Diameter(D) - 154 - 308	

4.4.3 Analisis Hasil Restorasi

Tabel 4.21 Analisis hasil restorasi citra dari derau periodik

Kode Metode Rujukan	Citra Hasil Restorasi	Spektrum Frekuensi Akhir
P4.1		
P4.2		

P4.3		

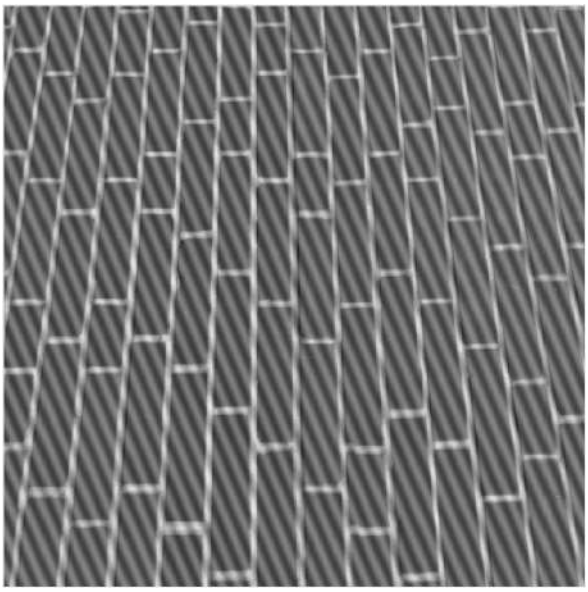
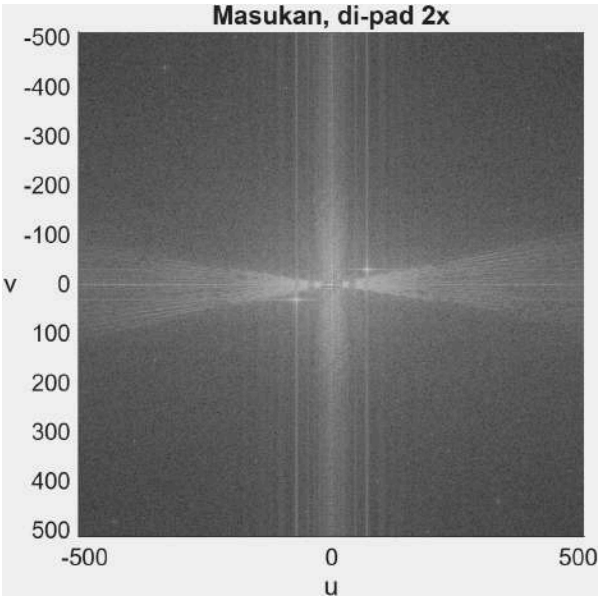
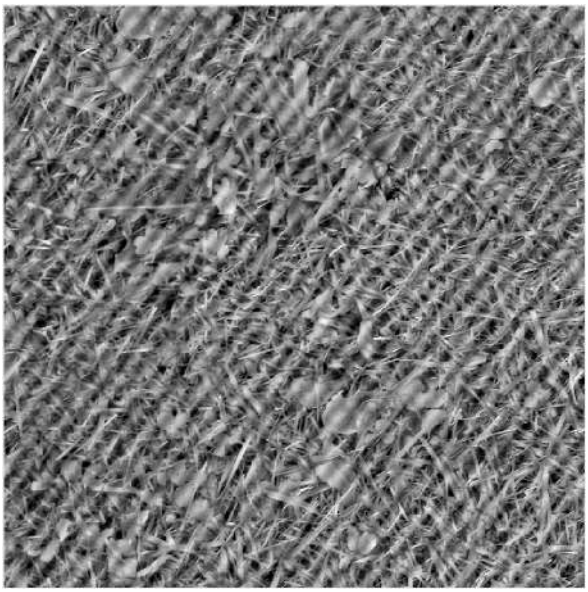
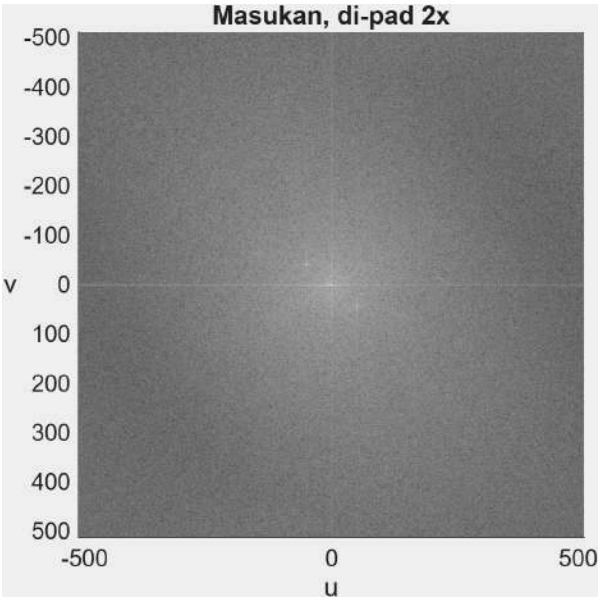
Evaluasi Kualitas Citra Terhadap Referensi		
Kode Metode Rujukan	**PSNR (dB)**	**MSE**
P4.1	30.65	55.93
P4.2	31.79	43.10
P4.3	29.51	72.73

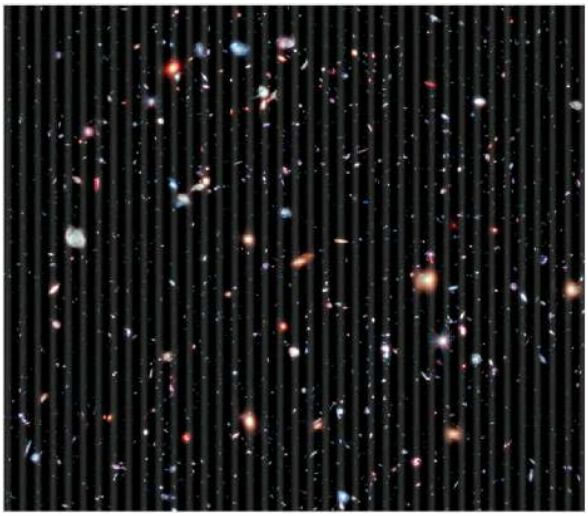
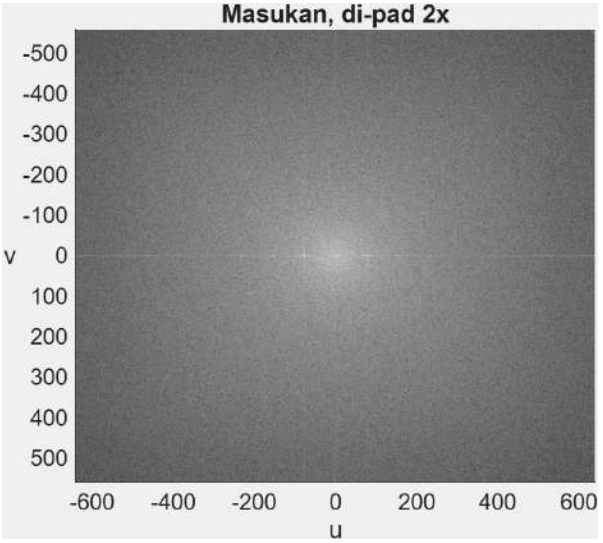
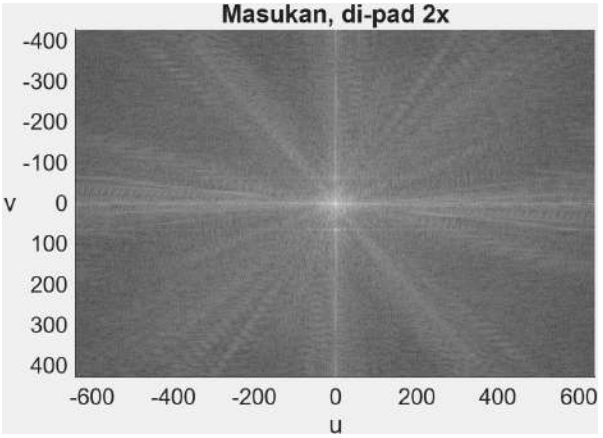
Notch reject pada empat titik (P4.2) paling selektif karena hanya meredam area kecil di sekitar peak sehingga frekuensi citra lainnya terjaga. Pada P4.1, garis vertikal berkurang terutama di bagian tengah citra, tetapi masih cukup tebal di dekat tepi. Pada P4.3 muncul ringing berupa pola gelombang yang mengelilingi objek terang, karena cincin band-reject membuang seluruh frekuensi pada jari-jari tersebut dari segala arah, bukan hanya pada sumbu horizontal.

Pada ketiganya derau hanya menipis, tidak hilang total, karena energinya tidak seluruhnya berada tepat pada peak yang diredam. Memperbesar notch atau pita menambah reduksi derau tetapi ikut membuang detail citra.

4.4.4 Rekapitulasi Restorasi Keseluruhan Citra Uji

Tabel 4.22 Analisis awal keseluruhan citra terdegradasi derau periodik

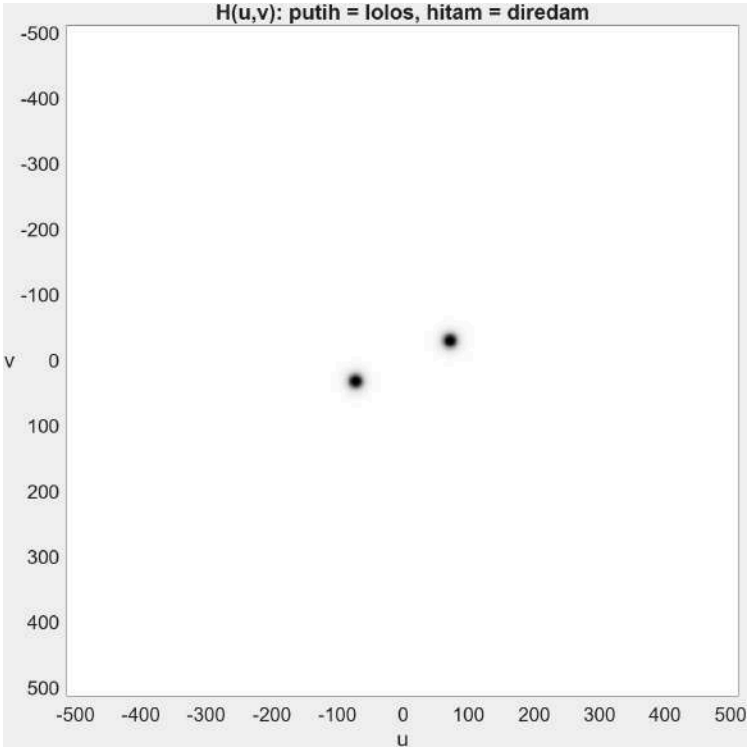
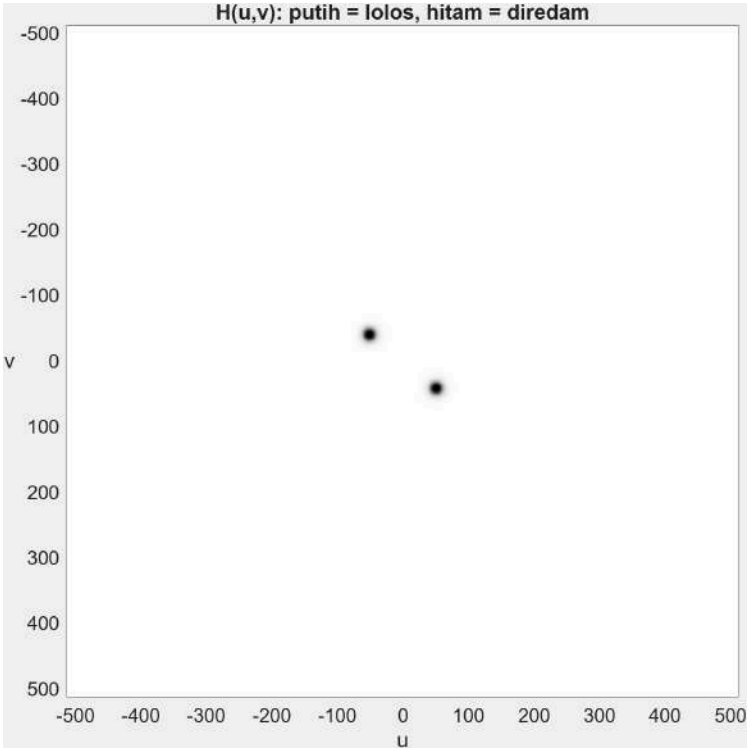
Kode Input	Citra Terdegradasi	Spektrum Frekuensi Awal
C4.1		
C4.2		

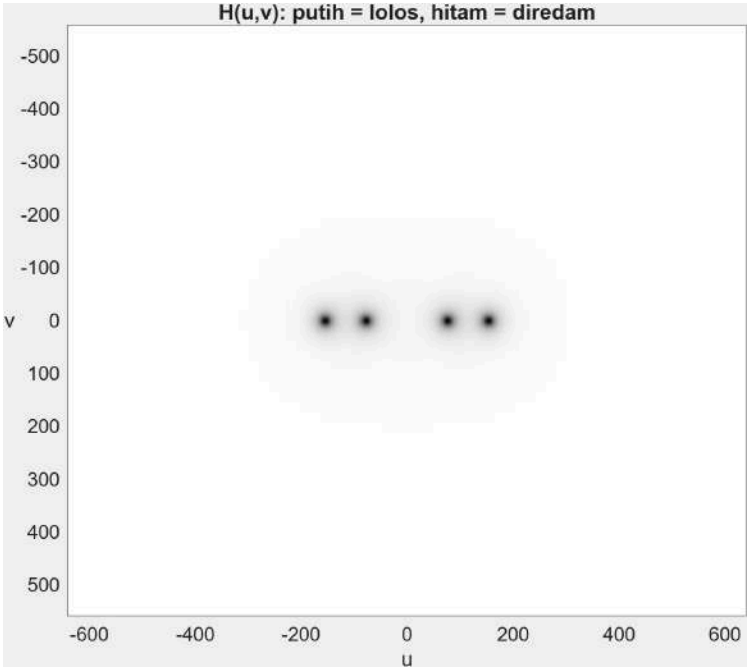
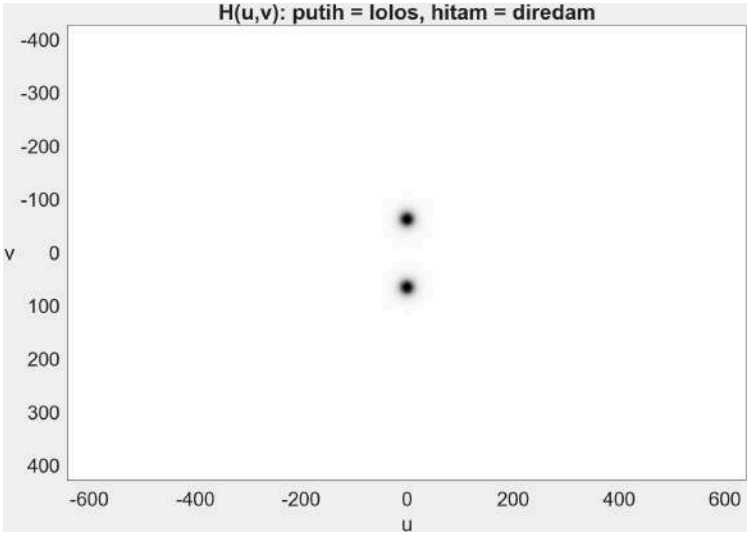
C4.3		
C4.4		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input	**PSNR (dB)**	**MSE**
C4.1	21.05	510.69
C4.2	20.06	640.95
C4.3	22.39	375.00
C4.4	21.73	436.89

Tabel 4.23 Konfigurasi restorasi derau periodik setiap kasus uji

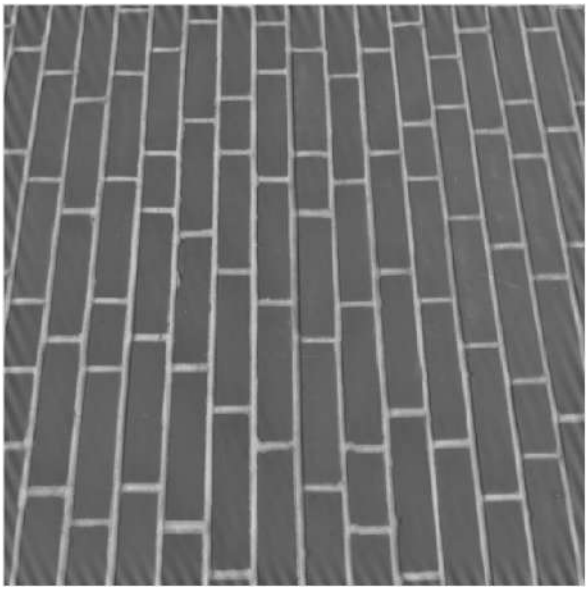
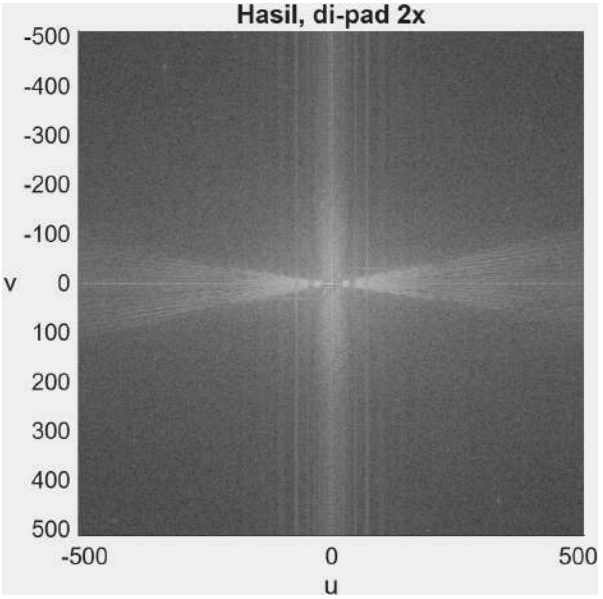
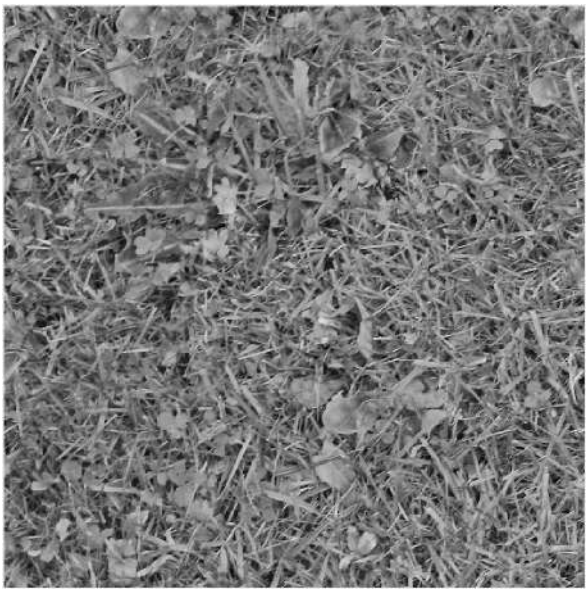
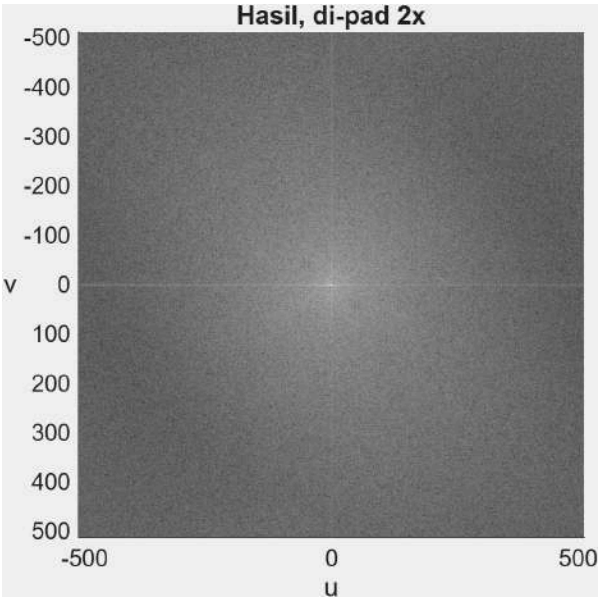
Kode	Nama Metode	Parameter	Mask Filter $H(u, v)$

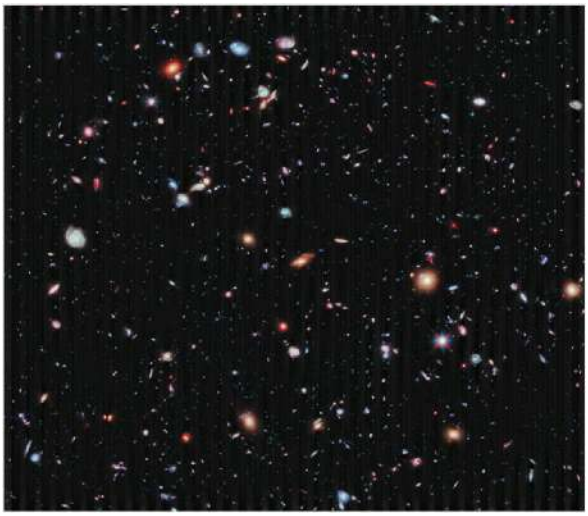
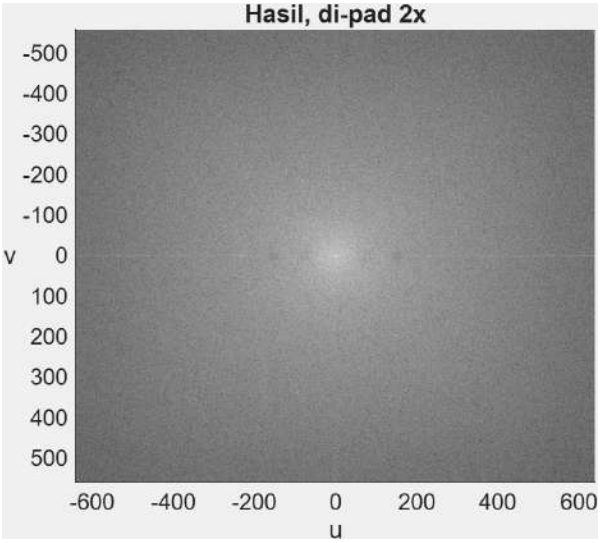
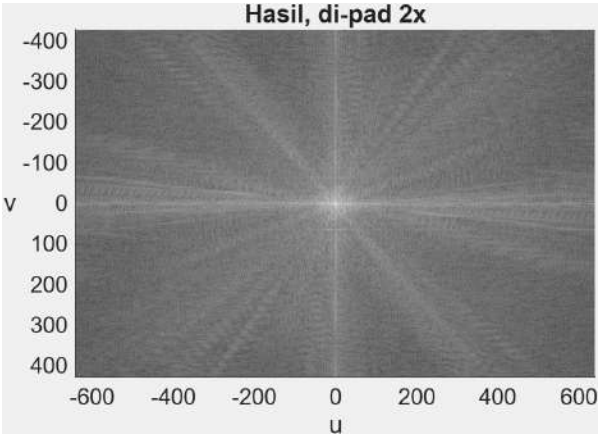
Input Rujukan			
C4.1	Notch Filter Reject Butterworth	Butterworth Reject. Tiap butterworth: - Diameter = 20 - Orde = 2 Posisi (u,v) - (-72, 31) - (72, -31)	
C4.2	Notch Filter Reject Butterworth	Butterworth Reject. Tiap butterworth: - Diameter = 19 - Orde = 2.1 Posisi (u,v) - (-51, -41) - (51, 41)	

C4.3	Notch Filter Reject Butterworth	Butterworth Reject. Tiap butterworth: - Diameter = 20 - Orde = 1 Posisi (u,v) - (-77, 0) - (77, 0) - (-154, 0) - (154, 0)	
C4.4	Notch Filter Reject Butterworth	Butterworth Reject. Tiap butterworth: - Diameter = 18 - Orde = 1.8 Posisi (u,v) - (0, -64) - (0, 64)	

Tabel 4.24 Analisis hasil restorasi derau periodik setiap kasus uji

Kode Input Rujukan	Citra Hasil Restorasi	Spektrum Frekuensi Akhir

C4.1		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u
C4.2		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u

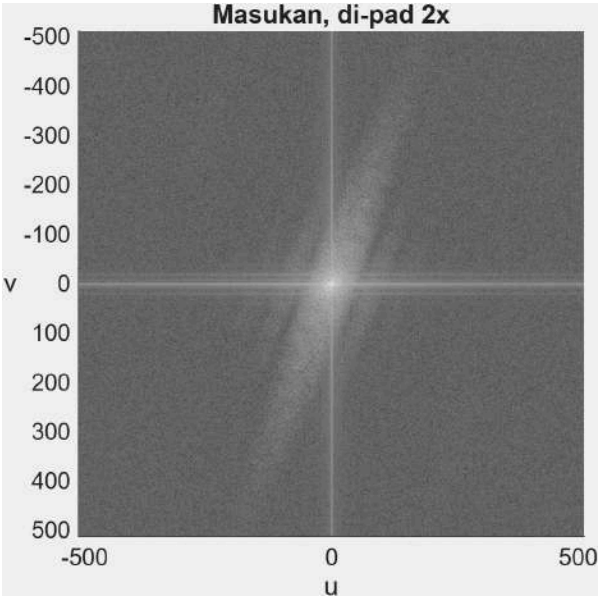
C4.3		
C4.4		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input Rujukan	**PSNR (dB)**	**MSE**
C4.1	38.23	9.77
C4.2	35.24	19.47
C4.3	31.79	43.10
C4.4	36.60	14.23

4.5 Pengujian Restorasi Motion Blur

4.5.1 Identifikasi Citra Terdegradasi

Tabel 4.25 Analisis awal citra terdegradasi `astronaut_color_motion_L19_A20_gaussian_noise.png`

Citra Terdegradasi	Spektrum Frekuensi Awal
	

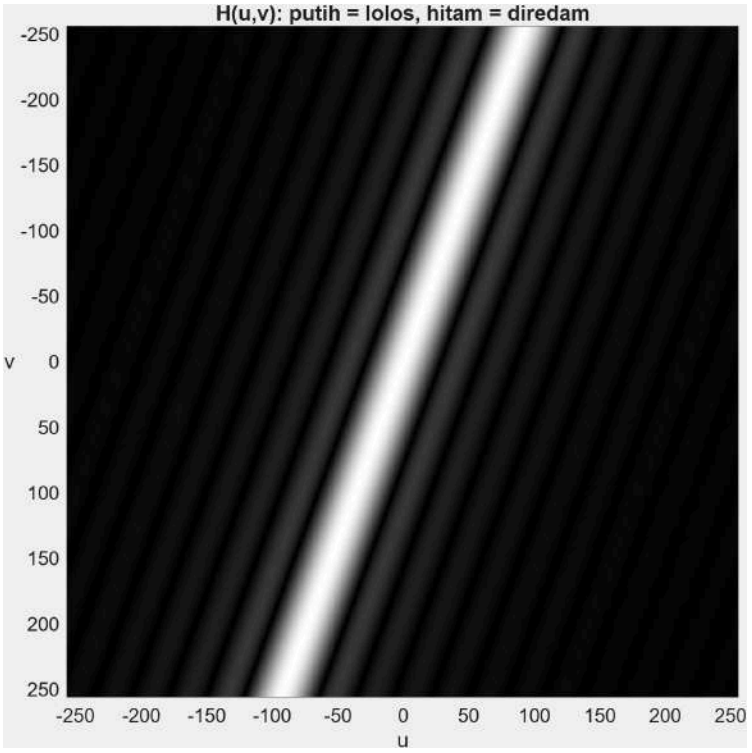
Evaluasi Kualitas Citra Terhadap Referensi	
PSNR (dB)	**MSE**
20.61	564.53

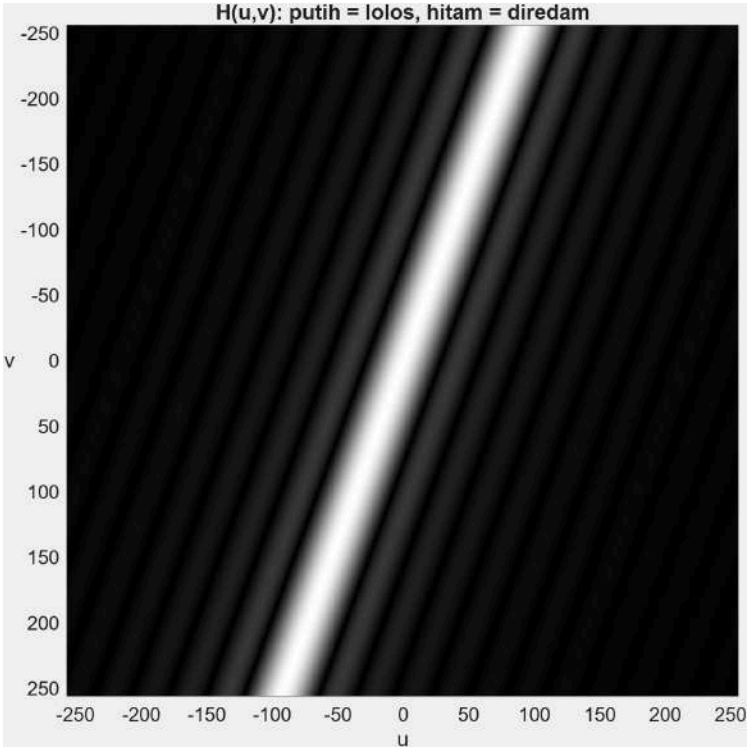
Citra terdegradasi `astronaut_color_motion_L19_A20_gaussian_noise.png` (Tabel 4.25) mengalami motion blur sepanjang `L` = 19 piksel pada sudut `θ` = 20° dan ditambah derau Gaussian. Motion blur menimbulkan garis-garis gelap (nol fungsi sinc) pada spektrum, sedangkan derau Gaussian menambah komponen acak pada seluruh frekuensi. Tujuan pemrosesannya adalah merestorasi citra dengan Wiener filtering, yang memperhitungkan derau dan membandingkannya dengan inverse filtering.

4.5.2 Pemilihan Metode Restorasi

Berdasarkan permasalahan kualitas citra tersebut, dilakukan percobaan pemrosesan dengan metode sebagai berikut:

Tabel 4.26 Metode restorasi yang diuji untuk menghilangkan motion blur

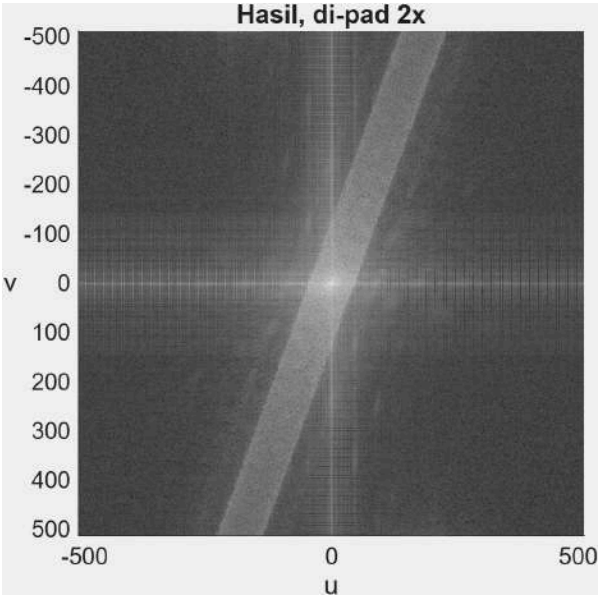
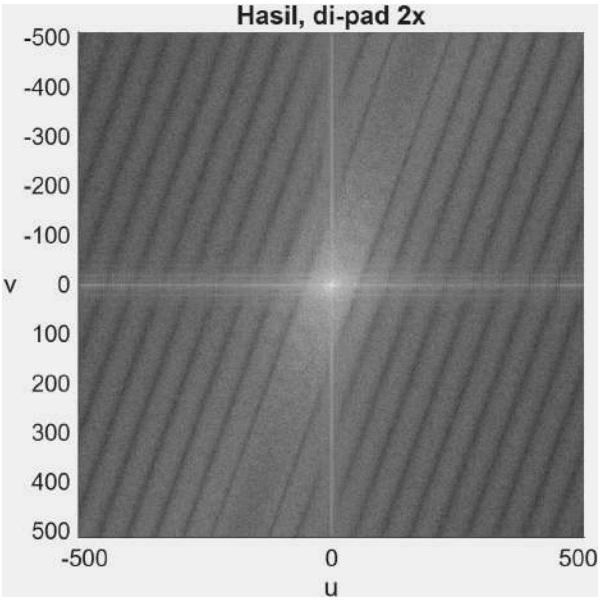
Kode Metode	**Nama Metode**	**Parameter**	**PSF** $H(u, v)$
P5.1	Inverse Filtering	L = 19 Tetha = 20 |H| = 0.235	

P5.2	Wiener Filtering	L = 19 Tetha = 20 K = 0.0448	

4.5.3 Analisis Hasil Restorasi

Tabel 4.27 Analisis hasil restorasi citra dari motion blur

Kode Metode Rujukan	Citra Hasil Restorasi	Spektrum Frekuensi Akhir

P5.1		
P5.2		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Metode Rujukan	**PSNR (dB)**	**MSE**
P5.1	21.23	489.43
P5.2	21.23	490.32

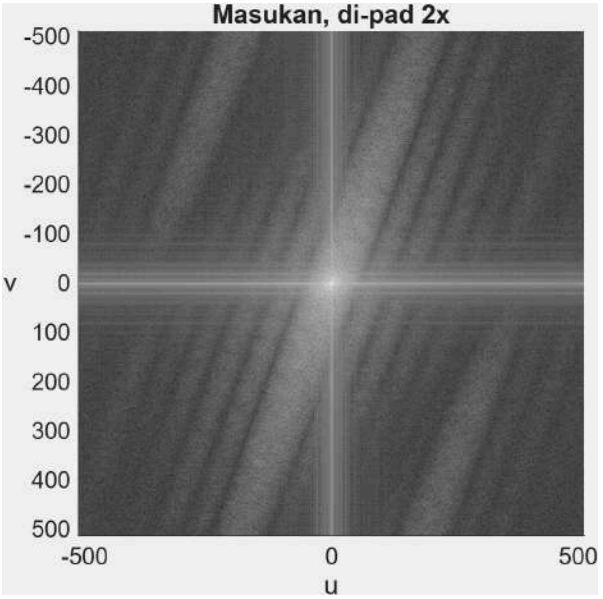
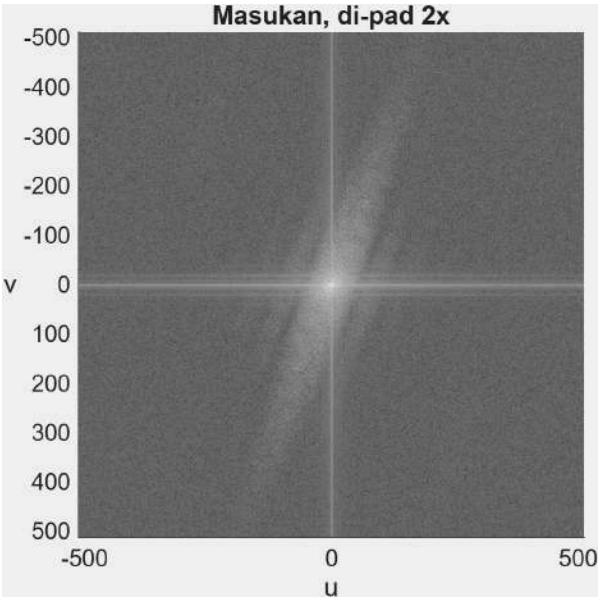
Secara angka kedua metode hampir identik dan MSE inverse sedikit lebih kecil, tetapi secara visual Wiener lebih baik (Tabel 4.27). Inverse filtering menyisakan lebih banyak pola bayangan berulang, sedangkan hasil Wiener memiliki bayangan lebih sedikit meski bayangan di sekitar tepi tampak lebih tajam. Perbedaan ini mengikuti model G = HF + N. Inverse filtering menghasilkan F + N/H sehingga derau diperkuat tanpa batas di sekitar nol H agar tidak meledak, threshold harus tinggi (0,235, jauh di atas nilai awal 0,01), yang berarti seluruh frekuensi dengan |H| di bawah itu dibuang. Wiener memakai H*/(|H|² + K) sehingga penguatan mengecil mulus tanpa membuang frekuensi secara tiba-tiba.

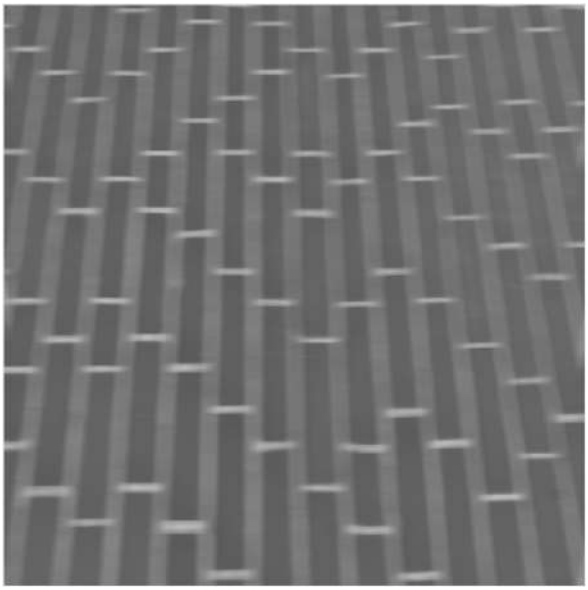
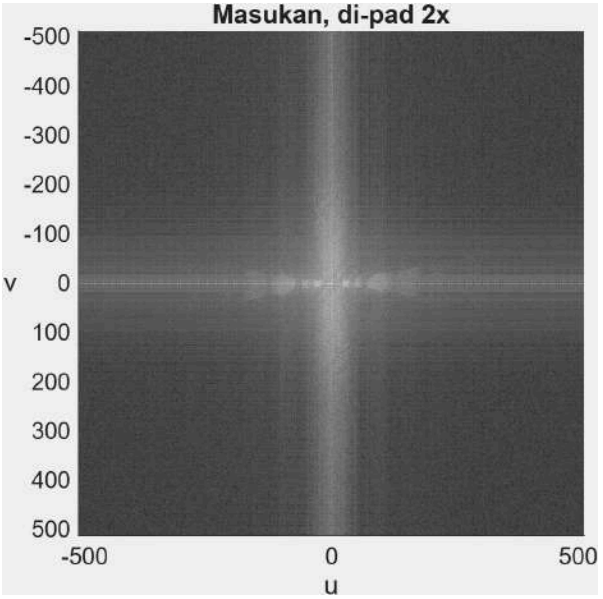
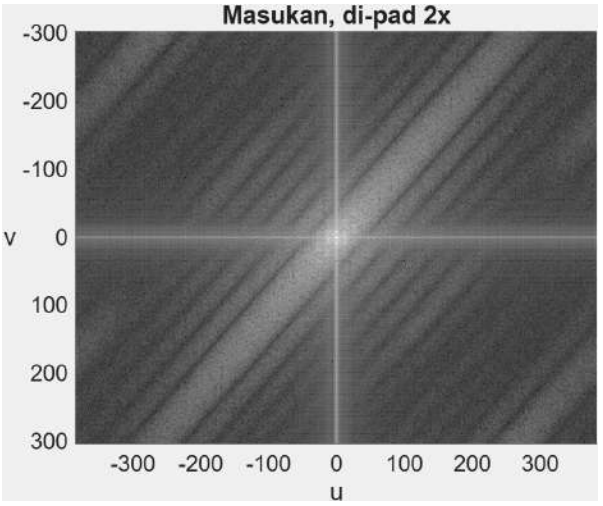
Keterbatasannya, restorasi dilakukan tanpa padding sehingga efek pembungkusan di tepi citra dapat menimbulkan bayangan, PSF diasumsikan sinc kontinu tanpa fase, dan K konstan mengasumsikan derau dan sinyal berspektrum rata. MSE juga tidak sepenuhnya mewakili persepsi visual, sehingga metode dengan MSE sedikit lebih tinggi dapat tampak lebih baik.

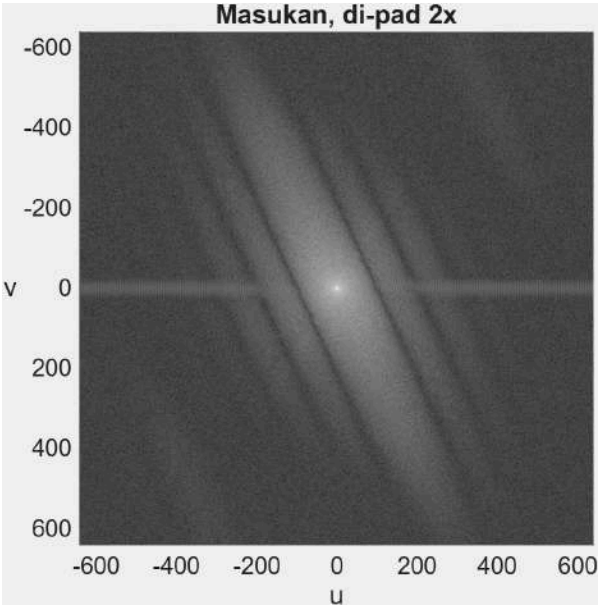
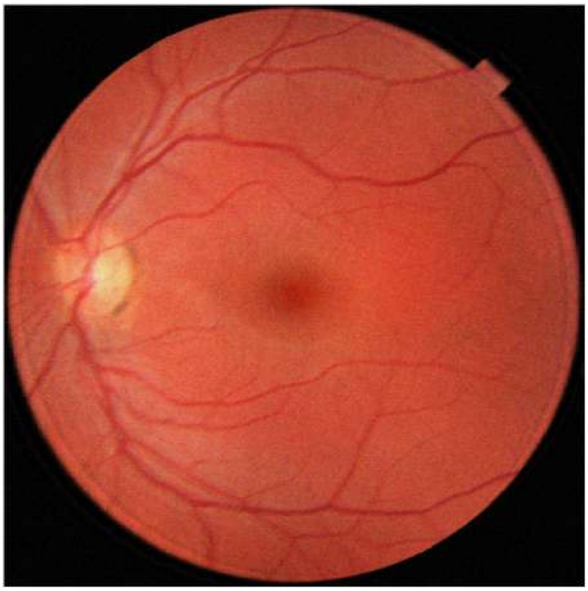
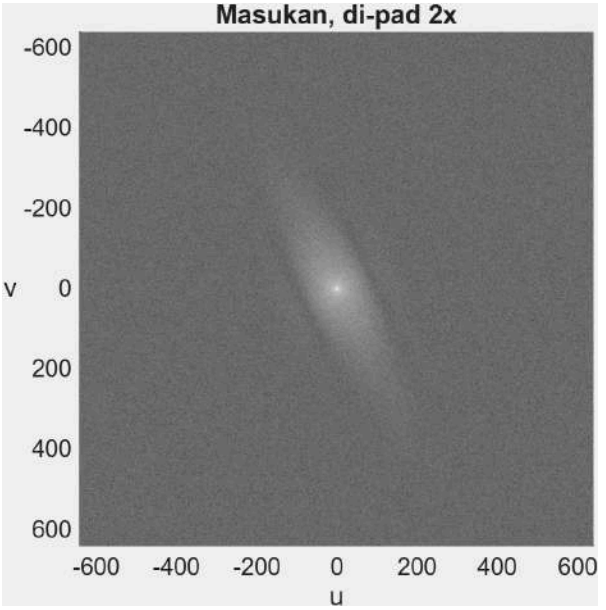
4.5.4 Rekapitulasi Restorasi Keseluruhan Citra Uji

Tabel 4.28 Analisis awal keseluruhan citra terdegradasi motion blur

Kode Input	Citra Terdegradasi	Spektrum Frekuensi Awal

C5.1		 Masukan, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u
C5.2		 Masukan, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u

C5.3		 Masukan, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u
C5.4		 Masukan, di-pad 2x -300 -200 -100 v 0 100 200 300 -300 -200 -100 0 100 200 300 u

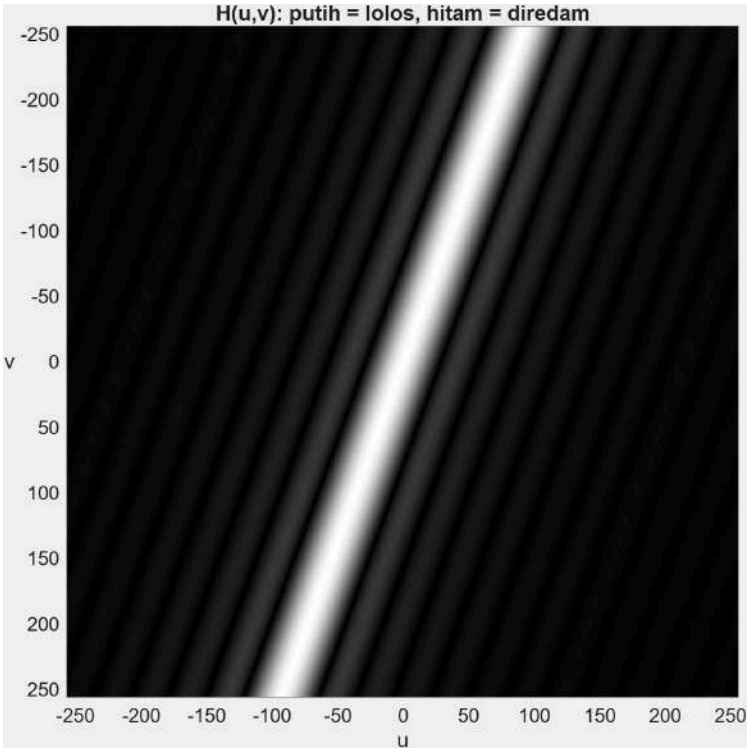
C5.5		
C5.6		

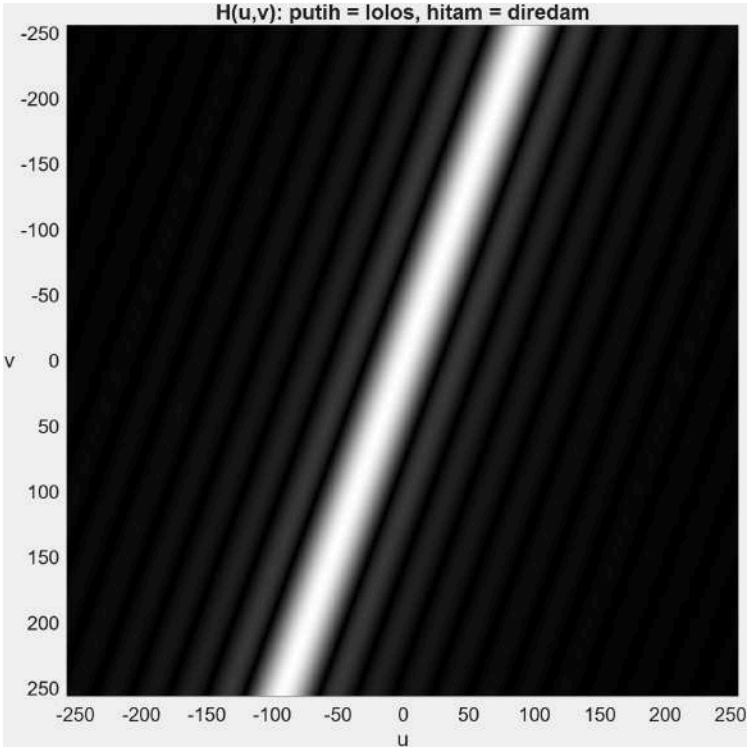
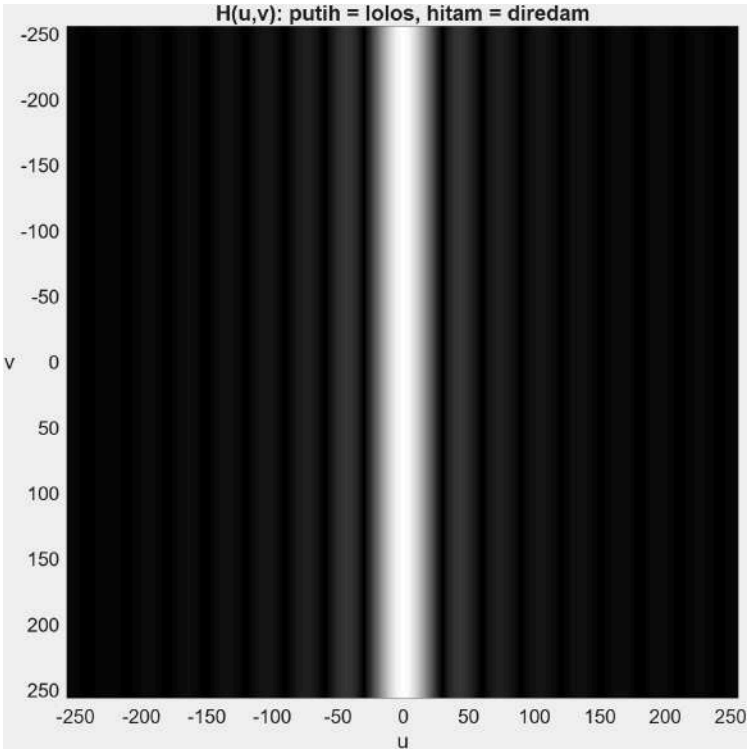
Evaluasi Kualitas Citra Terhadap Referensi

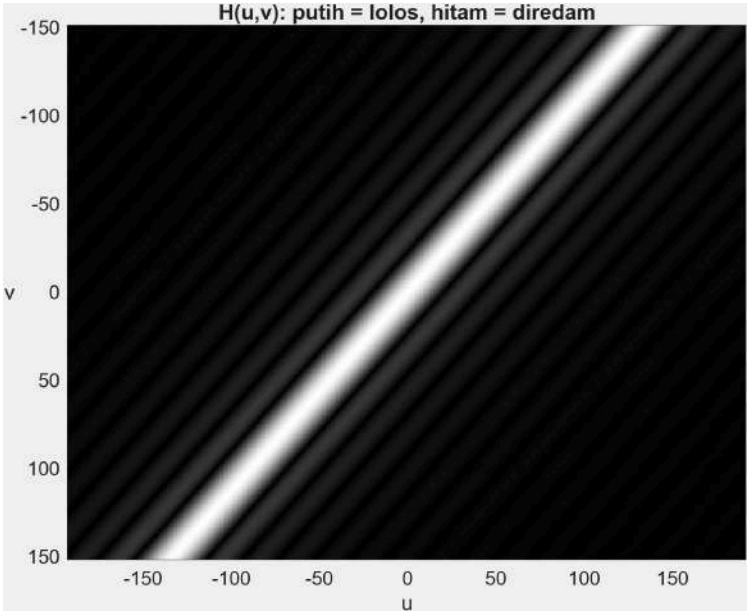
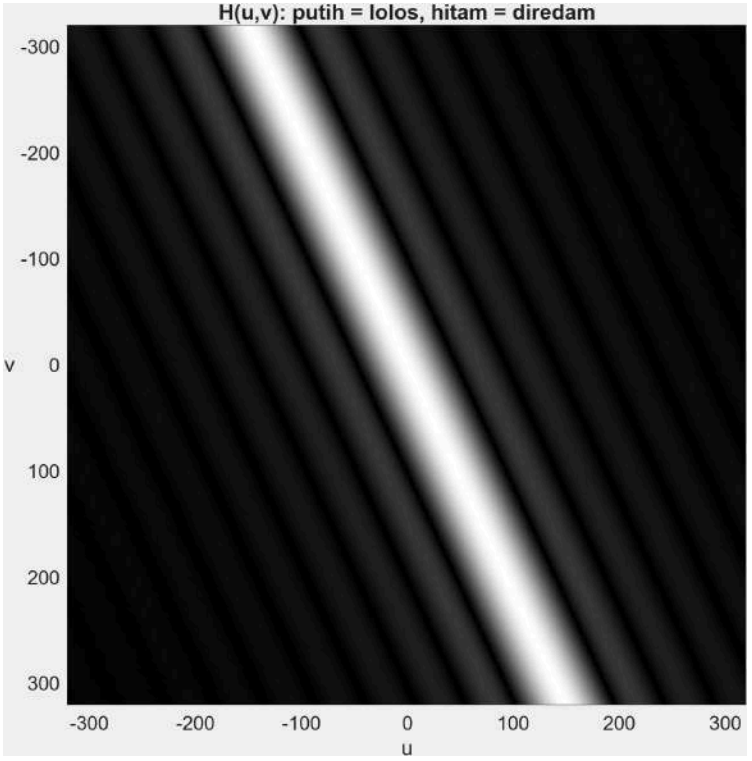
Kode Input	PSNR (dB)	MSE
C5.1	20.99	517.96
C5.2	20.61	564.53
C5.3	22.09	402.00

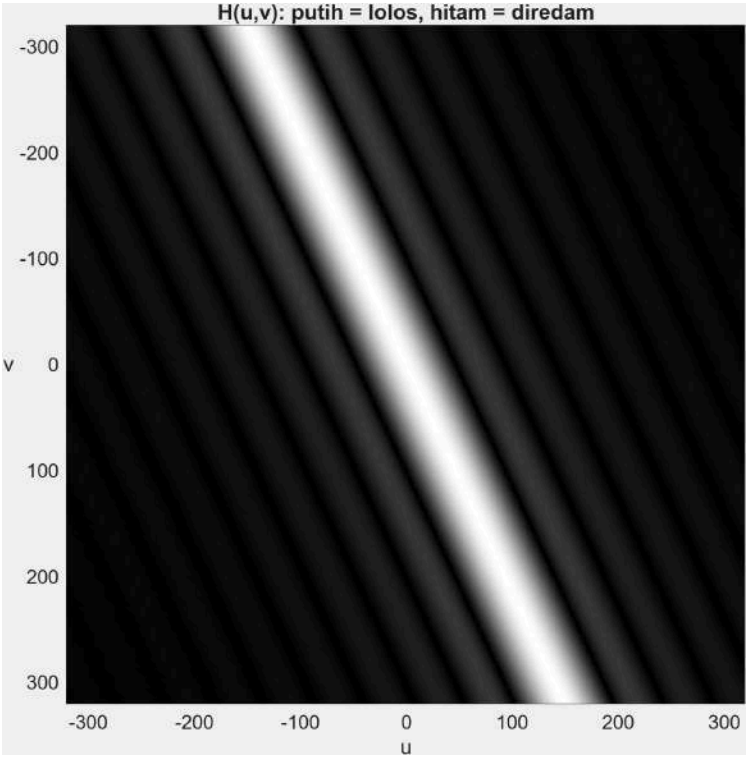
C5.4	20.35	599.48
C5.5	31.54	45.65
C5.6	28.64	88.92

Tabel 4.29 Konfigurasi restorasi motion blur setiap kasus uji

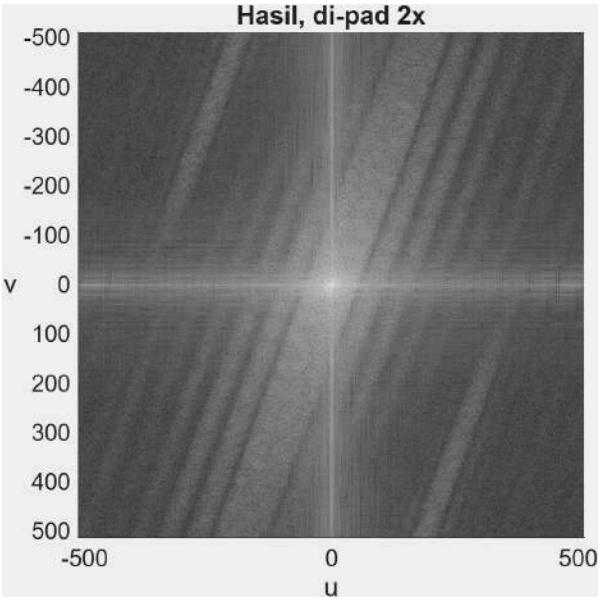
Kode Input Rujukan	**Nama Metode**	**Parameter**	**PSF** $H(u, v)$
C5.1	Wiener Filtering	L = 19 Tetha = 20 K = 0.0282	

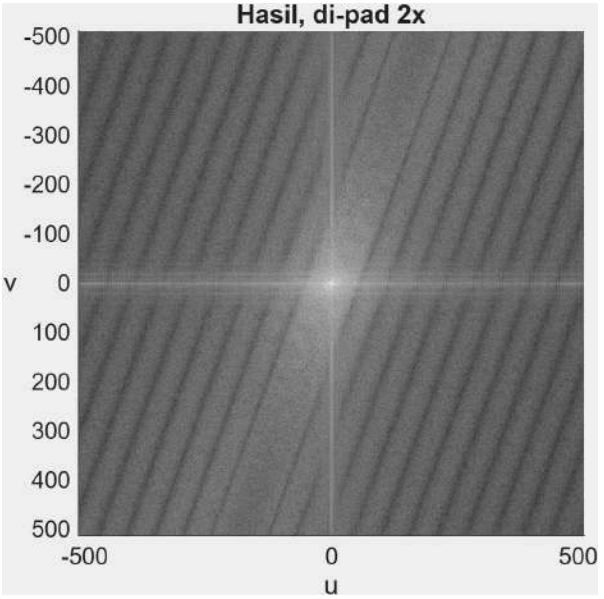
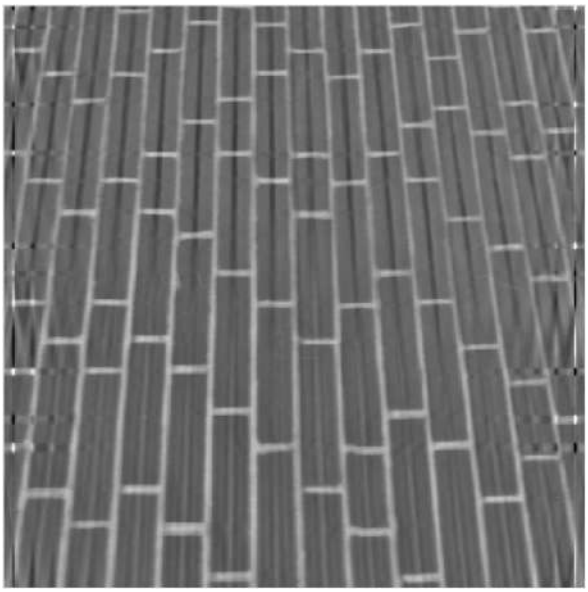
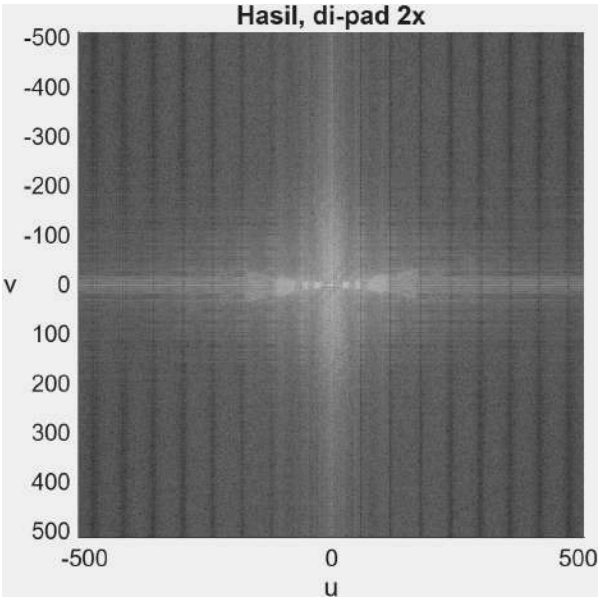
C5.2	Wiener Filtering	L = 19 Tetha = 20 K = 0.0448	
C5.3	Wiener Filtering	L = 17 Tetha = 0 K = 0.00611	

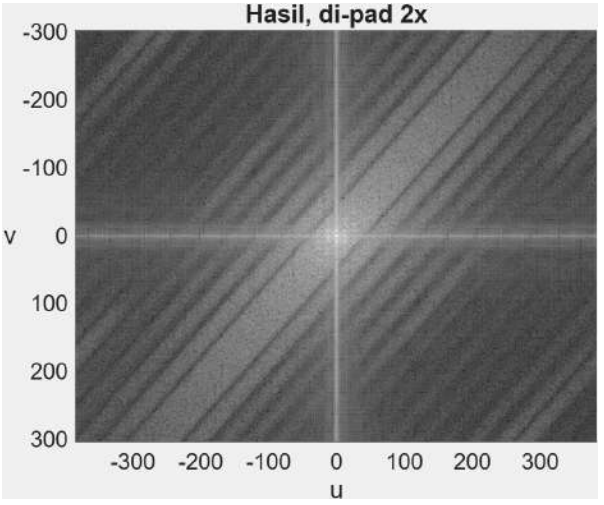
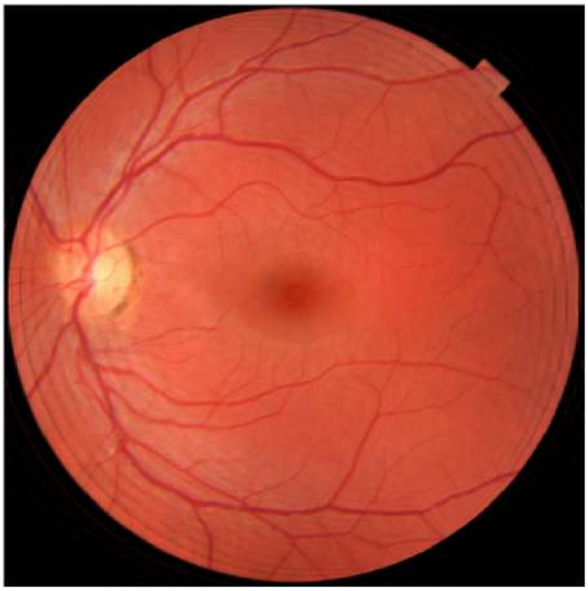
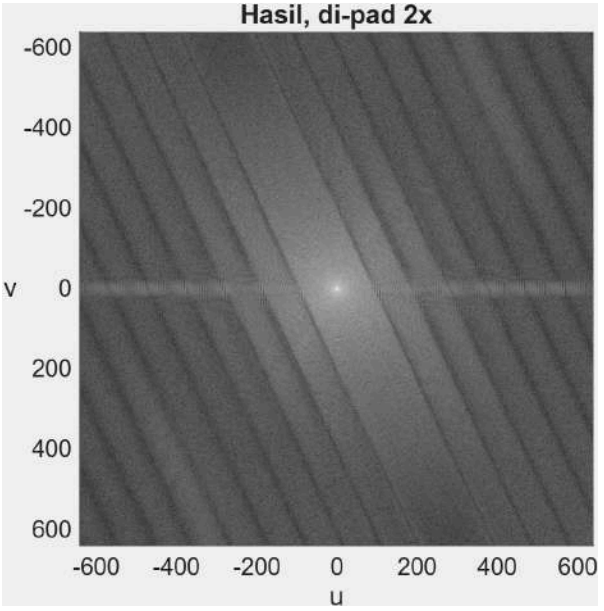
C5.4	Wiener Filtering	L = 23 Tetha = 35 K = 0.0244	
C5.5	Wiener Filtering	L = 15 Tetha = -25 K = 0.00557	

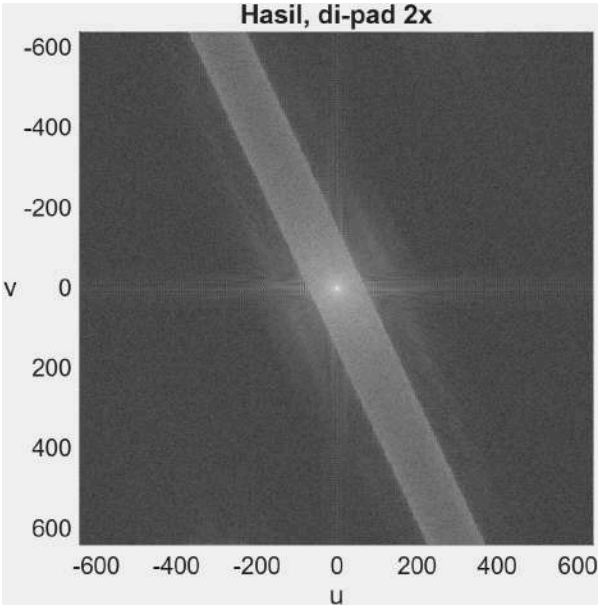
C5.6	Inverse Filtering (pada kasus citra uji ini, PSNR hasil inverse filtering memberikan nilai yang lebih kecil dibanding Wiener sehingga Inverse Filtering dipilih dalam restorasi citra khusus kasus uji ini)	L = 15 Tetha = -25 |H|= 0.291	

Tabel 4.30 Analisis hasil restorasi motion blur setiap kasus uji

Kode Input Rujukan	Citra Hasil Restorasi	Spektrum Frekuensi Akhir
C5.1		

C5.2		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u
C5.3		 Hasil, di-pad 2x -500 -400 -300 -200 -100 v 0 100 200 300 400 500 -500 0 500 u

C5.4		 Hasil, di-pad 2x -300 -200 -100 v 0 100 200 300 -300 -200 -100 0 100 200 300 u
C5.5		 Hasil, di-pad 2x -600 -400 -200 v 0 200 400 600 -600 -400 -200 0 200 400 600 u

C5.6		

Evaluasi Kualitas Citra Terhadap Referensi		
Kode Input Rujukan	**PSNR (dB)**	**MSE**
C5.1	22.09	401.64
C5.2	21.23	490.32
C5.3	26.34	151.20
C5.4	21.82	427.25
C5.5	36.75	13.75
C5.6	31.18	49.56

BAB V
KESIMPULAN

Hasil implementasi dan pengujian menunjukkan bahwa metode yang tepat ditentukan oleh letak energi degradasi pada spektrum. Derau Gaussian tersebar merata pada seluruh frekuensi sehingga ditangani dengan low-pass filter, dengan harga hilangnya sebagian detail halus. Derau periodik terkonsentrasi pada peak tertentu sehingga paling baik dibuang secara selektif dengan notch pada peak tersebut, bukan dengan filter yang menyapu seluruh frekuensi. Motion blur berupa fungsi degradasi sinc yang nol-nolnya ditentukan oleh L dan θ, sehingga dapat dibalik, tetapi pembalikan langsung tidak stabil di sekitar nol dan terhadap derau, yang diatasi Wiener filtering dengan konstanta K. Kecerahan rata-rata berkaitan langsung dengan komponen DC sehingga peningkatan kecerahan cukup dilakukan dengan menskalakan spektrum. Seluruh metode bergantung pada pemilihan parameter; fitur Tune membantu menemukannya, tetapi memerlukan citra referensi.

Metode yang paling sesuai untuk setiap kasus uji:

- Image smoothing: BLPF (D = 550, n = 1,6) terbaik pada astronaut_color_gaussian_noise.png, PSNR naik dari 23,61 dB ke 28,34 dB, dibanding GLPF 28,08 dB dan ILPF 27,22 dB. Pada empat citra uji PSNR hasil BLPF berada pada 24,99–28,34 dB.
- High-pass filtering: IHPF, GHPF, dan BHPF pada diameter yang sama menghasilkan tepi yang serupa pada citra, dengan perbedaan yang lebih jelas pada spektrum. Gaussian dan Butterworth lebih dipilih secara teori karena transisinya tidak diskontinu sehingga lebih bebas ringing.
- Peningkatan kecerahan: penskalaan seluruh spektrum dengan satu α global dari komponen DC. Dengan target mean hasil Tune, PSNR naik dari 10,02 dB ke

30,42 dB pada retina_color_dark.png, dan dari 6,36–12,86 dB ke 27,29–30,42 dB pada keempat citra.

- Restorasi derau periodik: notch reject Butterworth pada titik peak terbaik (31,79 dB) dibanding notch pass garis (30,65 dB) dan band-reject (29,51 dB) pada hubble_color_periodic_noise.png; pada keempat citra PSNR menjadi 31,79–38,23 dB.
- Restorasi motion blur (bagian F): Wiener filtering lebih stabil terhadap derau dan secara visual lebih baik daripada inverse filtering pada citra berderau, sedangkan inverse filtering memerlukan threshold yang tinggi (0,235) dan menyisakan pola berulang; keduanya memiliki PSNR mencapai 21,23 dB pada astronaut_color_motion_L19_A20_gaussian_noise.png. Wiener pada empat citra tanpa derau tambahan memiliki PSNR mencapai 21,82–36,75 dB, dan inverse filtering yang digunakan pada retina_color_motion_L15_A-25_gaussian_noise.png memiliki PSNR mencapai 31.18.

DAFTAR PUSTAKA

[1] Khanzadah, Anam dan Noshin Sabuwala. (2025). *"Exploring Digital Image Processing: A Study on Frequency Domain Filtering Techniques"*. ResearchGate. Tersedia online di: https://www.researchgate.net/publication/391051126_Exploring_Digital_Image_Processing_A_Study_on_Frequency_Domain_Filtering_Techniques [Diakses pada 11 Oktober 2026].

[2] Gonzalez, Rafael C. dan Richard E. Woods. (2018). "Digital Image Processing". Edisi ke-4. Pearson.

[3] Munir, Rinaldi. (2025). "Slide Kuliah IF4073 Pemrosesan Citra Digital: 11 – Transformasi Citra". Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

[4] Munir, Rinaldi. (2026). "Slide Kuliah IF4073 Pemrosesan Citra Digital: 12 – Penapisan Citra dalam Ranah Frekuensi". Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

[5] Munir, Rinaldi. (2025). "Slide Kuliah IF4073 Pemrosesan Citra Digital: 14 – Restorasi Citra (Bagian 2)". Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

LAMPIRAN

1. Pranala repositori

   evanderruizhix/tugas2-citra-frequency-domain-processing

2. Pembagian tugas

NIM	Nama	Kontribusi
13523104	Muhammad Zahran Ramadhan Ardiana	- Merancang visual, tata letak, dan alur interaksi antarmuka grafis (GUI), serta mengimplementasikannya ke dalam `FrequencyDomainApp.m` dan komponen `gui/`. - Mengembangkan seluruh fungsi pembantu/utilitas (`utils/`), notch filters (`notchLineFilter.m`), dan pass filters (`butterworthPass.m`). - Mengintegrasikan seluruh modul algoritma pemrosesan ke dalam GUI, melakukan pengujian menyeluruh, serta menangani bug sistem. - Menyusun Bab III (Struktur Program dan Alur Penggunaan, dan serta Implementasi Fungsi Pembantu), Bab IV (Pengujian Program dan Analisis Hasil), dan Bab V (Kesimpulan) - Menyusun dokumentasi repositori (README.md).
18223064	Ahmad Evander Ruizhi Xavier	- Mengembangkan modul band filters (`bandIdeal.m`, `bandGaussian.m`, `bandButterworth.m`), pencerahan citra (`brightenFrequency.m`), pemodelan dan restorasi motion blur (`motionBlurPSF.m`, `inverseFilter.m`, `wienerFilter.m`), serta pass filters (`idealPass.m`, `gaussianPass.m`). - Menyusun keseluruhan Bab I (Pendahuluan) dan Bab II (Teori Singkat).

		- Menyusun Bab III khusus bagian penjelasan algoritma dan implementasi kode modul penapis utama (di luar fungsi pembantu/utilitas, notch filters, dan Butterworth pass).